

Advances in Designing Scalable GNNs: The Perspective of Graph Data Management

Ningyi Liao

Siqiang Luo

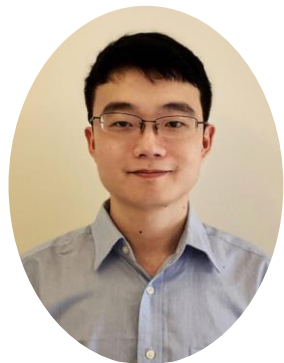
Xiaokui Xiao

Reynold Cheng



香港大學
THE UNIVERSITY OF HONG KONG

Tutorial Contributors / Presenters



Siqiang Luo

Assistant Professor, NTU

✉ siqiang.luo@ntu.edu.sg

🌐 <https://siqiangluo.com>

💡 Graph algorithms/systems, KV systems, ML data management



Ningyi Liao

Ph.D Candidate, NTU

✉ liao0090@e.ntu.edu.sg

🌐 <https://nyliao.github.io>

💡 ML graph algorithms, Graph neural network



Reynold Cheng

Professor, HKU

✉ ckcheng@cs.hku.hk

🌐 <https://www.reynold.hku.hk>

💡 Data science, Big graph analytics, Uncertain data management



Xiaokui Xiao

Professor, NUS

✉ xkxiao@nus.edu.sg

🌐 <https://www.comp.nus.edu.sg/~xiaoxk/>

💡 Privacy-preserving data management, Large data analysis

Outline



Introduction: Application | Challenge | Opportunity

Classical Approaches: Sampling | Partition | Decoupling

Hardware-aware Sampling

Subgraph Extraction

Outline

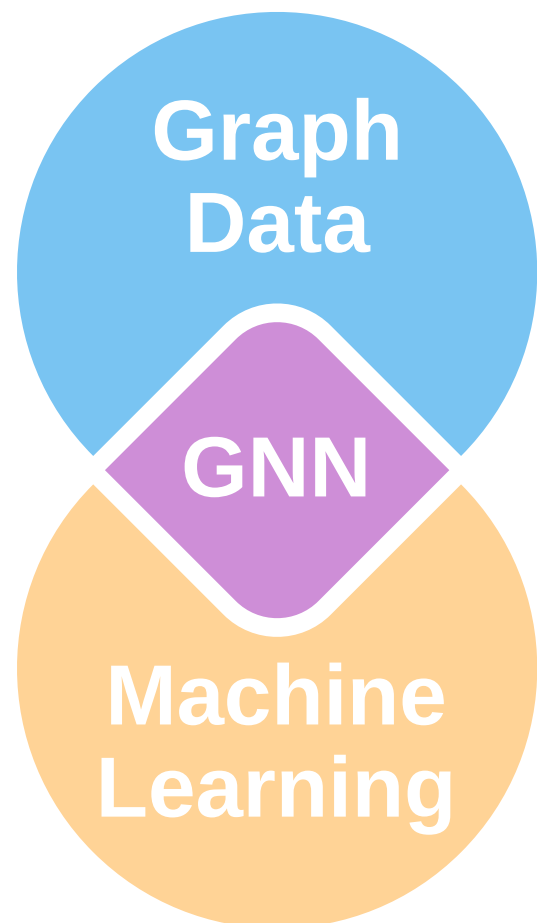
Node-wise Similarity

Spectral Embedding

Graph Simplification

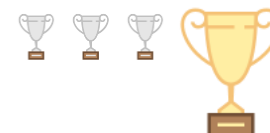
Future Direction: Graph+LM | Specific Data | Graph System

GNN: Current and Prospect

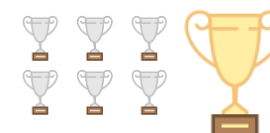


#1 WWW
2024

Ranking in keywords of accepted papers



#4 NeurIPS
2025



#7 ICLR
2024

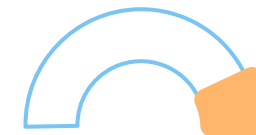


7% VLDB
2024

Percentage of topic in accepted papers



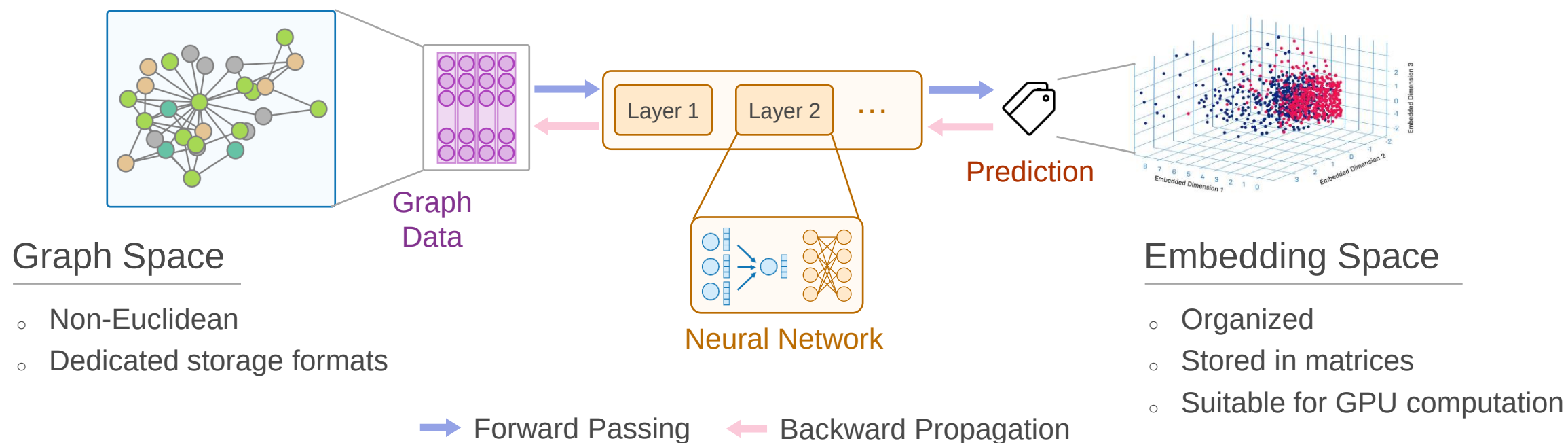
6% ICDE
2025



5% SIGMOD
2025

GNN: Learning on Graph Data

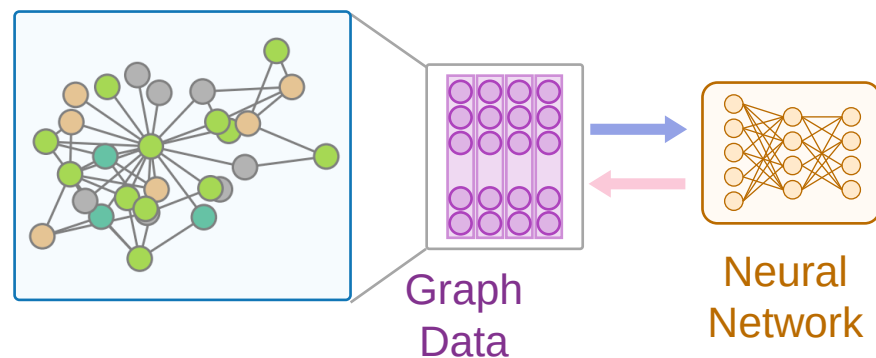
- Integrate **graph computation** and **neural network architecture**
- From semi-unstructured **graph data** to **structured embeddings**



GNN: Learning on Graph Data

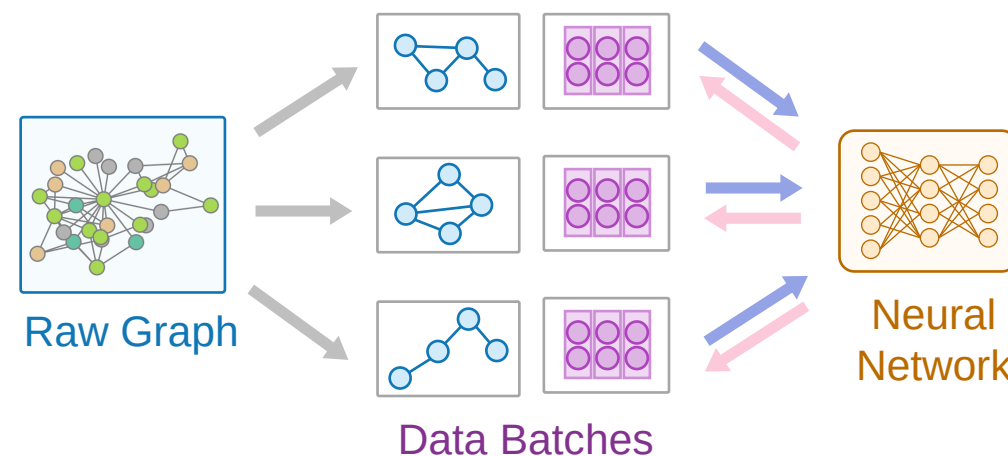
Full-batch (FB) Training

- NN learns graph data as a whole
- Preserve full graph information



Mini-batch (MB) Training

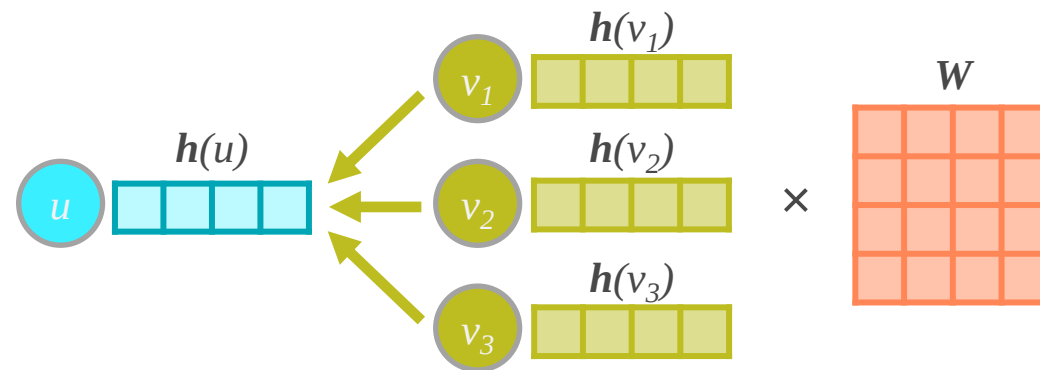
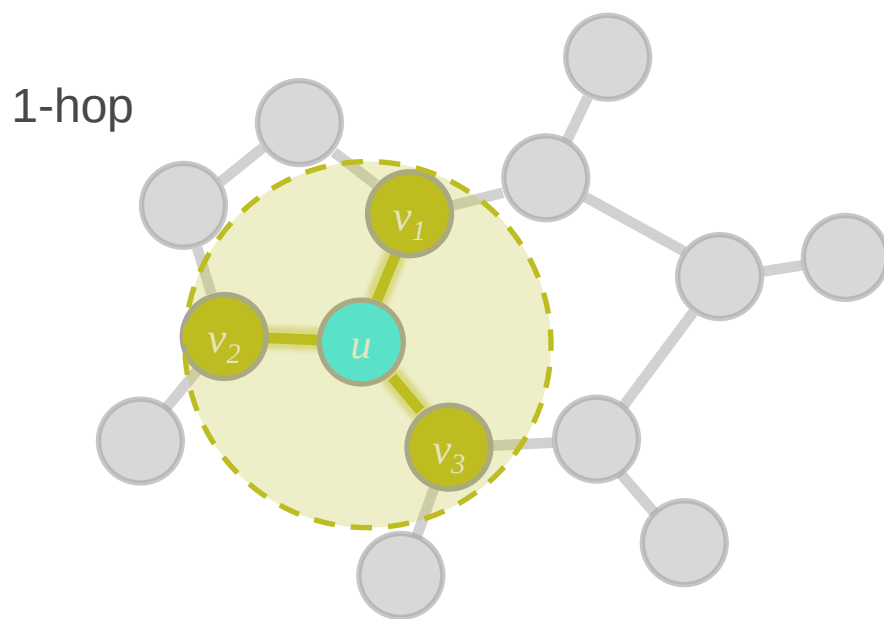
- NN learns part of graph each time
- Batch division affects performance



→ Forward Passing ← Backward Propagation

GNN: GCN [ICLR'17]

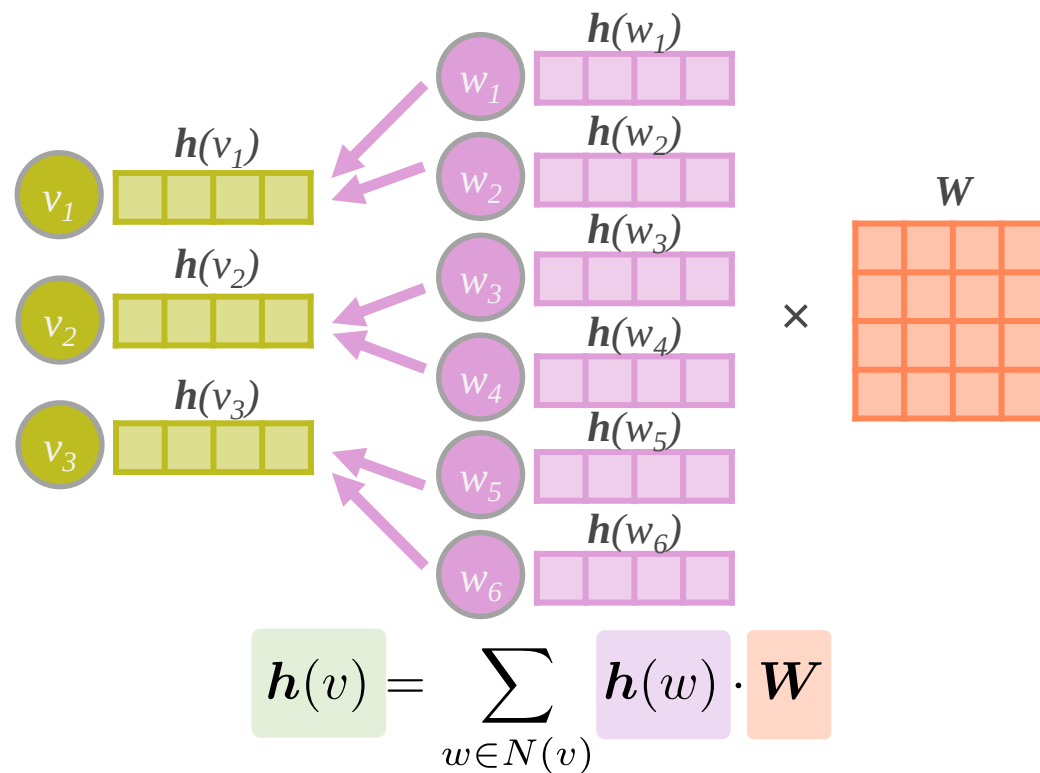
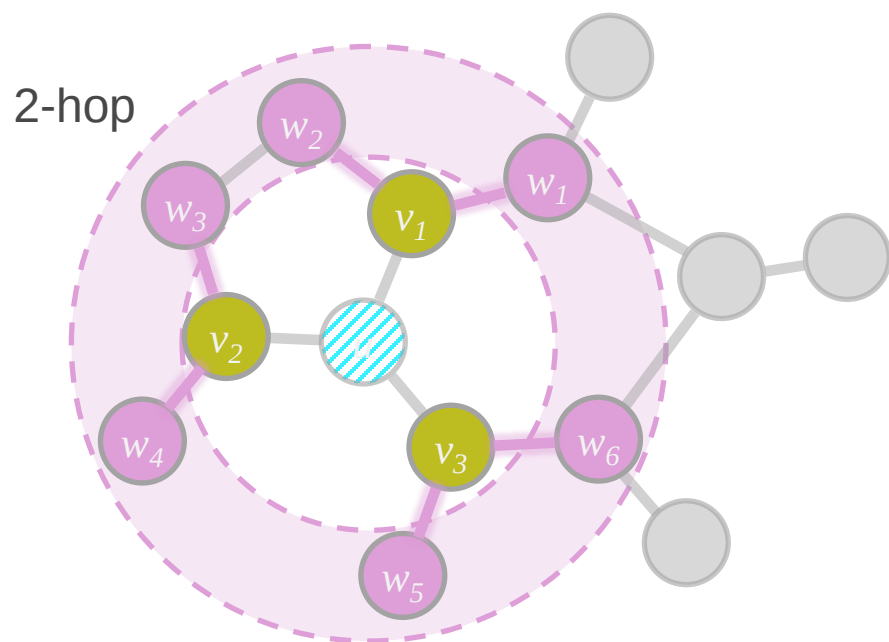
- Message Passing: aggregate neighbor information $N(u)$ by learnable weights W to each node as node representation h



$$h(u) = \sum_{v \in N(u)} h(v) \cdot W$$

GNN: GCN [ICLR'17]

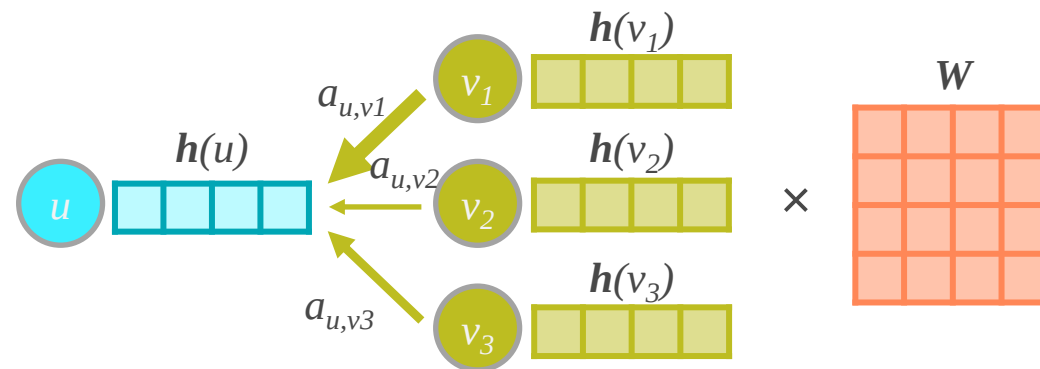
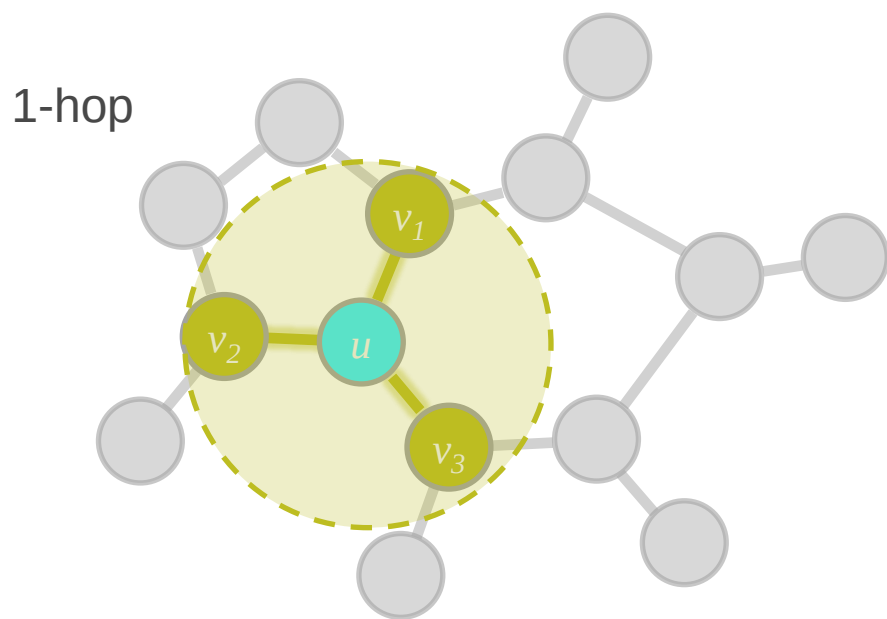
- GNN Layers: Stacking multi-hop graph convolutions



GNN: GAT [ICLR'18]

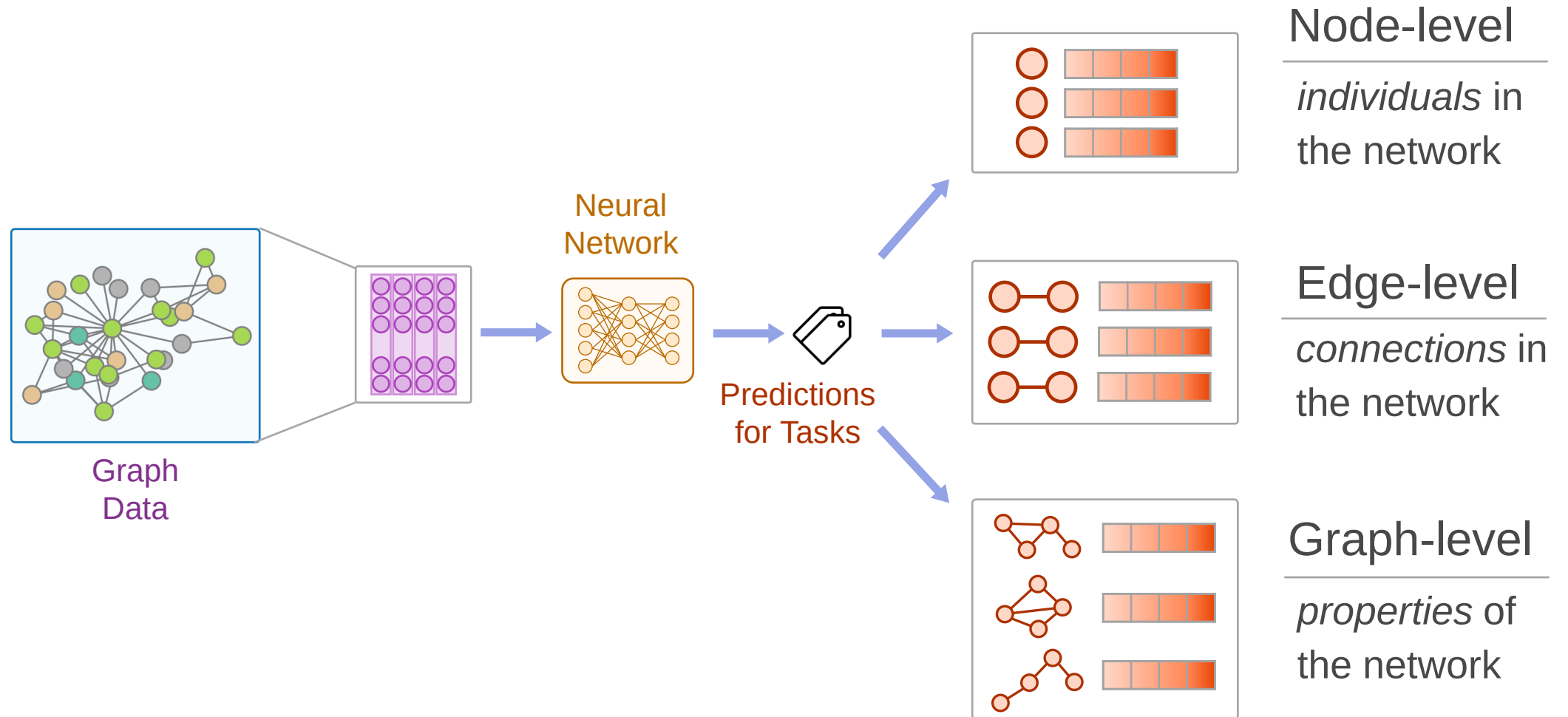
- Graph Attention: learnable weights for edge aggregation

$$a_{u,v} = \frac{\exp(\text{attention}(\mathbf{h}(u), \mathbf{h}(v)))}{\sum_{v \in N(u)} \exp(\text{attention}(\mathbf{h}(u), \mathbf{h}(v)))}$$



$$\mathbf{h}(u) = \sum_{v \in N(u)} a_{u,v} \cdot \mathbf{h}(v) \cdot \mathbf{W}$$

GNN: Tasks and Applications



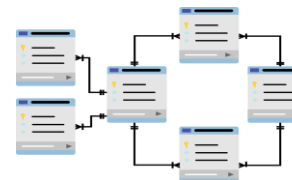
GNN: Tasks and Applications

Node-level

individuals in the network



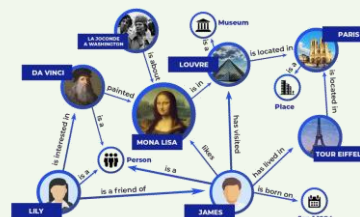
User Profiling
E-commerce



Text-to-SQL
DB Schema



Friend Suggestion
Social Network



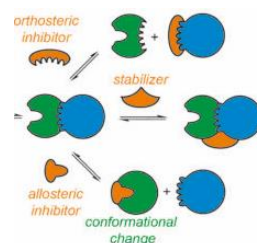
Knowledge Discovery
Knowledge Graph

Edge-level

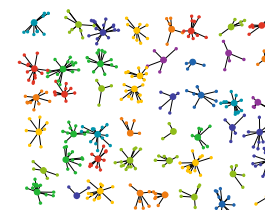
connections in the network

Graph-level

properties of the network



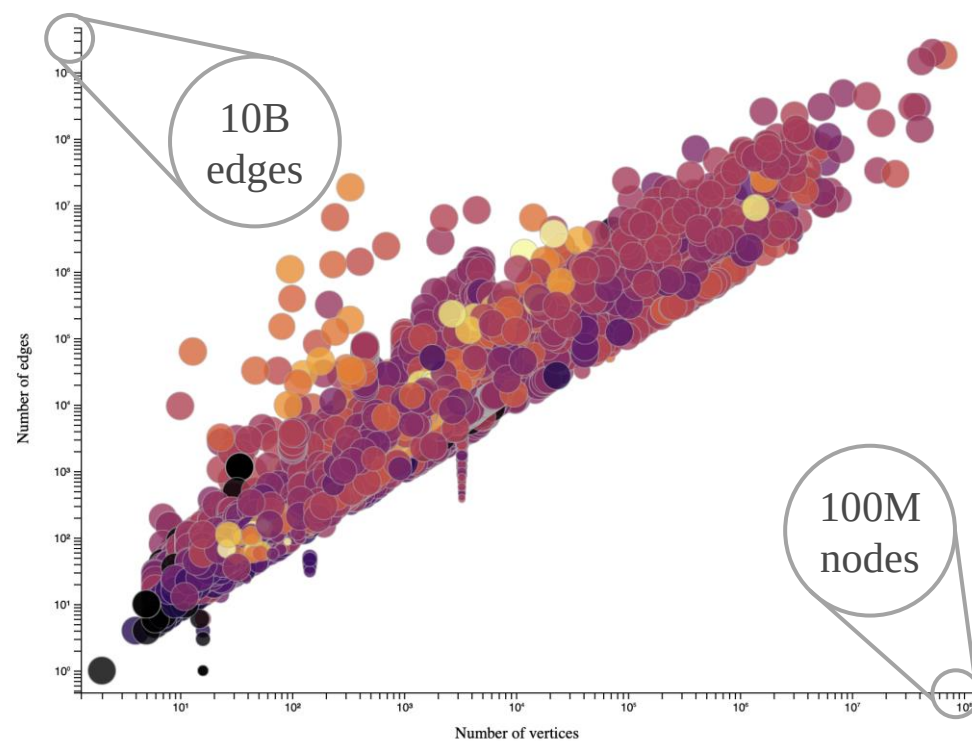
Drug Discovery
Protein Structure



Policy Optimization
Wireless Network

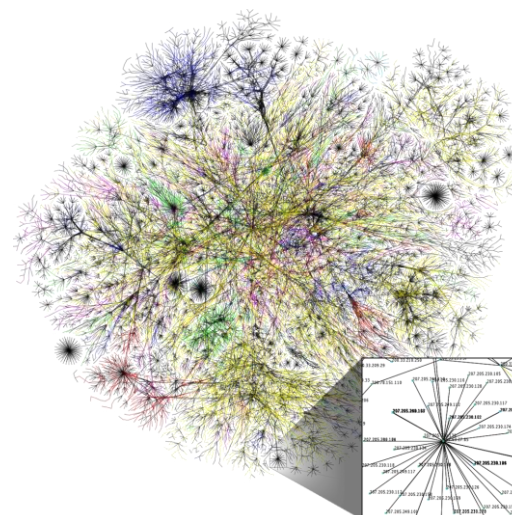
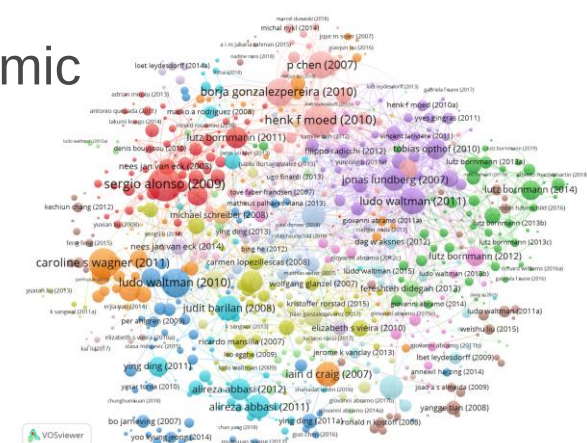
The Challenge: Graphs at Scale

- Modern real-world graphs are on the scale of millions or billions



Scales of graph datasets

Microsoft Academic
240M nodes



Internet IP
4B nodes

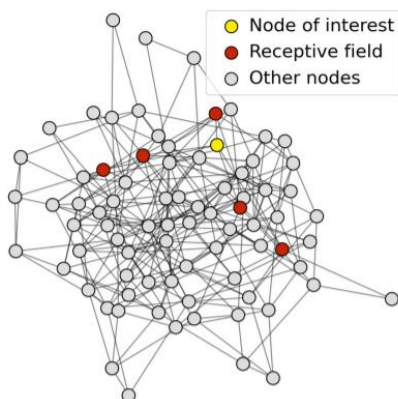
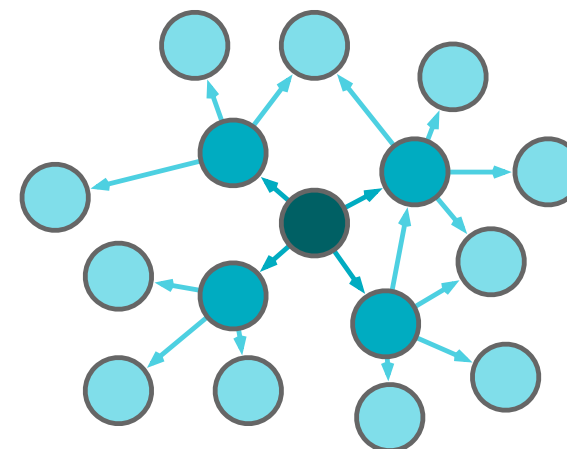
The Challenge: Graphs at Scale

- Large scale graphs: diverse domains, diverse patterns

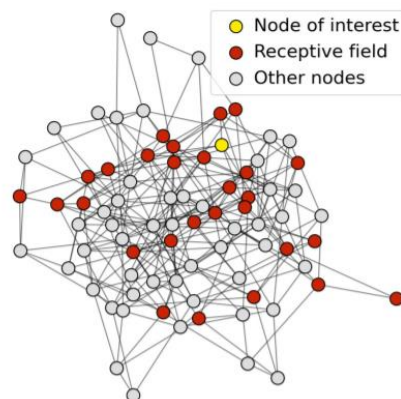
Name	Graph Size	Data Size	Domain
MAG	240M	200 GB	Heterogeneous academic graph
WikiKG	90M	160 GB	Wikidata knowledge graph
PCQM4Mv2	3M × 15	10 GB	Molecules in quantum chemistry interaction
Amazon	2M	8 GB	Product co-purchasing network
MD17	3M × 15	8 GB	Molecular dynamics trajectories
COCO-SP	120K × 500	12 GB	Segmentation of computer vision images
Snap-Patents	3M	20 GB	Patent citation network
Hyperlink Graph	4B	400 GB	Crawled webpage hyperlink

The Challenge: Neighbor Explosion

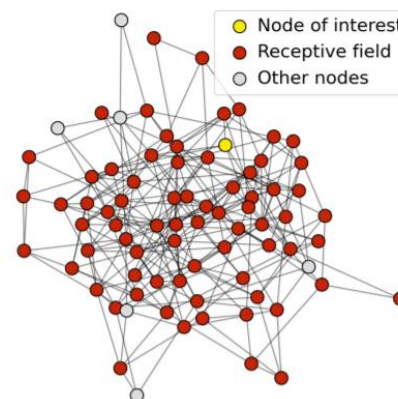
- Neighboring nodes intensively increases with network depth
- Exponential, related with edge size
- Lost of node identity



Receptive field for
1-layer GNN



Receptive field for
2-layer GNN



Receptive field for
3-layer GNN



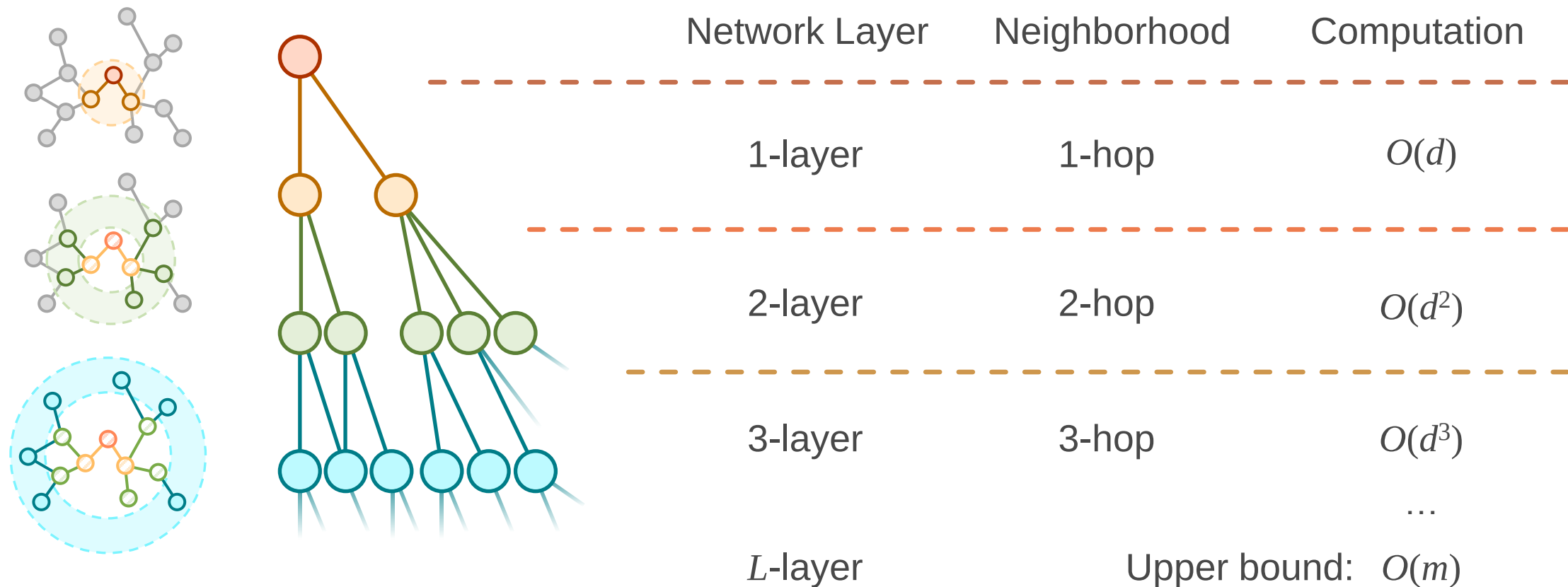
Time overhead



Expressiveness

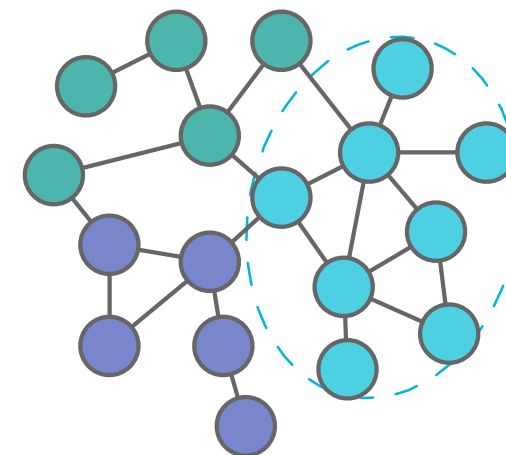
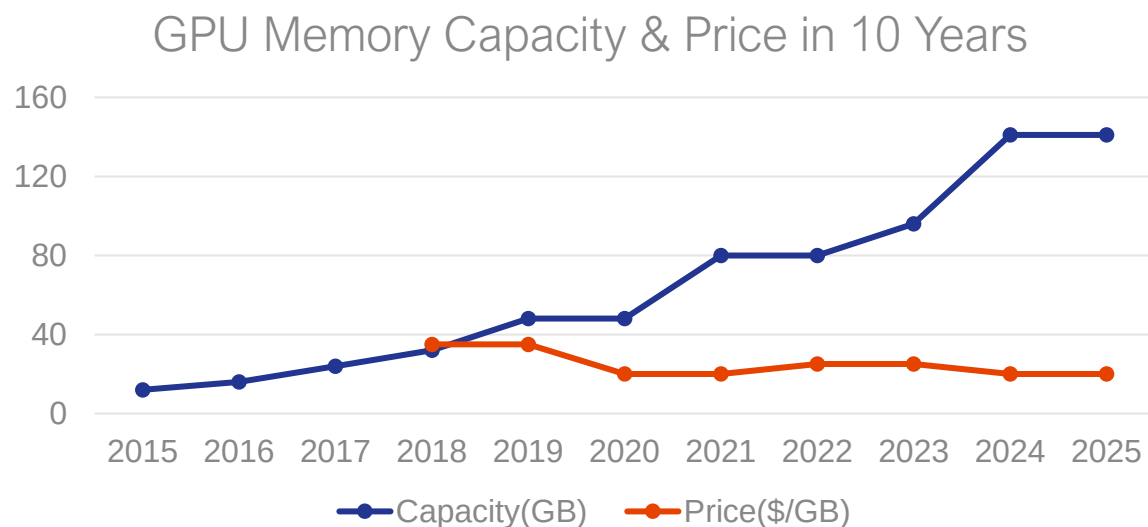
The Challenge: Neighbor Explosion

Case study: GCN computation graph



The Challenge: 🧠 Limited Memory

- GPU memory is **limited** and **costly**
- Storing and processing graph data requires large memory space



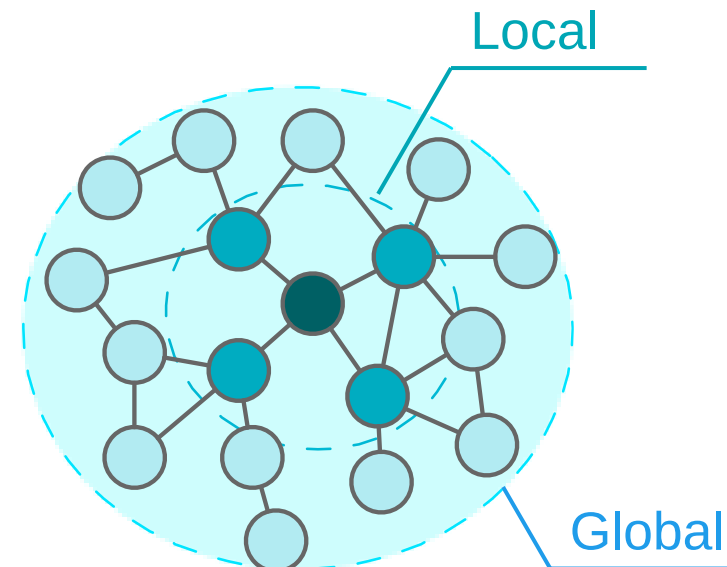
Memory overhead



Training scheme

The Challenge: Multi-scale

- Not only **local** nodes are important in prediction
- Need to capture **global** graph information efficiently
- Homophily: the inductive case, similar nodes cluster together
- Heterophily: the opposite case, connected nodes tend to be of dissimilar classes



Time overhead



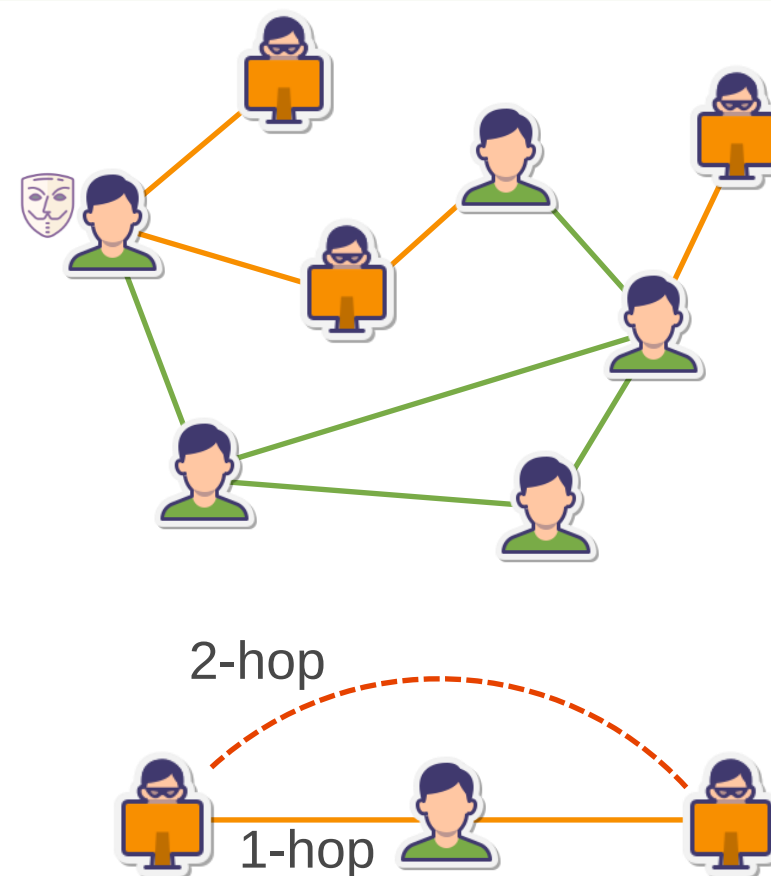
Expressiveness

The Challenge: 📁 Multi-scale

Example: Heterophily in fraud detection

- Graph: financial transaction network with **benign** or **fraud** users
- Task: identify fraud users
- **Fraudsters** use **benign** accounts as springboards 🎭 to reach other **fraudsters**

❗ *1-hop connection is not enough!*

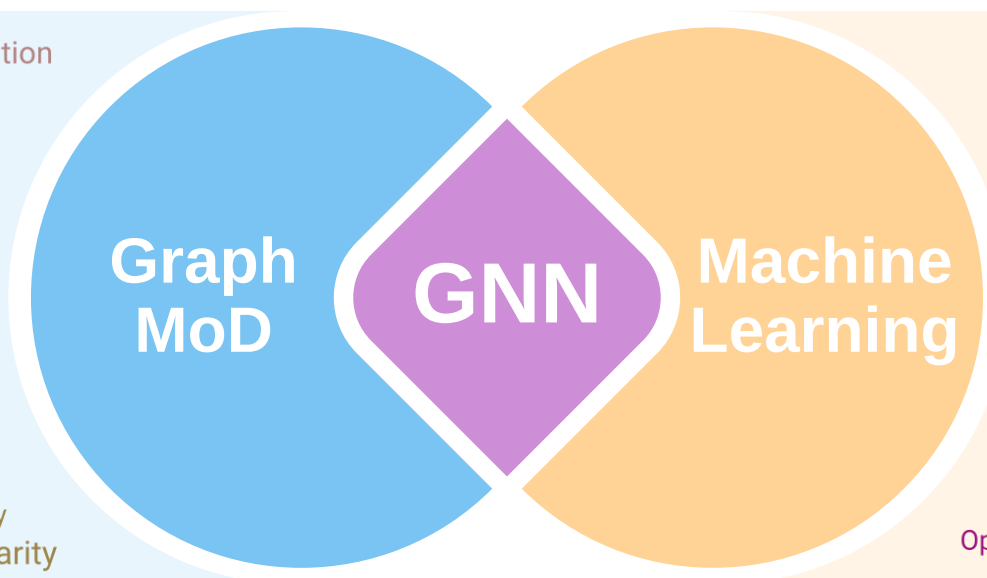


The Opportunity: Graph Data Management



What can the DB community do for GNN challenges?

Graph Embedding Simplification
Graph Storage Sampling
Subgraph Extraction
Graph Analytics Embedding
Graph Query Coarsening
Graph Similarity Sparsification
Graph Analytics Embedding
Graph Query Coarsening
Graph Similarity Sparsification

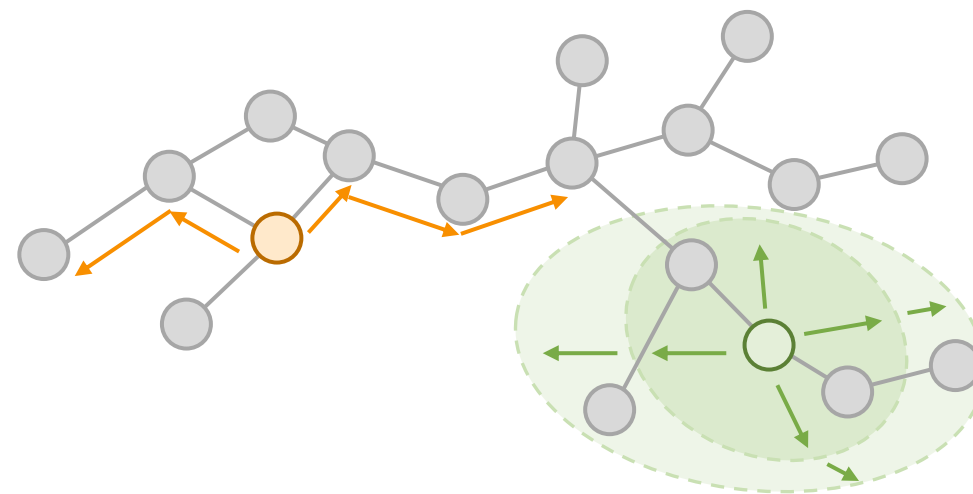


Batching Objective
Architecture Loss Function
Training Training
Network Structure Objective
Loss Function
Objective Optimization
Learning Scheme
Learning Loss
Training Optimization
Learning Loss

Preview: Neighbor Explosion

Solution from Node-wise Similarity: GBP [NeurIPS'20]

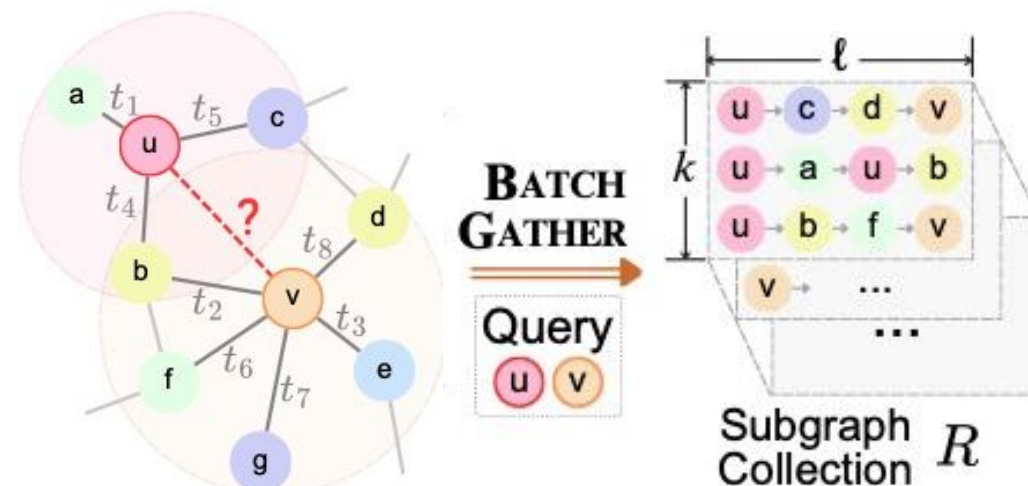
- Replace GNN computation by Personalized PageRank
- Reduce neighborhood by bidirectional search
- Decoupling CPU and GPU computation



Preview: 🧠 Limited Memory

Solution from Subgraph Extraction: GENTI [VLDB'24]

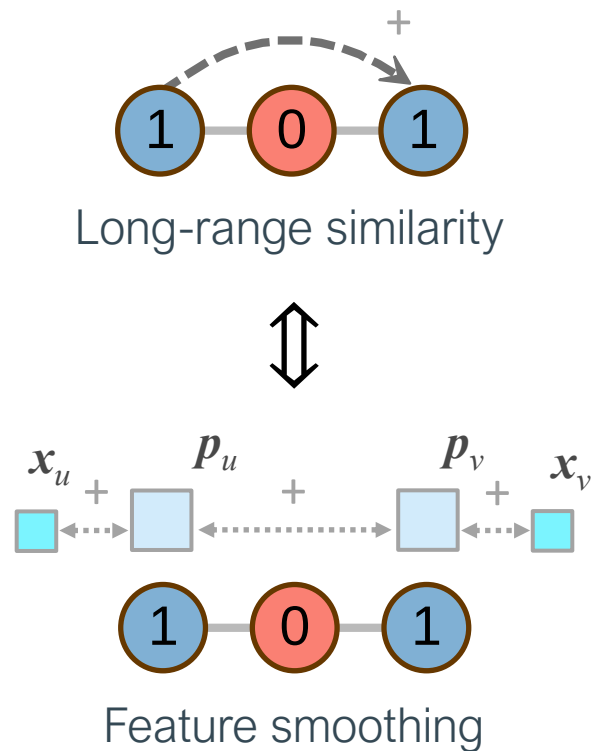
- Only load subgraph (part of the graph) for GNN computation
- Structured subgraph sampled by k -many random walks
- GPU-oriented data structure and batch computation



Preview: Multi-scale

Solution from Spectral Embedding: LD² [NeurIPS'23]

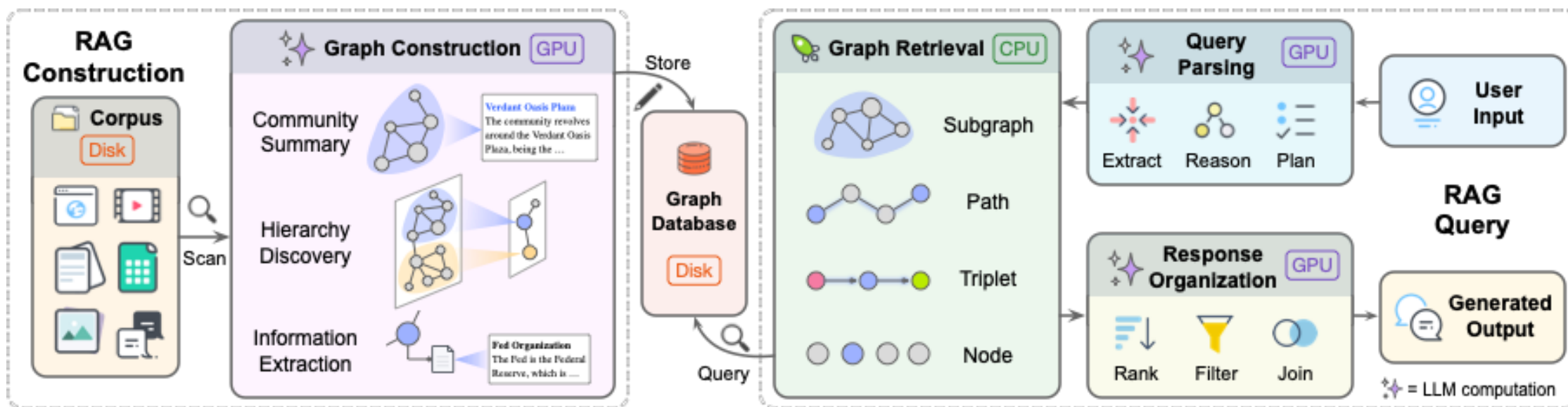
- Embed full-graph information in spectral domain
- Bridge spatial and spectral long-range graph operations
- Fast & scalable spectral embedding computation



Preview: Scalable Graph+LM

Future Direction in Graph-based RAG

- Scalable graph embedding, indexing, and retrieval
- Integration between graph data & LLM



Advances in Designing Scalable GNNs

Classical Approaches for Scalable GNNs

Outline

Introduction

Classical Approaches for Scalable GNNs

Basic GNN: GCN (T Kipf & M Welling | ICLR'17)

Graph Sampling: GraphSAGE (W Hamilton et al | NeurIPS'17)

Graph Partition: AliGraph (R Zhu et al | VLDB'19)

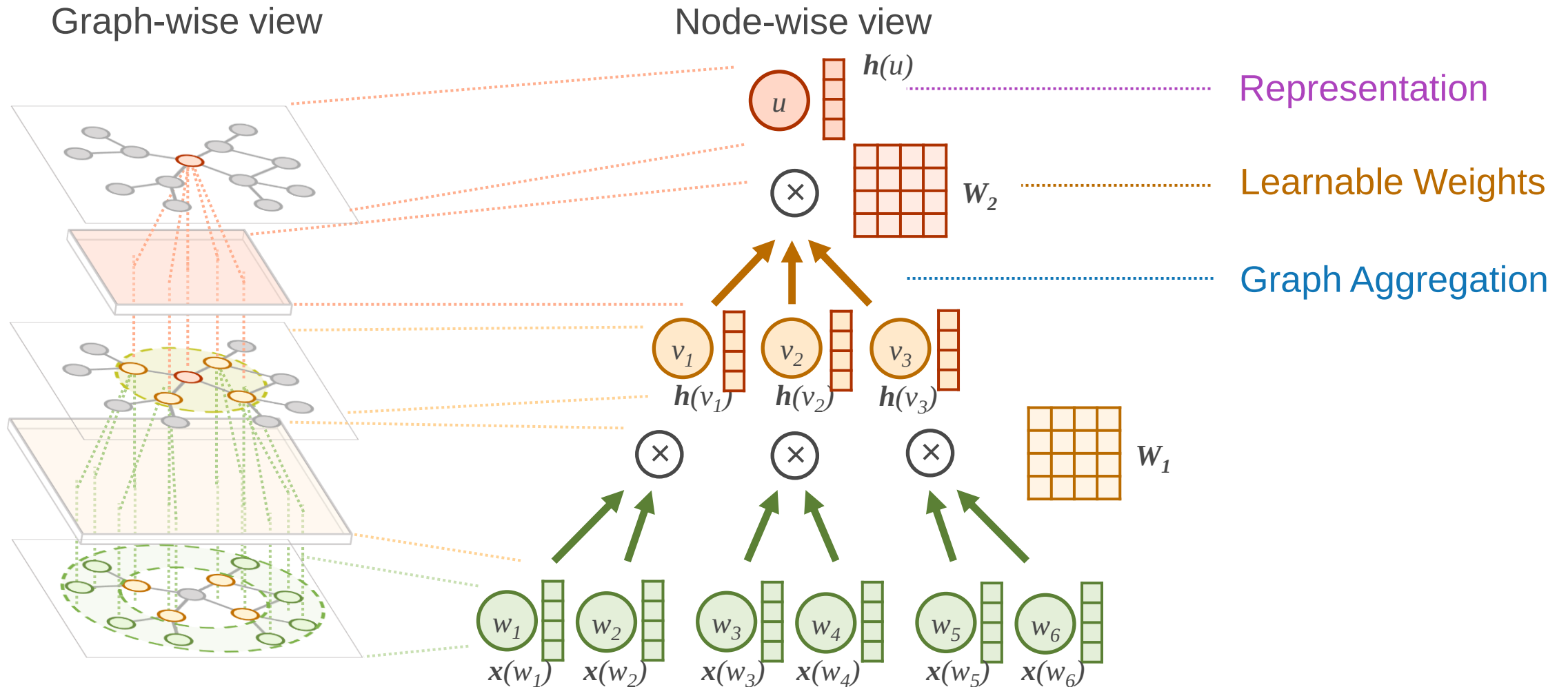
Decoupled Propagation: SGC (F Wu et al | ICLR'19)

Application: Social Network Friend Recommendation

Hardware-aware Sampling

Subgraph Extraction

Basic GNN: GCN Scalability Analysis

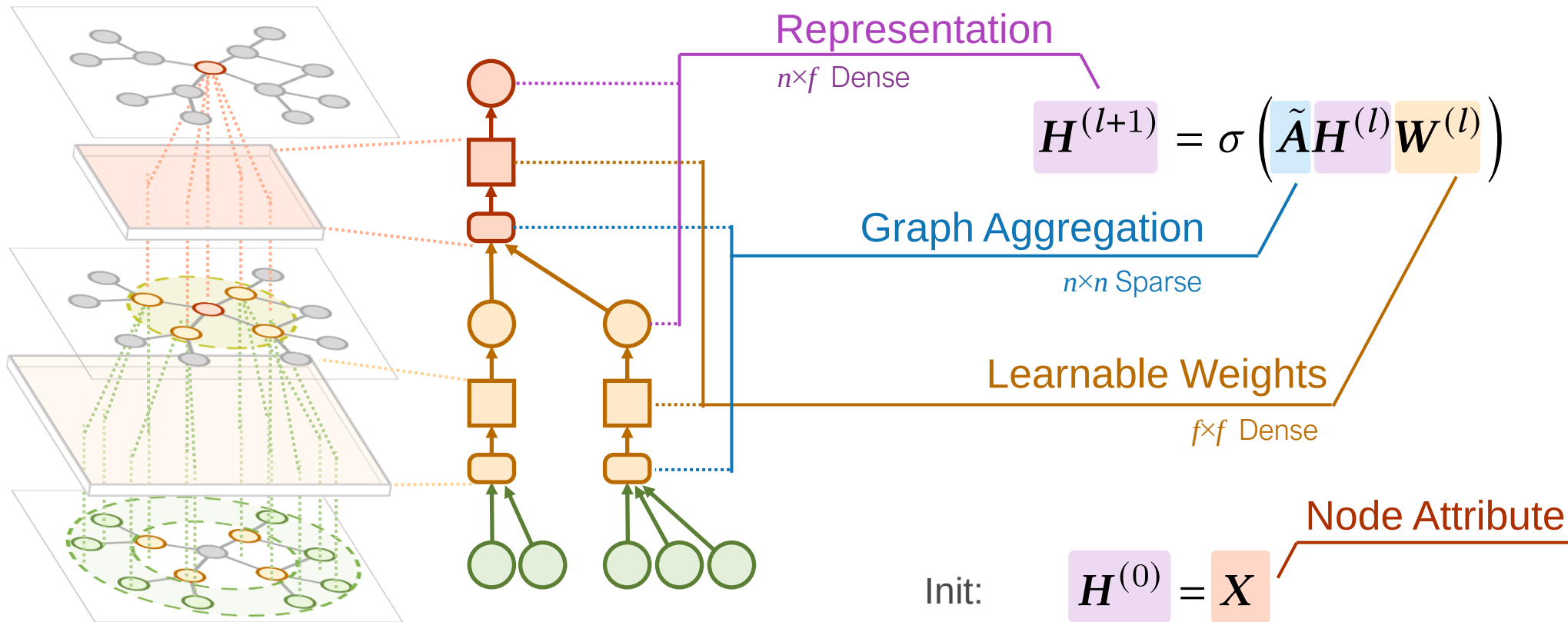


Basic GNN: GCN Scalability Analysis

Graph-wise view

Node-wise view

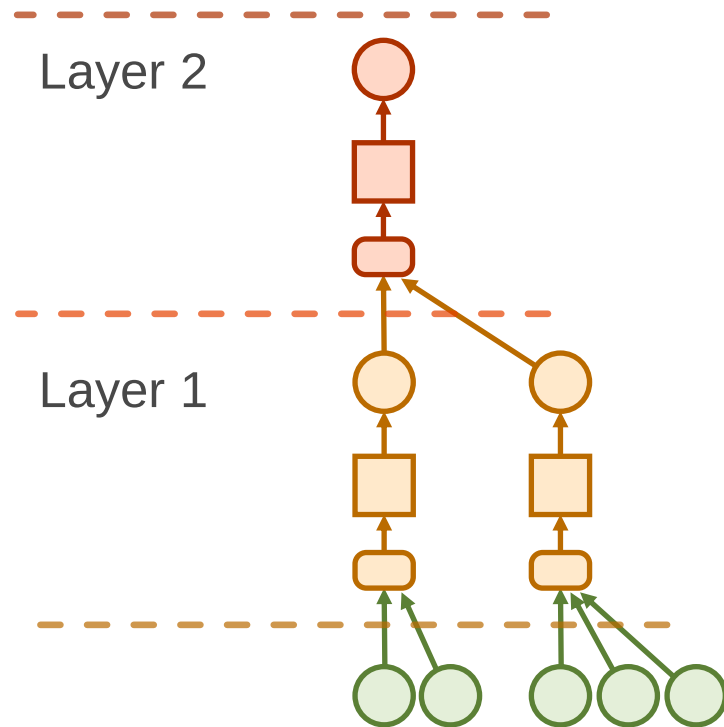
Computation



Basic GNN: GCN Complexity

Node-wise view

Computation



Graph Aggregation

Sparse-dense Matrix Multiplication

Weight Transformation

Dense-dense Matrix Multiplication

$$H^{(l+1)} = \sigma \left(\tilde{A} H^{(l)} W^{(l)} \right)$$

$O(mf)$

$O(nf^2)$

L -layers:

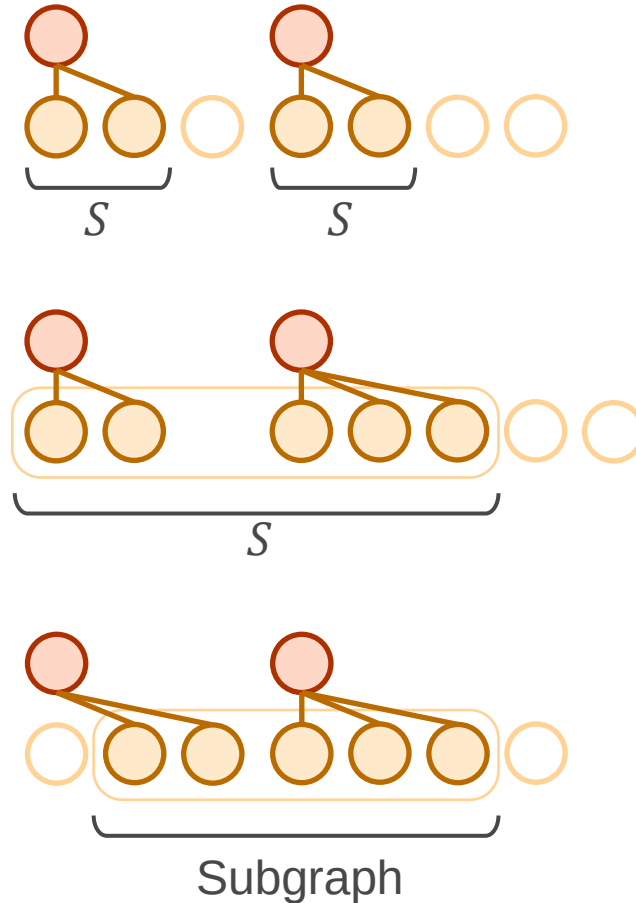
$O(Lmf + Lnf^2)$

Graph Sampling

- Node-wise sampling: limit neighborhood size of each node
- Layer-wise sampling: limit total neighborhood size of each layer
- Subgraph-wise sampling: limit neighborhood within a subgraph



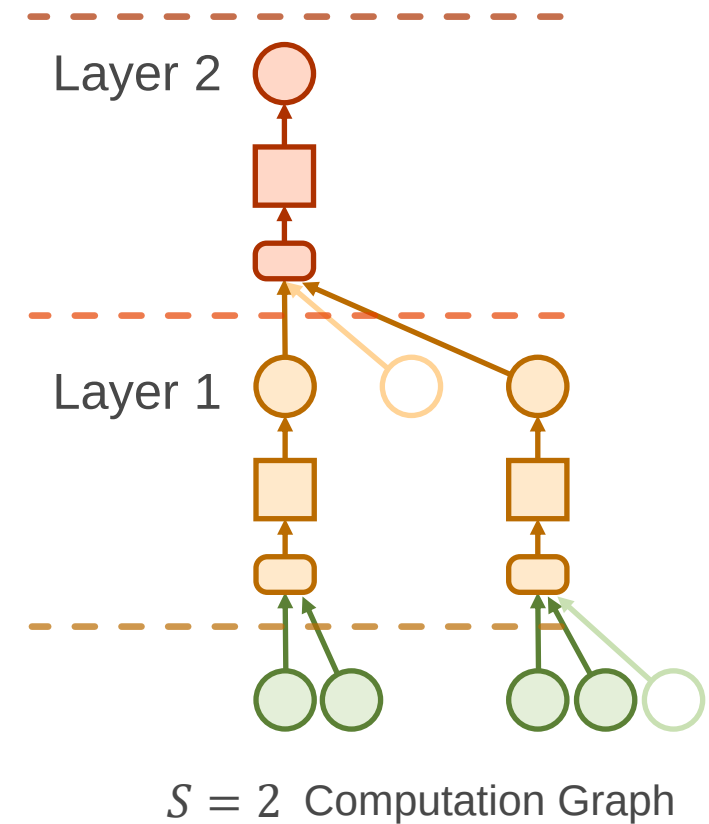
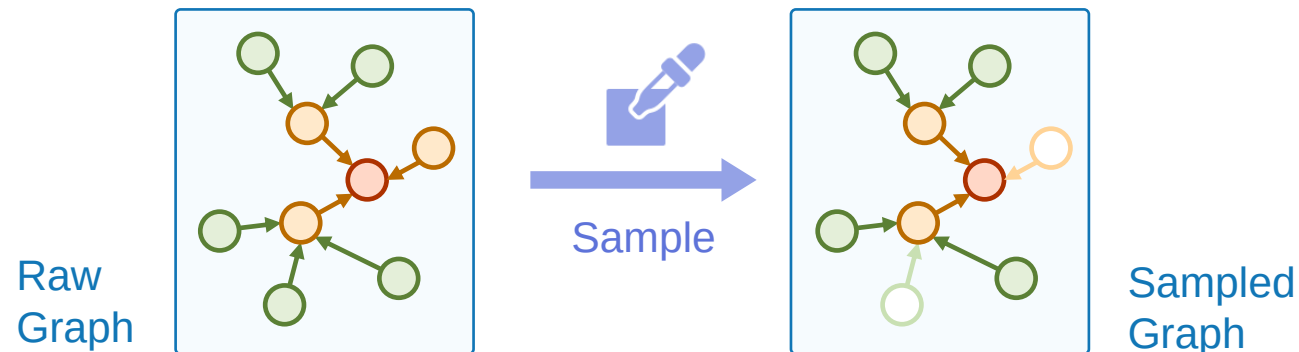
X Liu et al, "Sampling Methods for Efficient Training of Graph Convolutional Networks: A Survey", *J. Autom. Sinica* 2022.



Graph Sampling: GraphSAGE (NeurIPS'17)



- Sampling in graph aggregation: limit neighborhood size of each node to S
- Structured neighborhood: sampled representations are fixed-length
- Feasible for various aggregation schemes

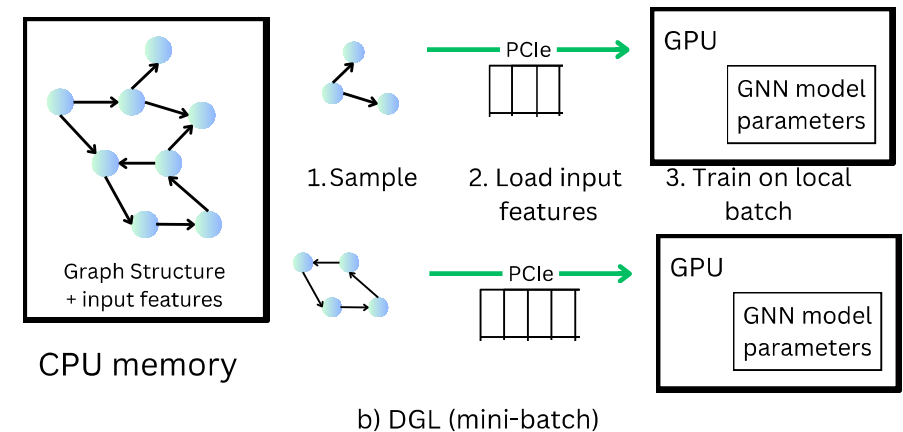
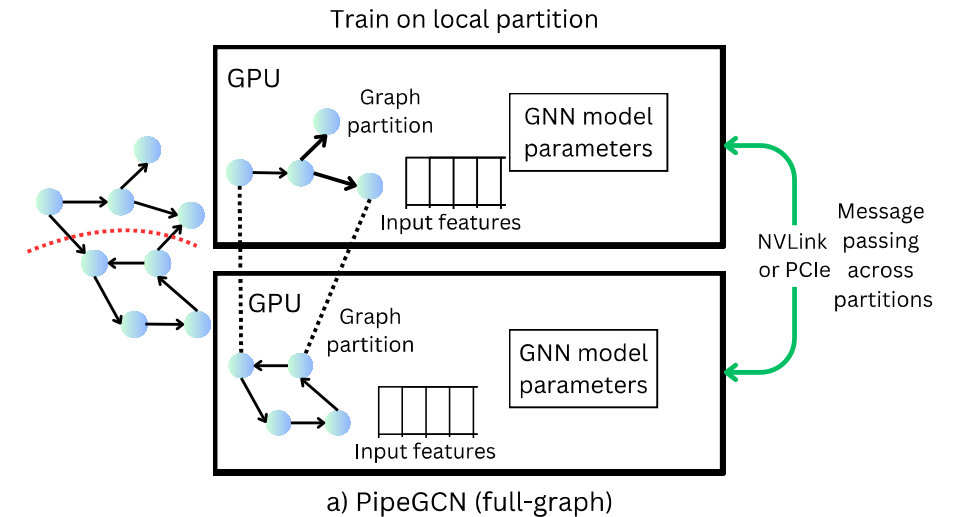


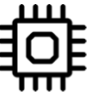
Graph Partition

- Objective: minimize data loss in subgraphs during training
- Commonly used in multi-host/
multi-GPU training
- MB: only load subgraph onto GPU



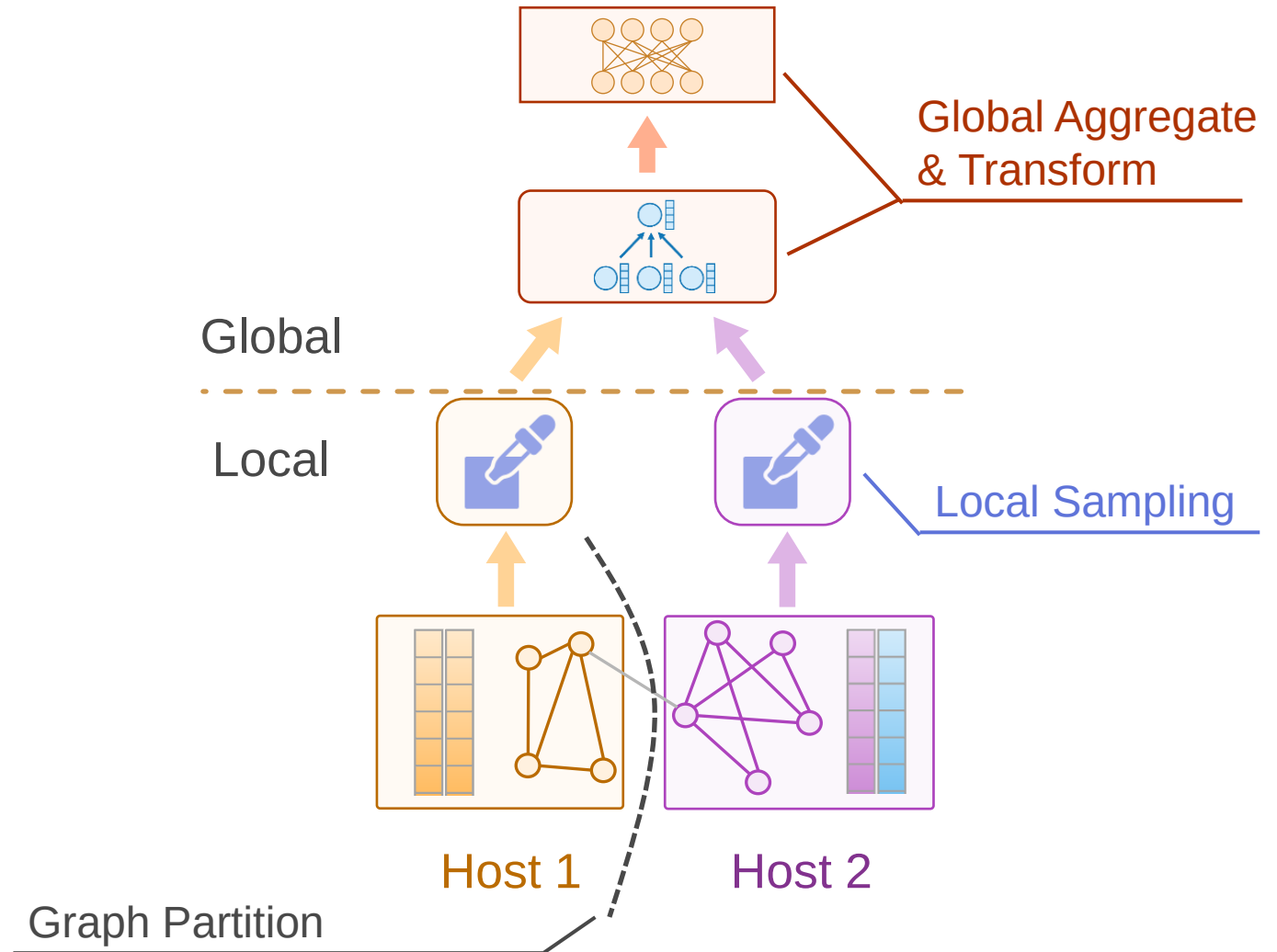
H Yuan et al, "Comprehensive Evaluation of GNN Training Systems: A Data Management Perspective", *VLDB'24*.





Graph Partition: AliGraph (VLDB'19)

- Available partitions:
 - METIS
 - Vertex/Edge cut
 - 2D partition
 - Streaming
- Local Sampling within subgraphs on each host



Decoupled Propagation

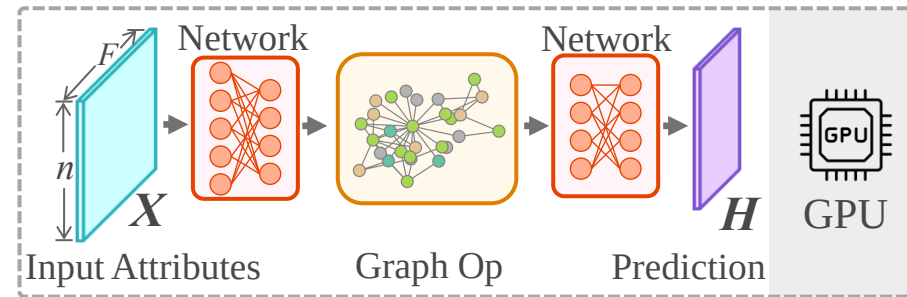
- Graph propagation is the computational bottleneck
- Graph computation $\rightarrow$ CPU
- Network learning $\rightarrow$ GPU
- MB: random sampling/
hashing



S Bajaj et al, "Graph Neural Network Training Systems: A Performance Comparison of Full-Graph and Mini-Batch", *VLDB'25*.

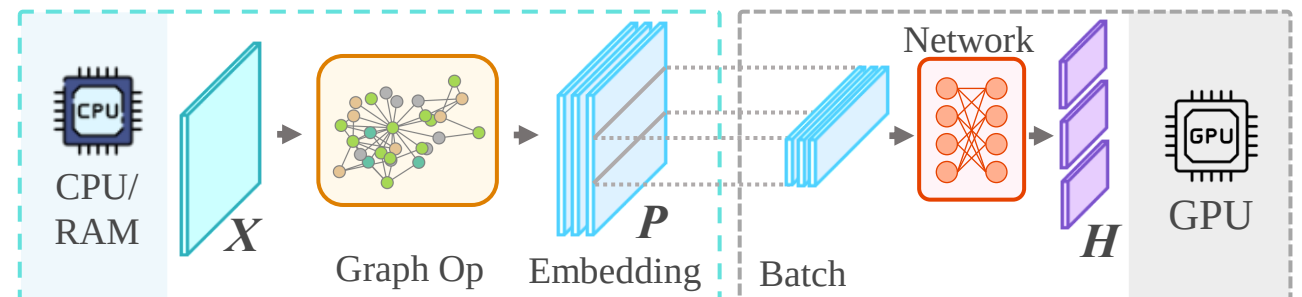
Full-batch

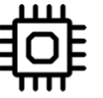
Full GPU Workload



Mini-batch

CPU+GPU Workload





Decoupled Propagation: SGC (ICLR'19)

- Combine the iterative propagations in GCN
- Reduce GPU memory overhead: $O(m) \rightarrow O(s)$
- No change in time overhead: remain $O(Lm)$

GCN (iterative)

$$\mathbf{H}^{(L)} = \sigma(\underbrace{\tilde{\mathbf{A}} \sigma(\tilde{\mathbf{A}} \cdots \tilde{\mathbf{A}} \mathbf{X} \mathbf{W}^{(1)} \cdots) \mathbf{W}^{(L-1)}}_{L \text{ Weight Transformation}})$$

L Graph Propagation

SGC (decoupled)

L Graph Propagation

$$\mathbf{H}^{(0)} = \mathbf{P} = \tilde{\mathbf{A}}^L \cdot \mathbf{X}$$

 CPU/RAM

L Weight Transformation

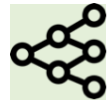
$$\mathbf{H}^{(l+1)} = \sigma(\mathbf{H}^{(l)} \mathbf{W}^{(l)})$$

 GPU

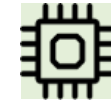
Classical Approaches: Takeaways

- Graph computation is the bottleneck in GNN scalability

- Challenges:



Neighbor Explosion



Limited Memory

- Approaches:

Graph Sampling

- Reduce neighborhood aggregation

Graph Partition

- Reduce subgraph size

Graph Partition

- Batch training on subgraphs

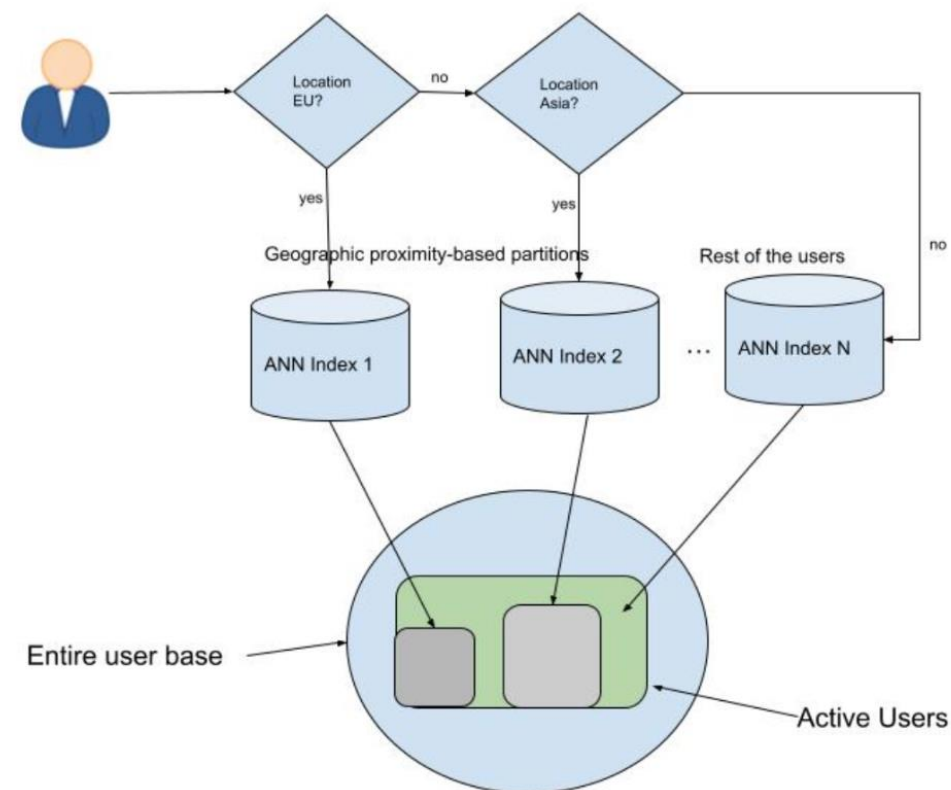
Decoupled Propagation

- Transfer computation to CPU

Classical Approaches: Applications

Social Network Friend Recommendation

- Candidates are partitioned: based on different retrieval criteria
- GraphSAGE sampling: fast neighbor lookup
- MB: sampling within partitions



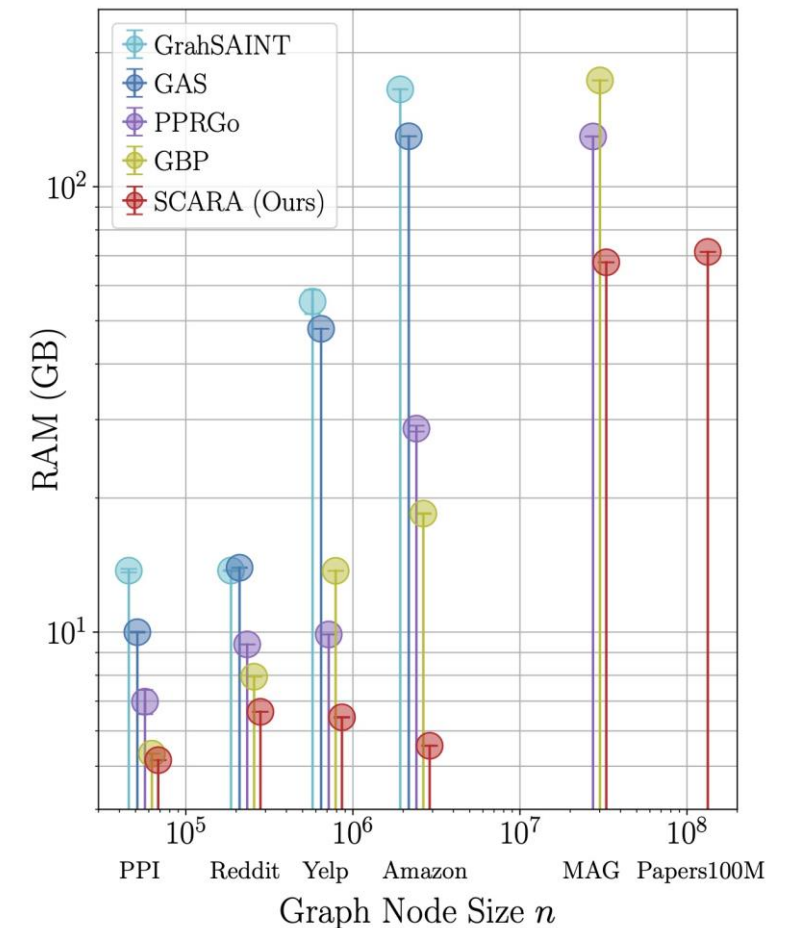
Classical Approaches: Evaluation

Memory Footprint

- Sampling: suffer from neighbor explosion for using more redundant memory
- Decoupling: better memory scalability for less additional memory overhead
- Possible OOM under limited memory



N Liao et al, "Scalable Decoupling Graph Neural Networks with Feature-Oriented Optimization", VLDBJ'23.



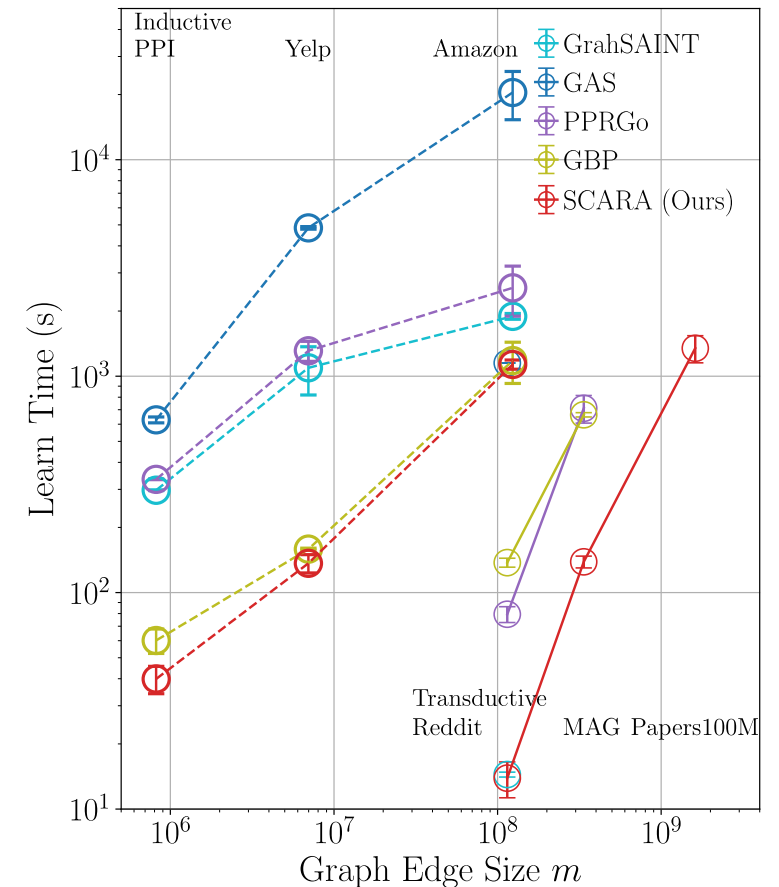
Classical Approaches: Evaluation

Time Efficiency

- Sampling: higher time overhead caused by device idleness and transmission
- Decoupling: speed mostly determined by graph computation algorithms



N Liao et al, "Scalable Decoupling Graph Neural Networks with Feature-Oriented Optimization", VLDBJ'23.



Advances in Designing Scalable GNNs

Hardware-aware Sampling

Outline

Introduction

Classical Approaches

Hardware-aware Sampling

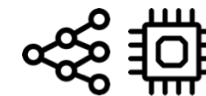
Transmission ADGNN [Z Song et al | SIGMOD'23]

GIDS [J Park et al | VLDB'24]

Computation NeutronOrch [X Ai et al | VLDB'24]

DAHA [Z Li et al | VLDB'24]

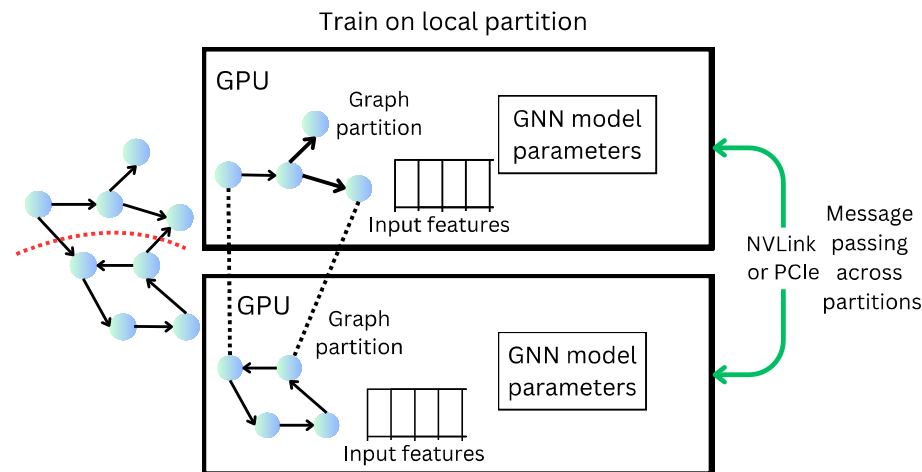
Subgraph Extraction



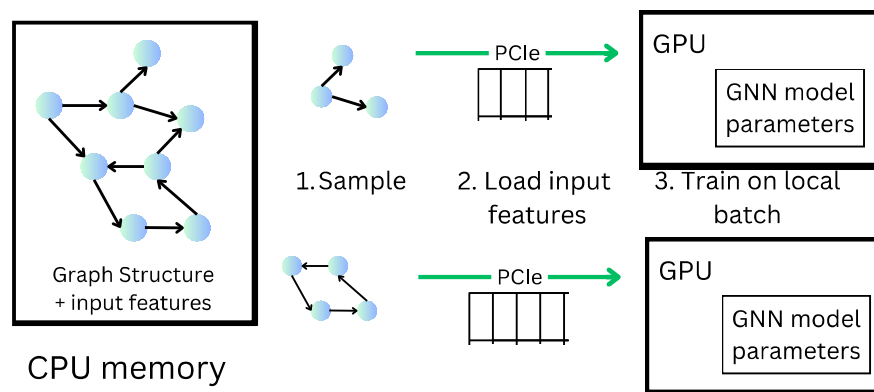
Transmission: ADGNN [SIGMOD'23]

Global Sampling

- Full graph view
- Precision guarantee
- Transmission overhead



a) PipeGCN (full-graph)



b) DGL (mini-batch)

Local Sampling

- Distributed storage & computation
- Information loss
- Depends on partition



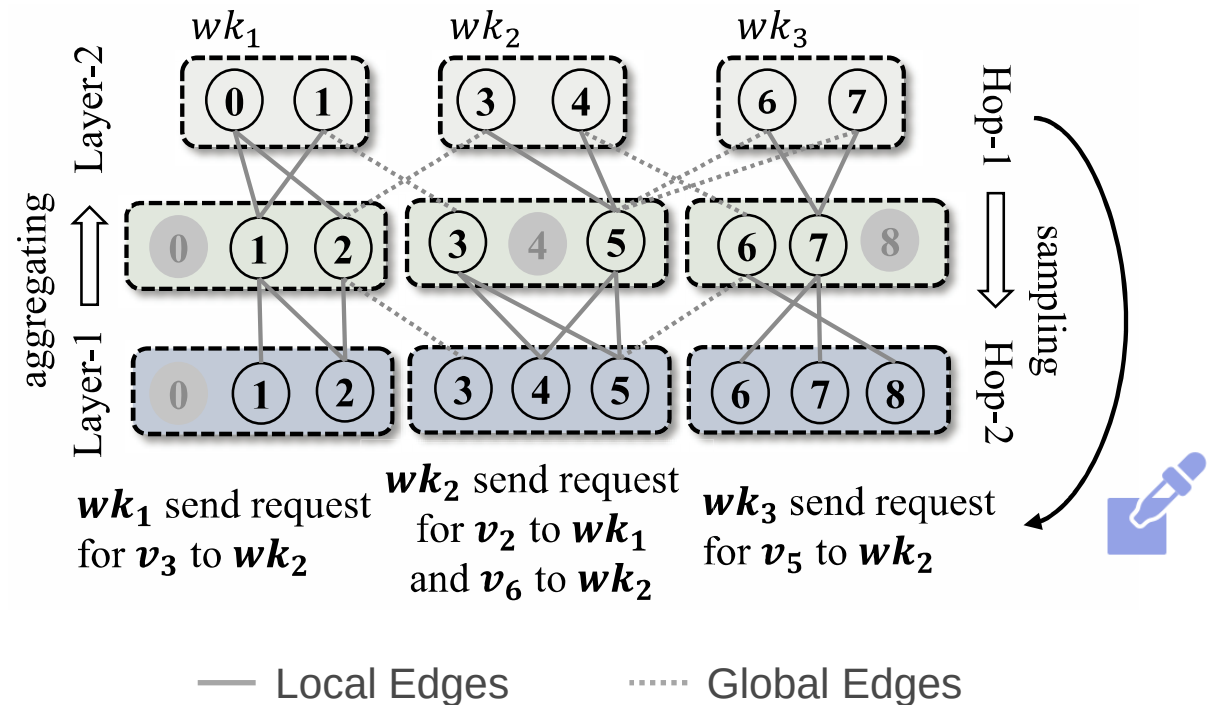
Transmission: ADGNN [SIGMOD'23]

Balance Local & Global Sampling

- Maintain a set of effective neighbors locally
- Only communicate for top remote nodes that reduce precision

$$AD_{S(v;k)}^{t,\ell} = \left\| \frac{1}{|\mathcal{N}(v)|} \sum_{u \in \mathcal{N}(v)} H_u^{t,\ell} - \frac{1}{k} \sum_{u \in S(v;k)} H_u^{t,\ell} \right\|^2,$$

Precision (above the first term)
Sampled Repr (below the first term)
Sampled Repr (below the second term)

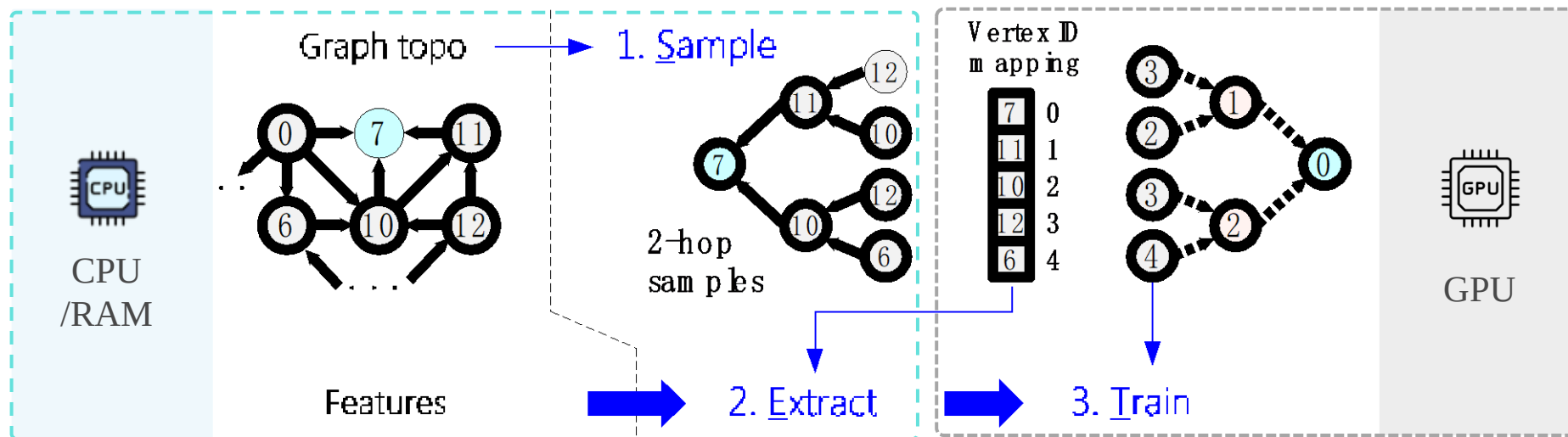




Transmission: GIDS [VLDB'24]

Graph Structure: Parallel Sampling

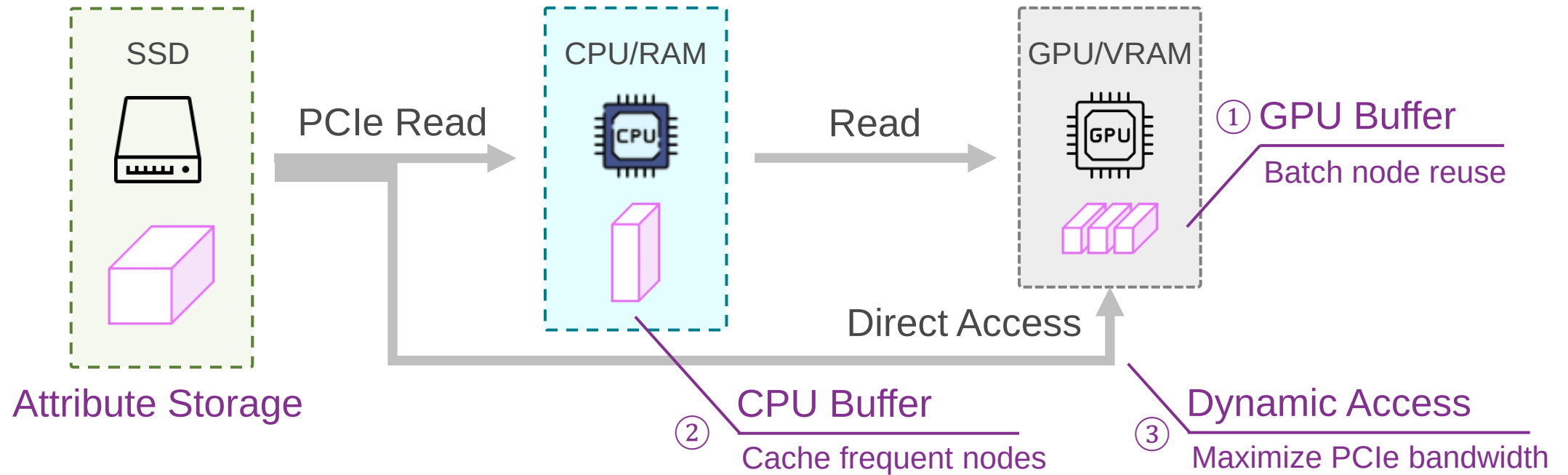
- Sampling can be performed independently from NN training
- Parallel sampling for batches on CPU graph storage

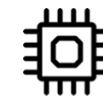




Transmission: GIDS [VLDB'24]

Node Attribute: Hierarchical Buffer

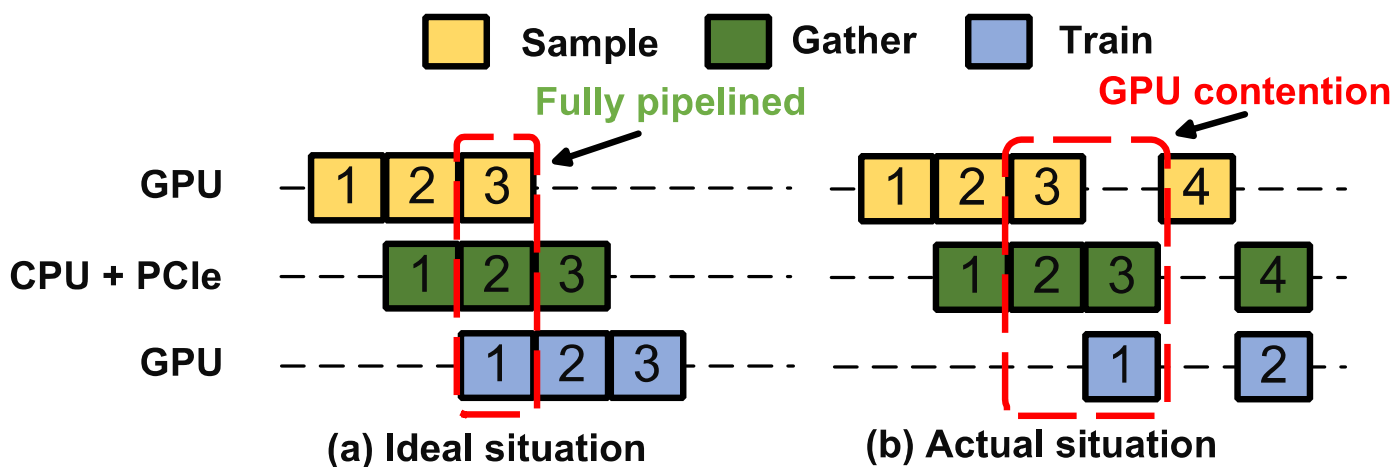




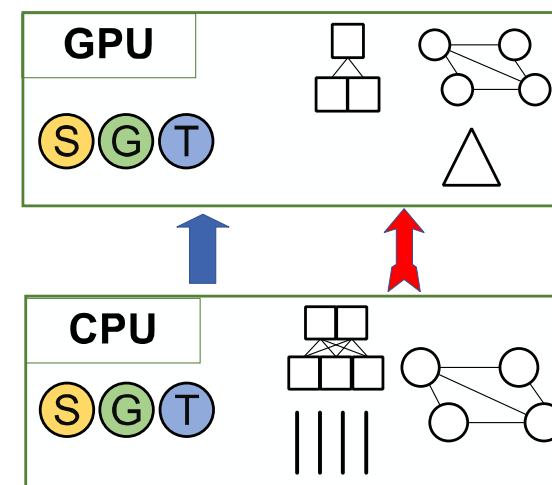
Computation: NeutronOrch [VLDB'24]

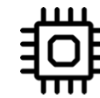
Layer-based Computation Orchestrating

- CPU: 1st layer, from attribute to embedding
- GPU: higher layers, only process embeddings



➡ Feature transfer
➡ Embedding transfer

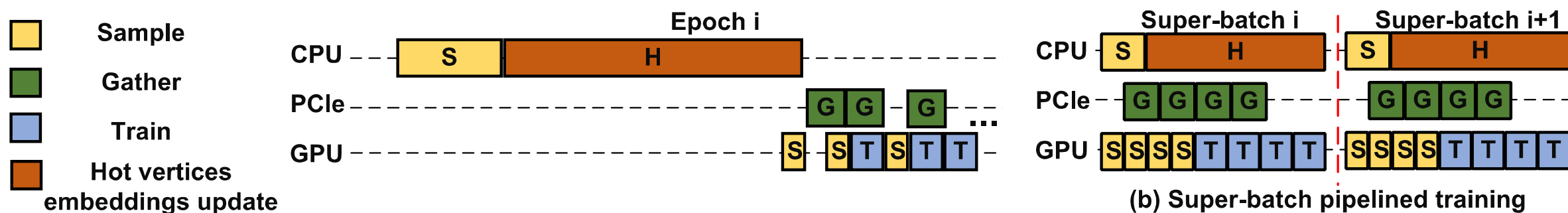


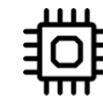


Computation: NeutronOrch [VLDB'24]

Cross-batch Data Pipelining

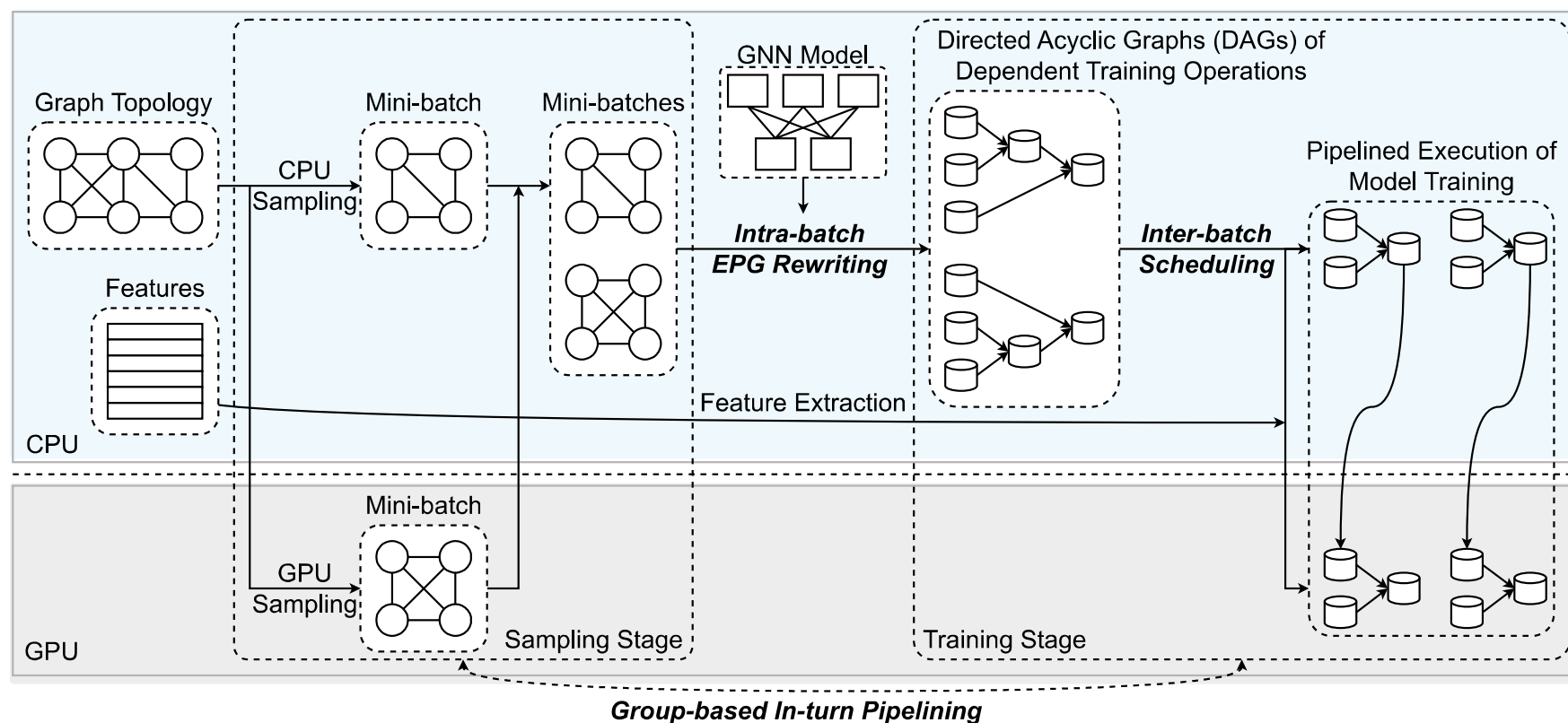
- Hotness-aware sampling across batches on CPU
- Constrain staleness within super-batch





Computation: DAHA [VLDB'24]

- Profiling & scheduling between CPU and GPU computation



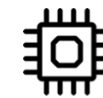
CPU

- Feature processing
- Weight transformation
- Major sampling



GPU

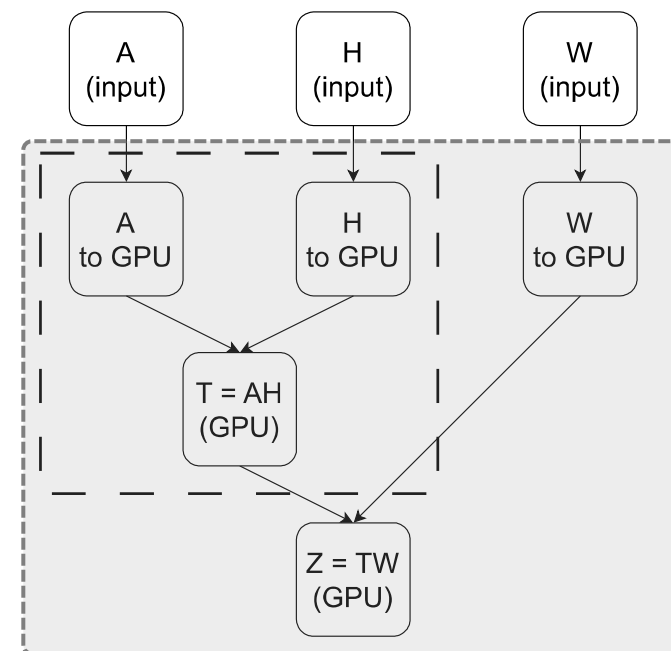
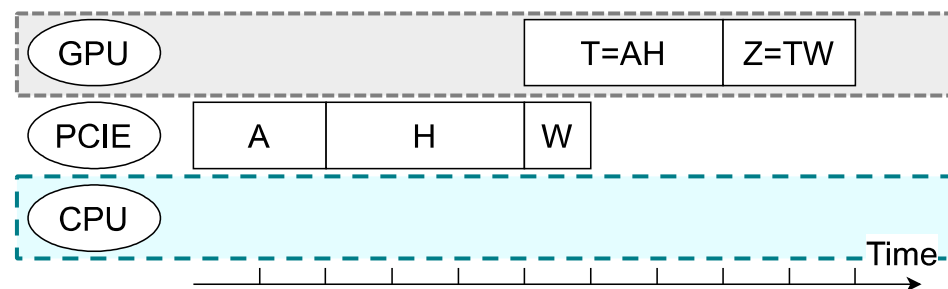
- Accelerate sampling
- Graph aggregation



Computation: DAHA [VLDB'24]

Computation 1: Pure GPU

- Fast feature processing (dense MM)
- Long data transfer

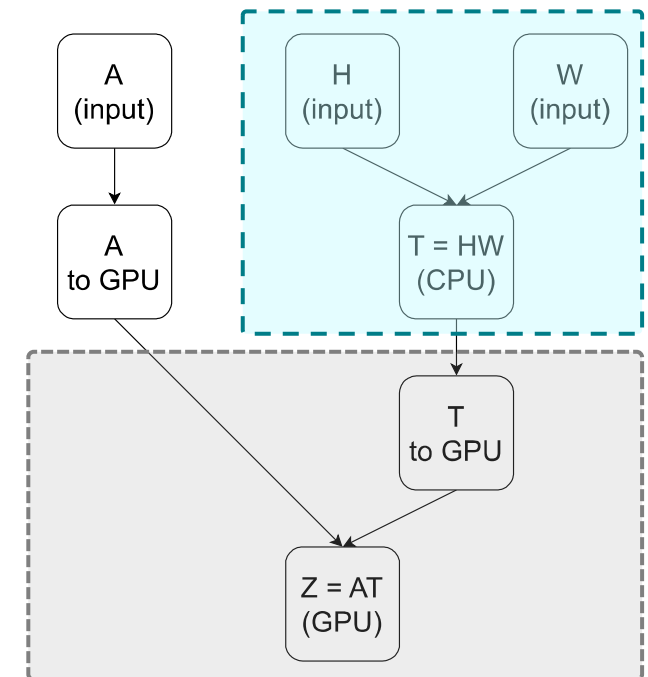
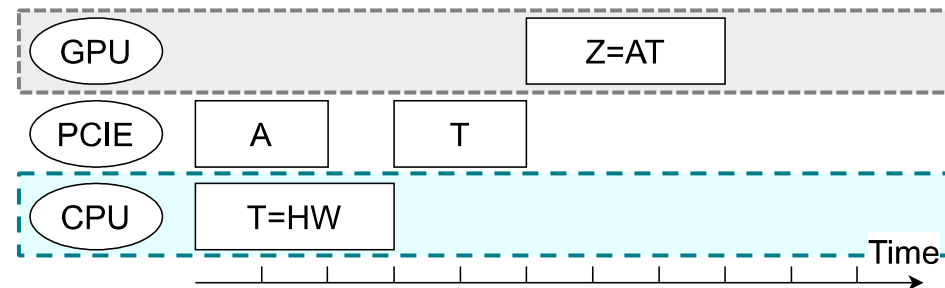




Computation: DAHA [VLDB'24]

Computation 2: Hybrid GPU

- Shift computation workload to CPU
- Only transfer embedding with lower dimension



Hardware-aware Sampling: Takeaways

Transmission

ADGNN [SIGMOD'23]

- Between computational devices
- Balance Local & Global Sampling

GIDS [VLDB'24]

- Between storage devices
- Hierarchical node attribute buffer

Computation

NeutronOrch [VLDB'24]

- Among GNN operations
- Layer- & batch-wise pipelining

DAHA [VLDB'24]

- Among data partitions
- Balance CPU & GPU computation

Advances in Designing Scalable GNNs

Subgraph Extraction

Outline

Introduction

Classical Approaches

Hardware-aware Sampling

Subgraph Extraction

Partition G3 [X Wan et al | SIGMOD'23]

Generation SUREL [H Yin et al | VLDB'23] & GENTI [Z Yu et al | VLDB'24]

Slicing TIGER (K Wang et al | VLDB'24)

Application: Hardware Obfuscation Analysis

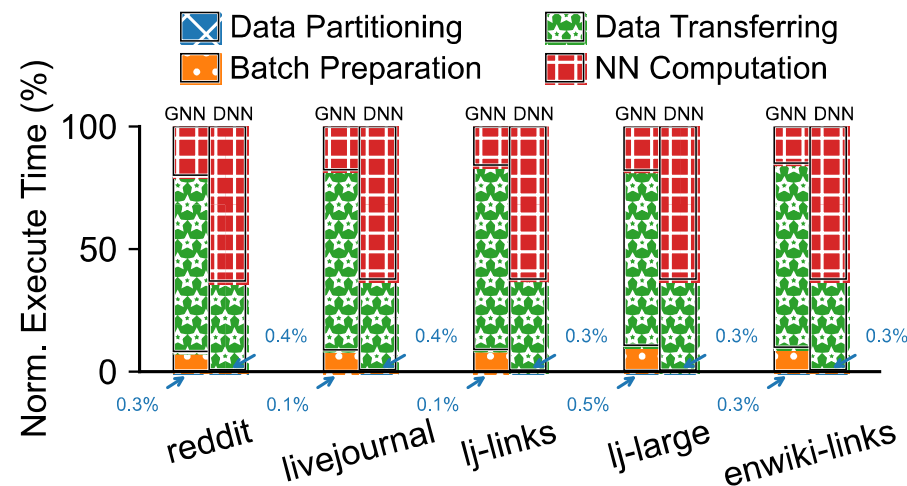
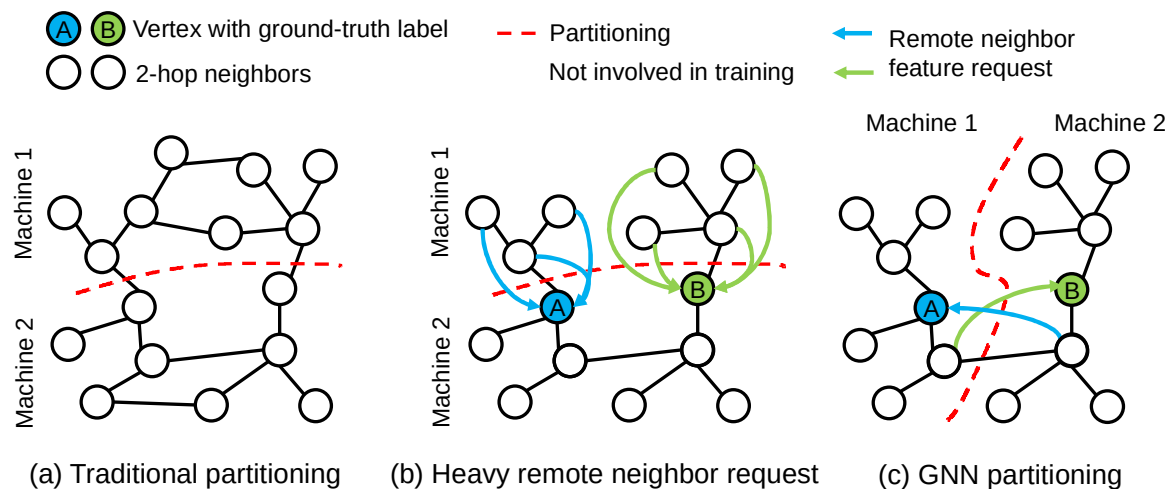
Subgraph Extraction: Challenges

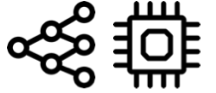
🎯 Effectiveness

- Topology-based partition may be not optimal

🧠 Efficiency

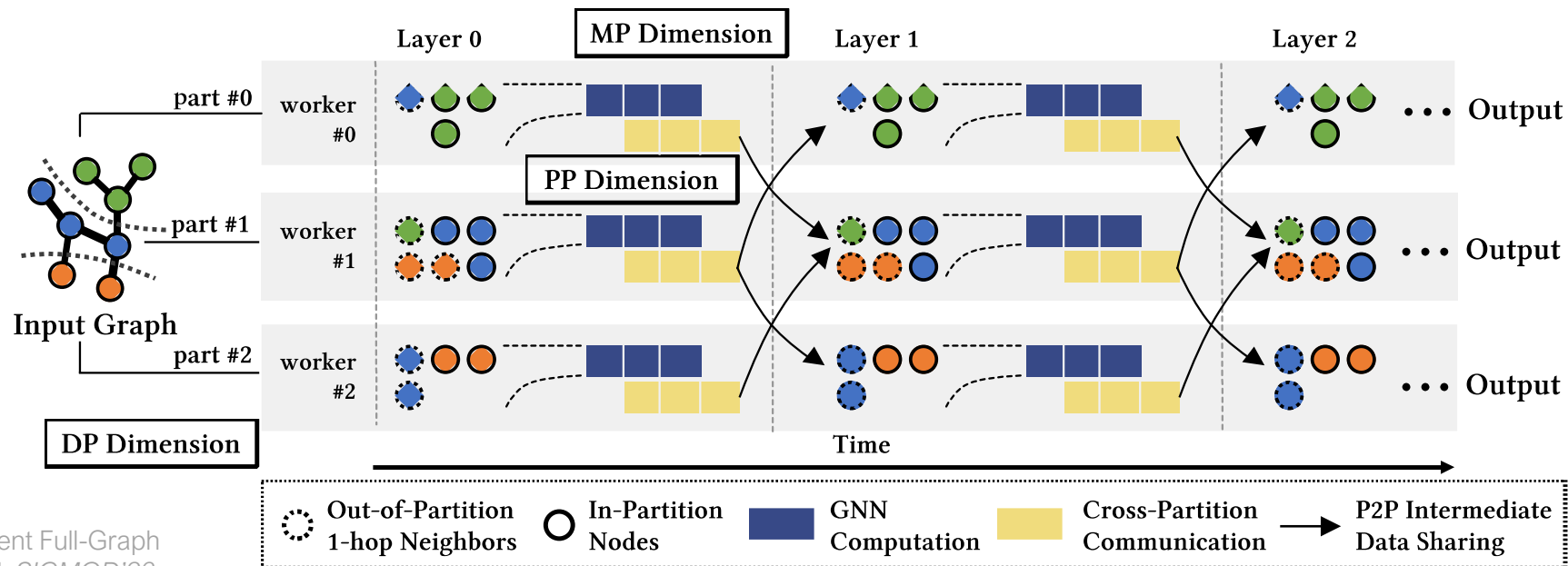
- Data transferring overhead is significant





Partition: G3 [SIGMOD'23]

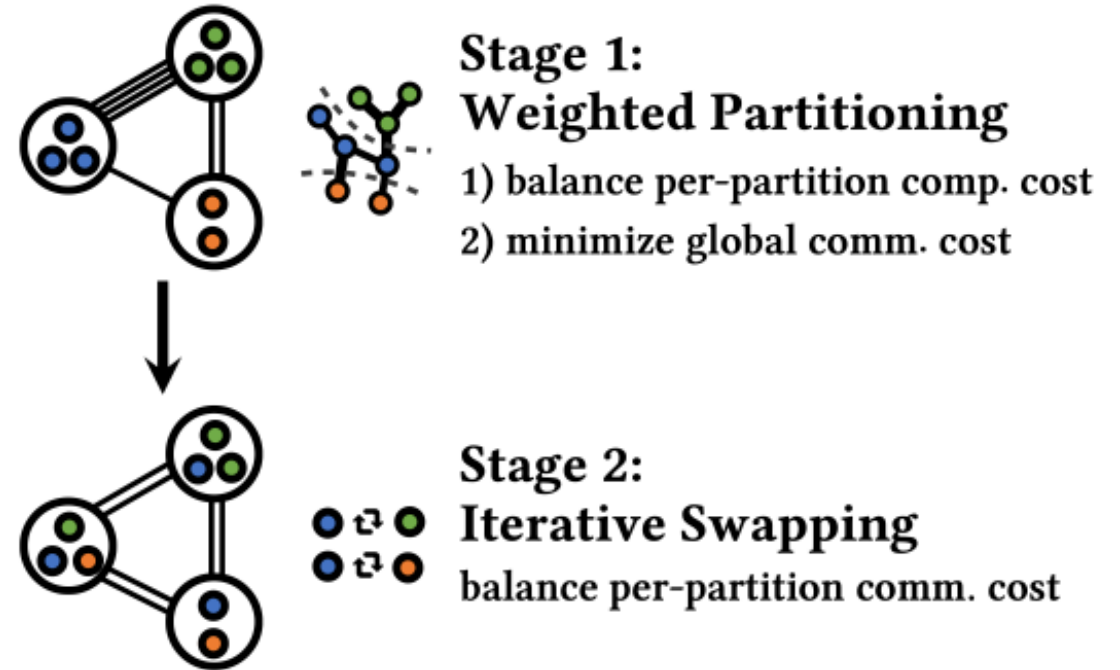
- Hybrid Parallelism
 - Data-parallel: graph partitions on workers
 - Model-parallel: full-graph training with P2P transmission
 - Pipeline-parallel: overlapped computation and transmission





Partition: G3 [SIGMOD'23]

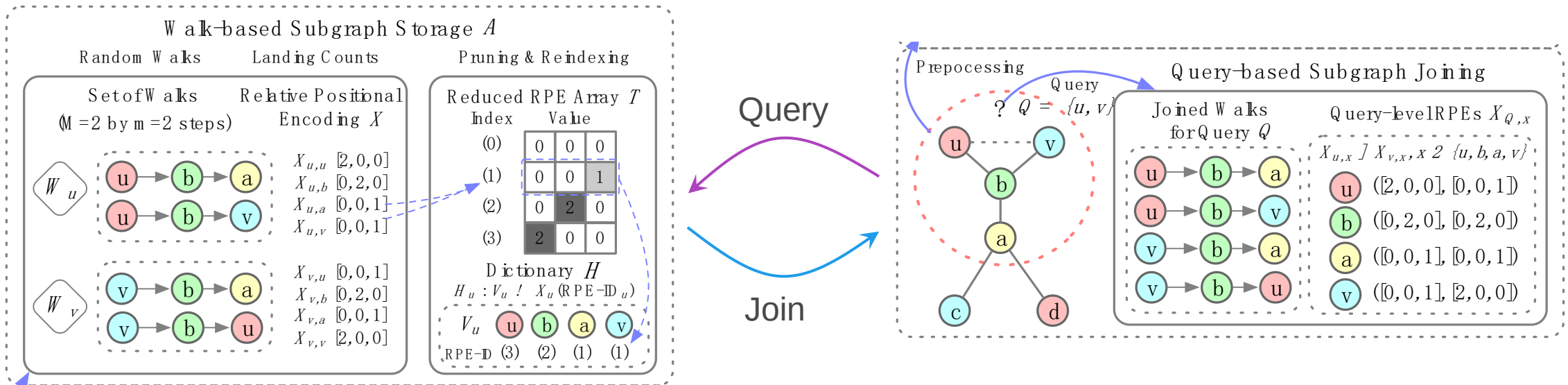
- 2-stage graph partition
 - Stage 1: unsupervised, minimize global communication by reducing cut edges
 - Stage 2: adaptive updated, balance empirical communication

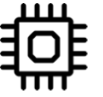




Generation: SUREL [VLDB'23]

- Walk-based Subgraph Extraction
 - Sample multiple walks for each query node
 - Join walks to generate a subgraph

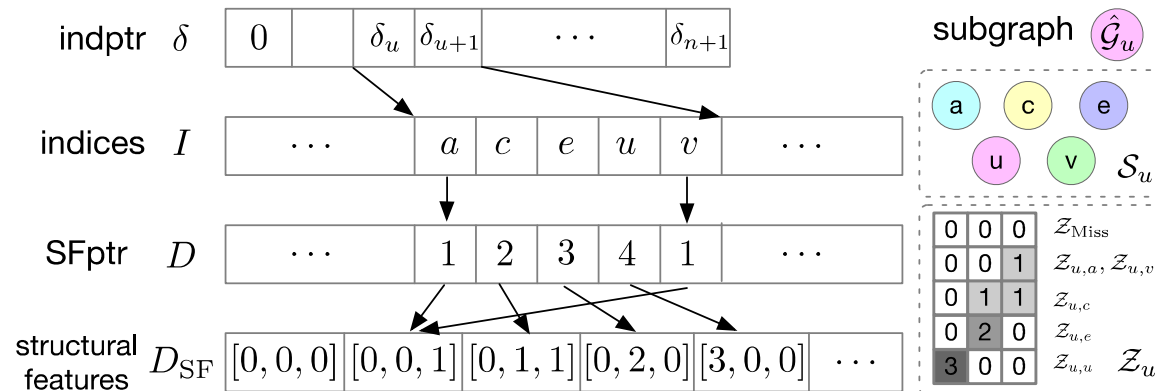




Generation: SUREL [VLDB'23]

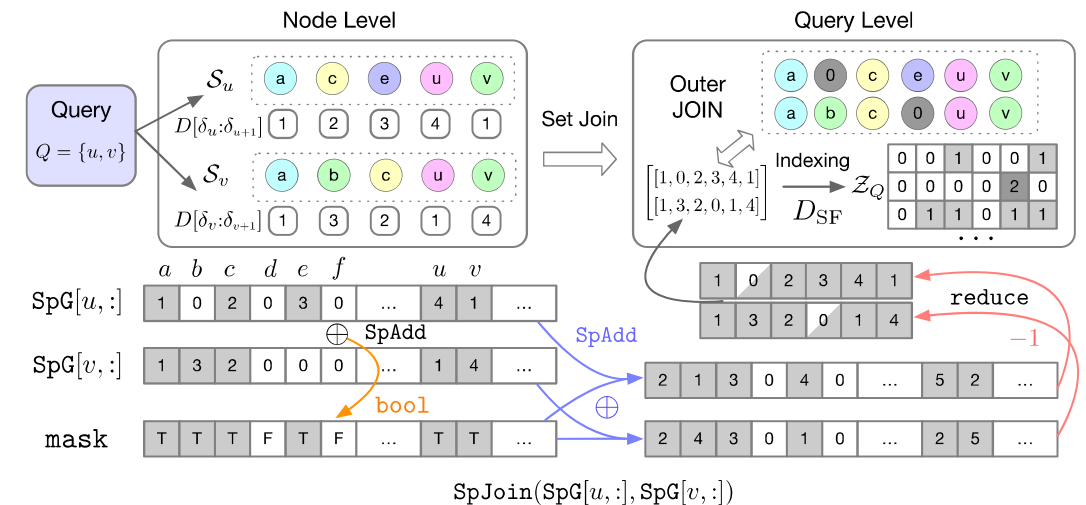
Walk-based Storage

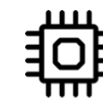
- Fast neighbor access with pointer
- Reduce feature overlap



Sparse Join Operator

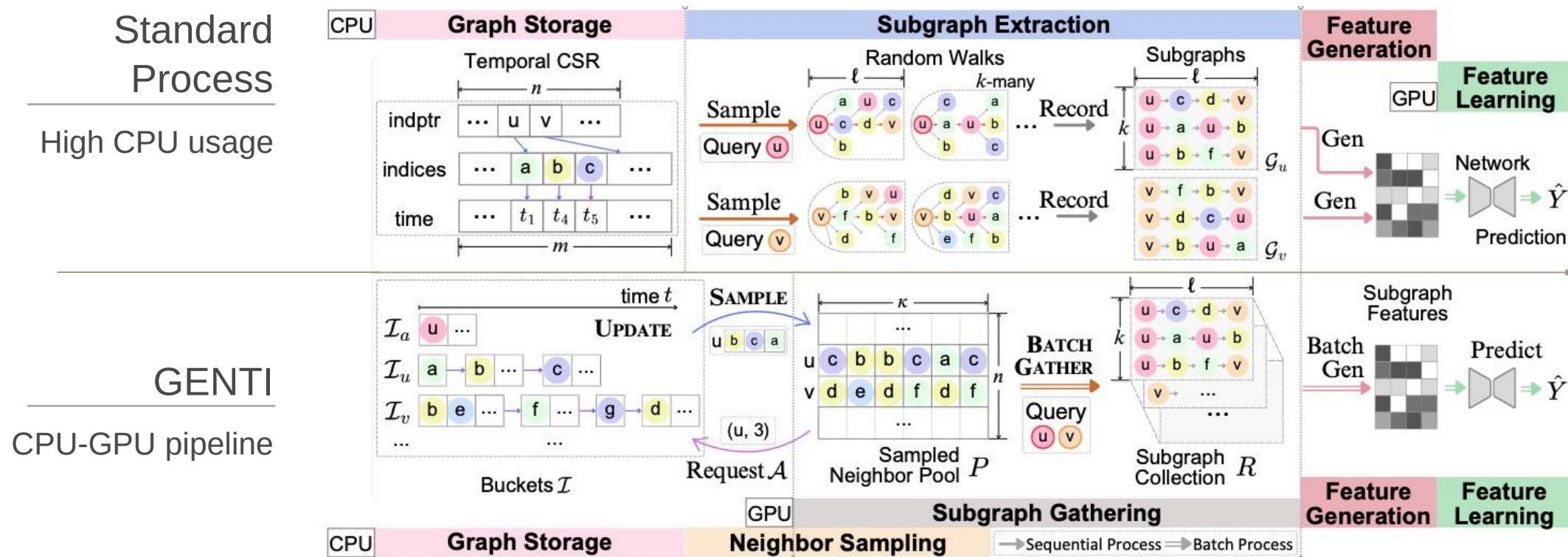
- Remove duplicate nodes
- Reduce sparse storage

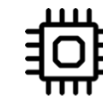




Generation: GENTI [VLDB'24]

- Dynamic Subgraph Extraction: streaming of subgraph sampling & join

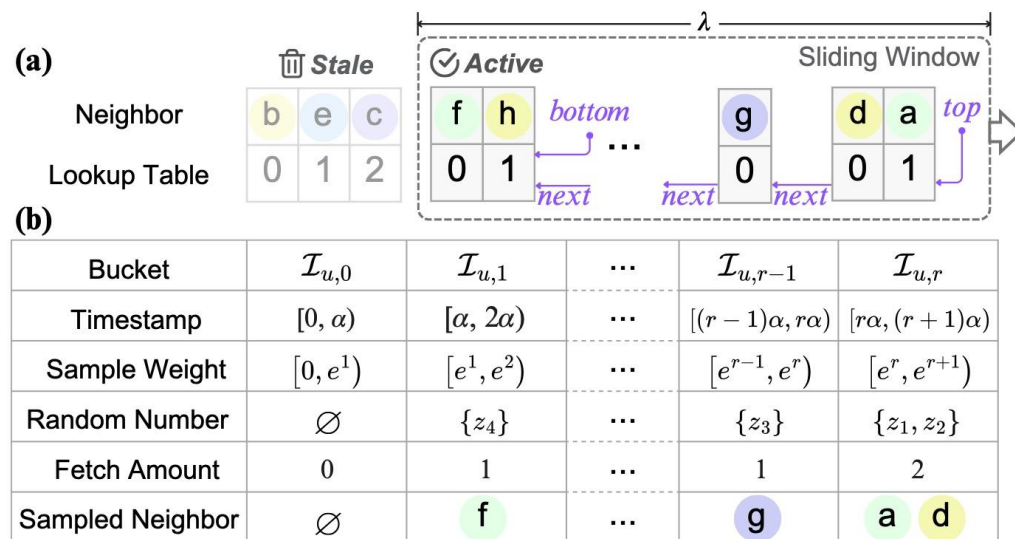




Generation: GENTI [VLDB'24]

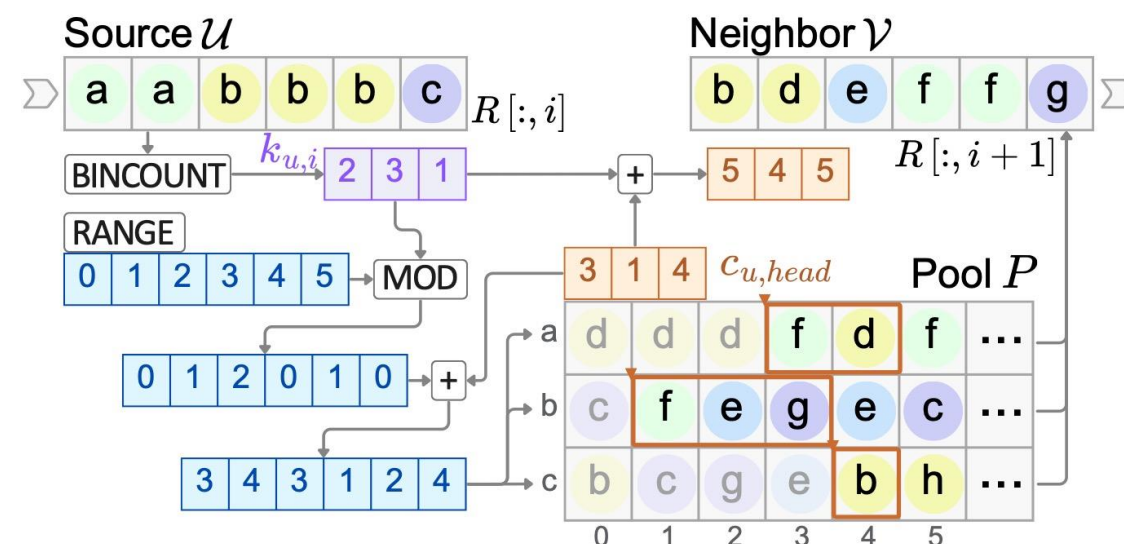
Time-based Storage

- Fast sampling across buckets
- Constrain time by sliding window



Batch Join GPU Operator

- Dense operations on GPU
- Complexity: reduce from L to $\sqrt{L}$



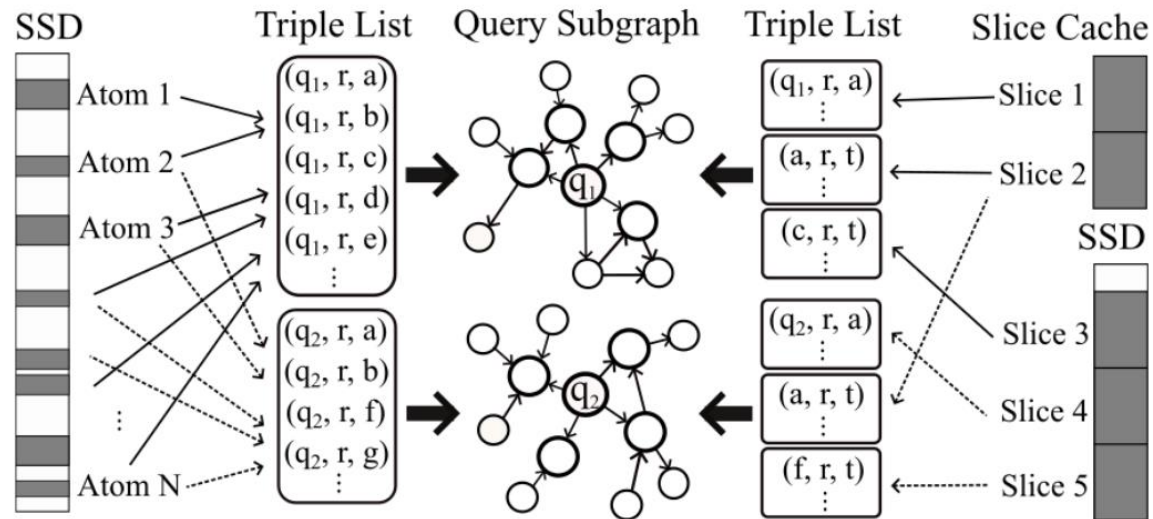


Slicing: TIGER [VLDB'24]

- Subgraph extraction on heterogeneous graph: based on edge triples
- Bottleneck: element lookup on disk
- Slicing: store elements of same triples together ← NP-Hard

Simple Extraction

- Random disk access



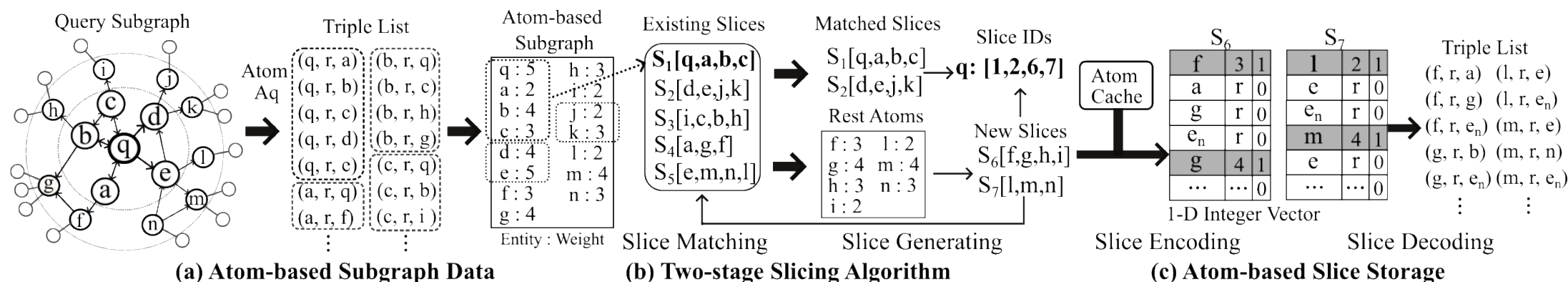
Slice-based Extraction

- Sequential disk access
- Slicing overhead

Slicing: TIGER [VLDB'24]



- Slicing: match-then-generate to reduce overlap storage
- Caching: recently accessed elements in memory



Subgraph Extraction: Takeaways

Partition

G3 [SIGMOD'23]

- 3 Parallel enhancements
- Communication overhead persist under full-graph training

Generation

SUREL [VLDB'23]

GENTI [VLDB'24]

- Generated subgraphs may be more expressive
- Generation and gathering overhead

Slicing

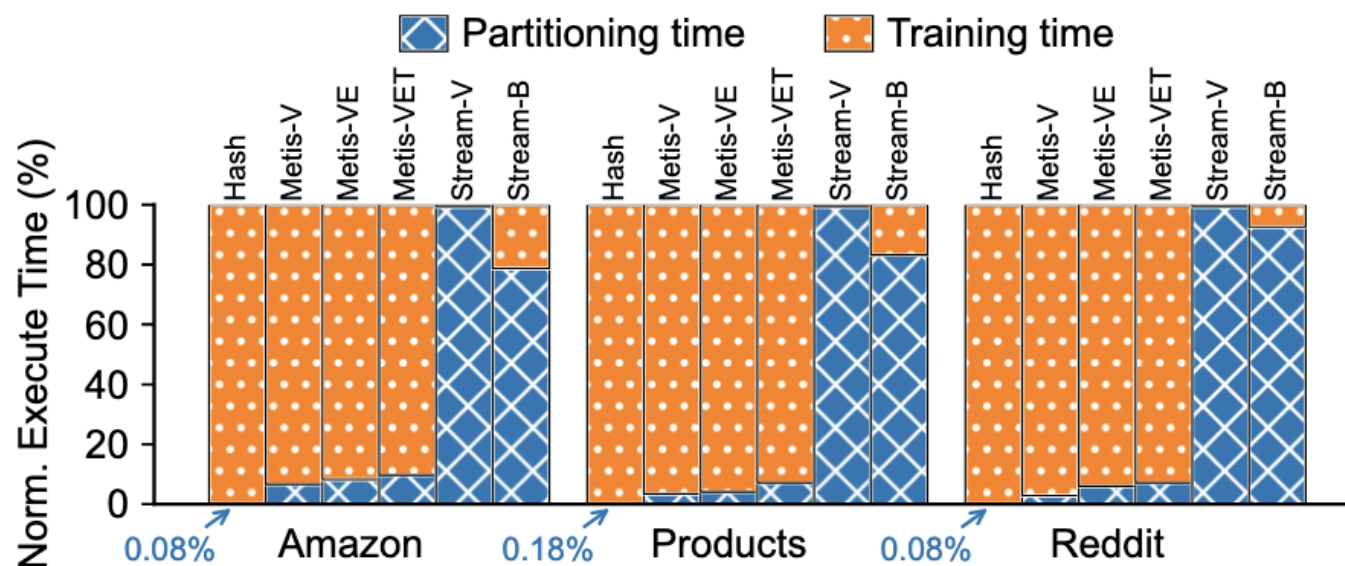
TIGER [VLDB'24]

- On-disk subgraph extraction
- 2-stage slicing and caching for better local access

Subgraph Extraction: Evaluation

Model Operations

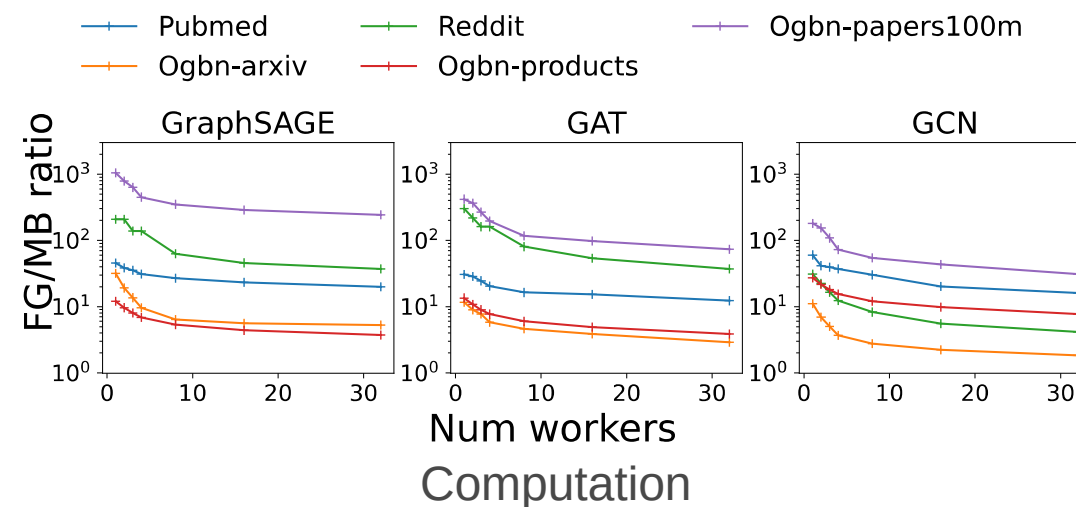
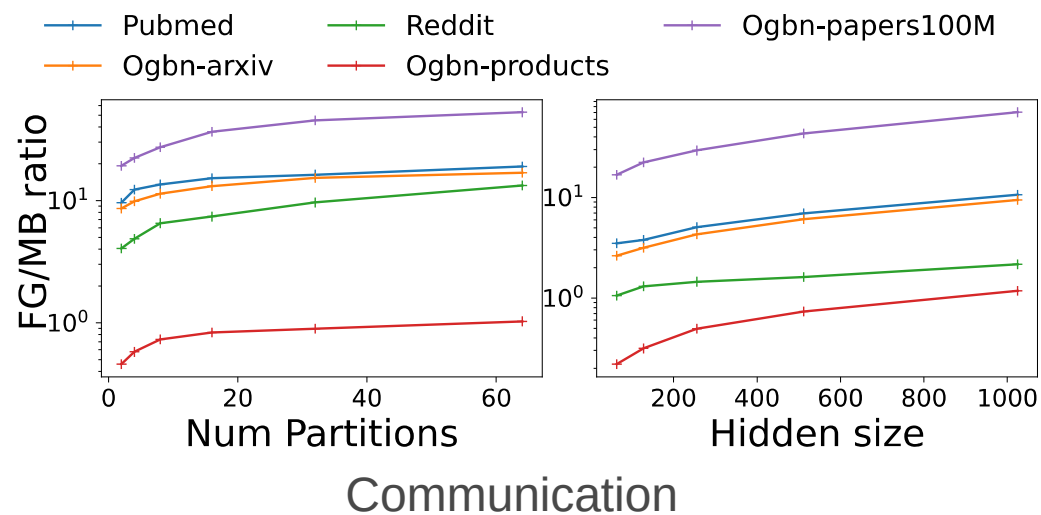
- Hash: constantly fastest & insignificant
- Metis family: fast but less scalable
- Streaming family: slow & less scalable



Subgraph Extraction: Evaluation

Training Schemes

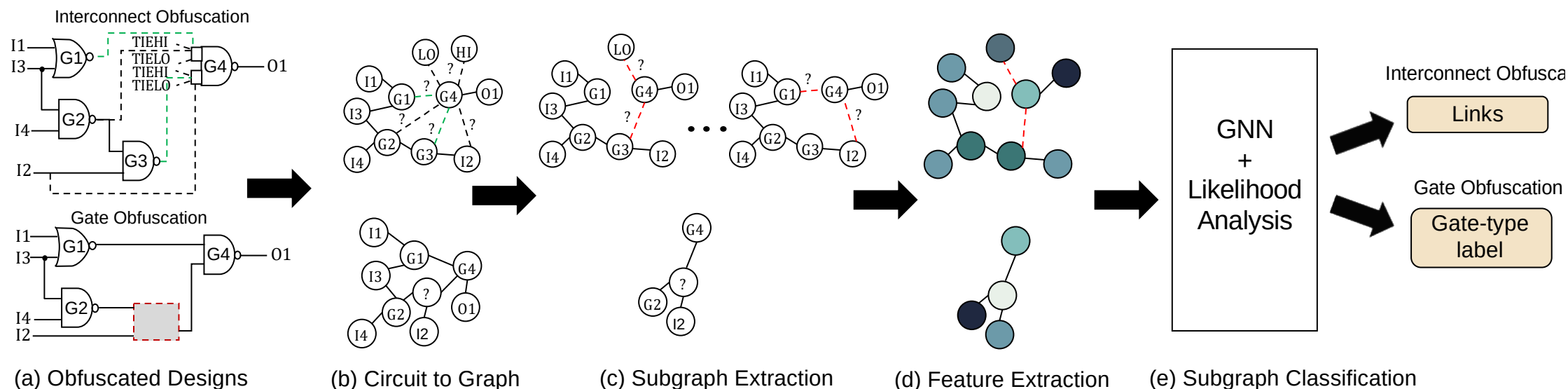
- Mini-batch training is more scalable than full-graph training
- Mini-batch + local sampling is effective in reducing computation



Subgraph Extraction: Applications

Hardware Obfuscation Analysis

- Graph: integrated circuits logic gates
- Task: identify obfuscated gates & interconnections



Advances in Designing Scalable GNNs

Node-wise Similarity

Outline

Node-wise Similarity

Personalized PageRank GBP [Chen et al | NeurIPS'20] & SCARA [Liao et al | VLDB'22]

Pair-wise Similarity SIGMA [H Liu et al | ICDE'25]

Hub Labeling CFGNN [Preprint] | HubGT [Preprint]

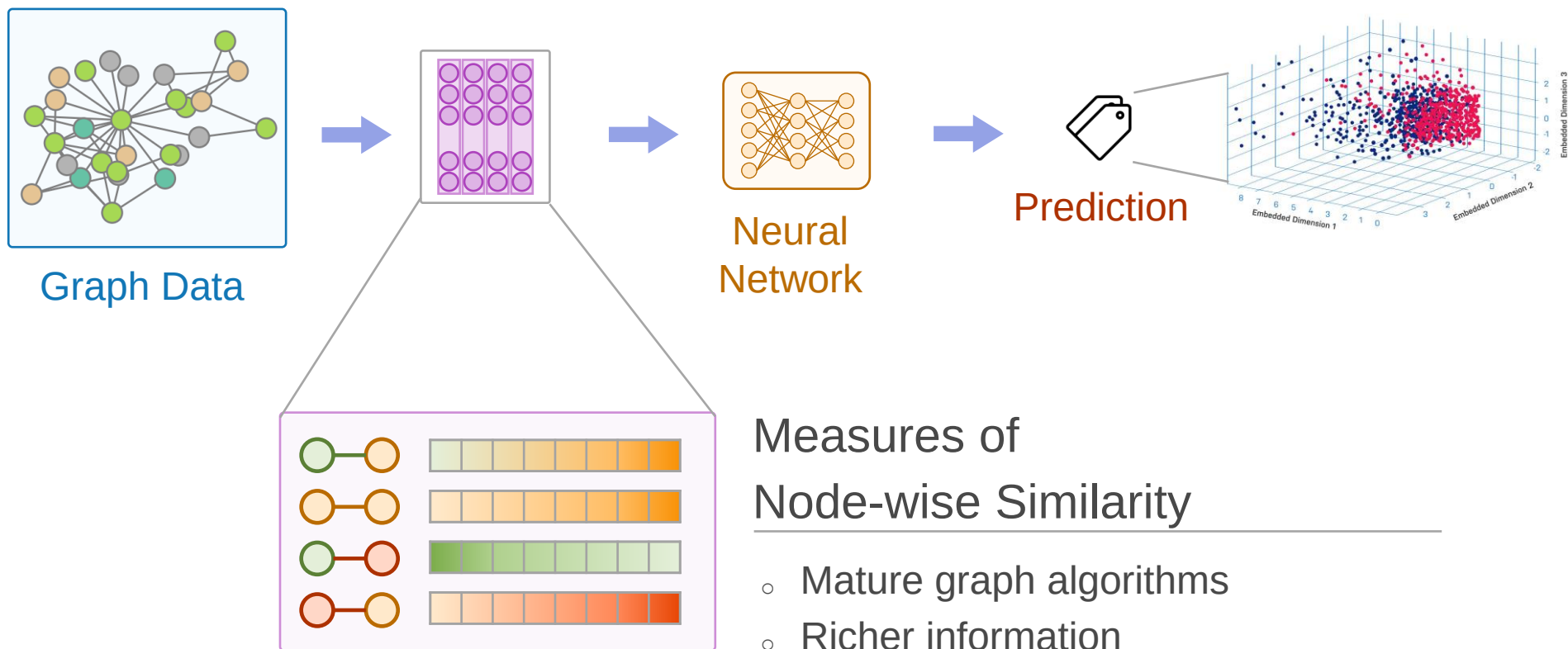
Application: Recommendation Popularity

Spectral Embedding

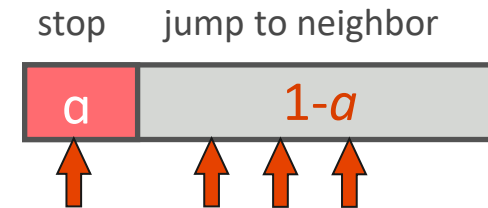
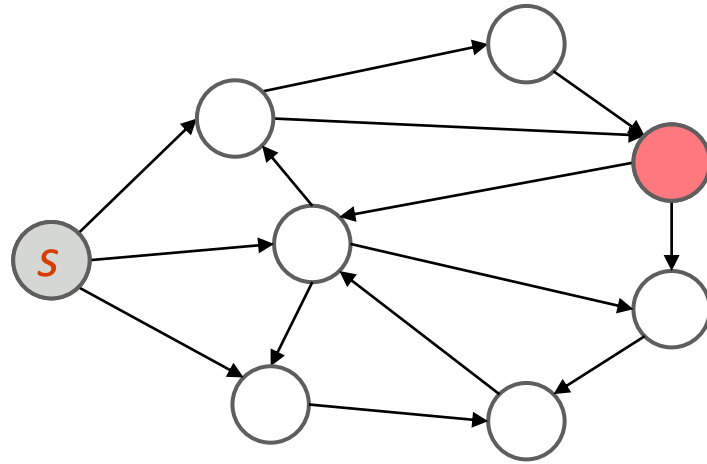
Graph Simplification

Future Direction

Node Similarity: Schema

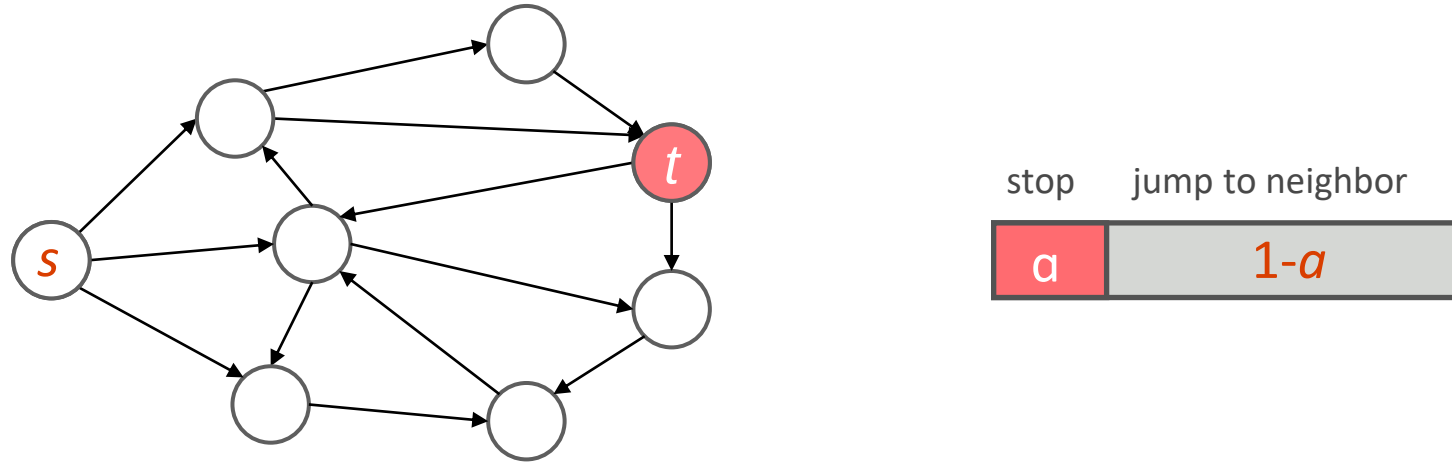


Personalized PageRank (PPR)



- A *α -decay random walk*:
 - Stop with α probability at each step
 - If not stop, randomly jumps to one of its out-neighbors

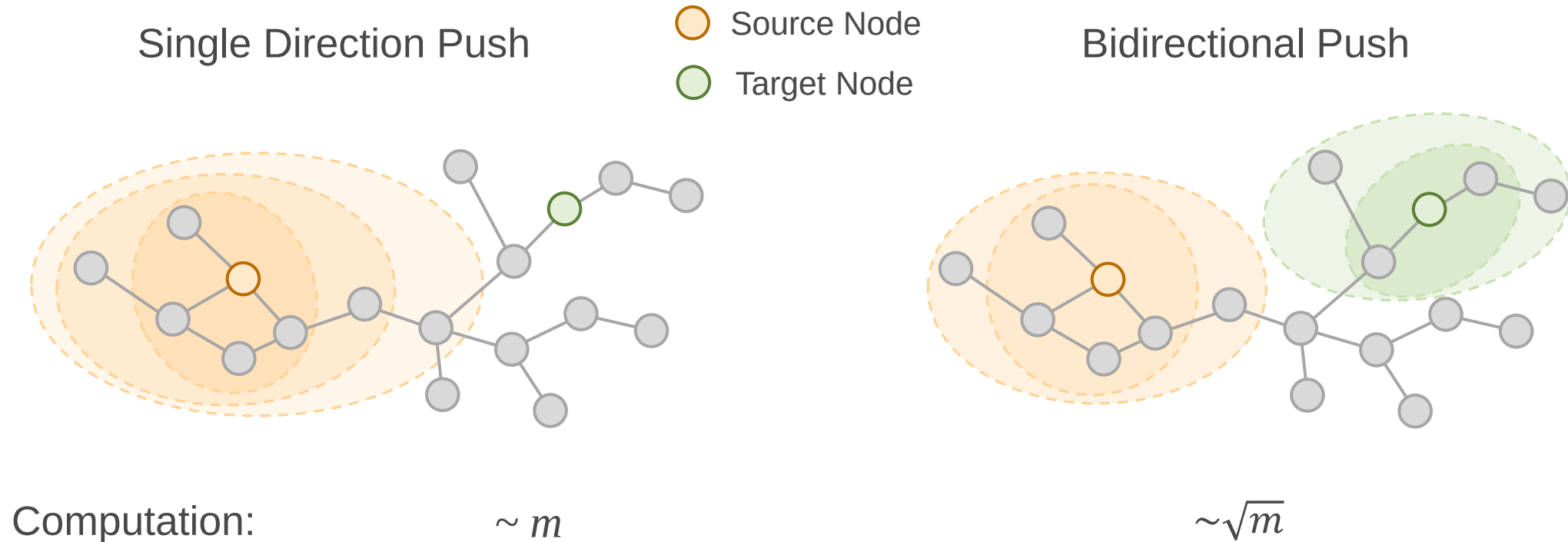
Personalized PageRank (PPR)



- Random walk interpretation of $\text{PPR}(\textcircled{s}, \textcircled{t})$:
 - probability that a random walk from $\textcircled{s}$ stops at $\textcircled{t}$

Personalized PageRank (PPR)

- Bidirectional push for faster node-wise computation

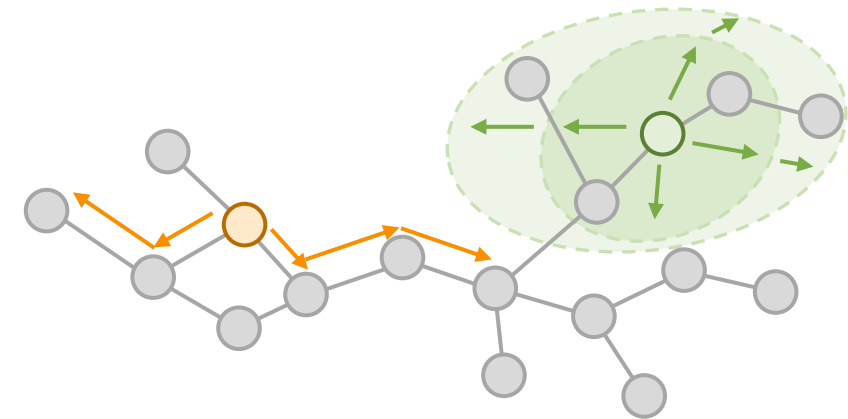




PPR: GBP [NeurIPS'20]

- Bidirectional propagation
 - Forward: Random Walk from ego nodes
 - Backward: Reverse Push from feature vectors
- Decoupled propagation and network
- Reduced propagation complexity

$$O(L\sqrt{Lm \log(Ln)} / \varepsilon)$$



→ Forward: Random Walk

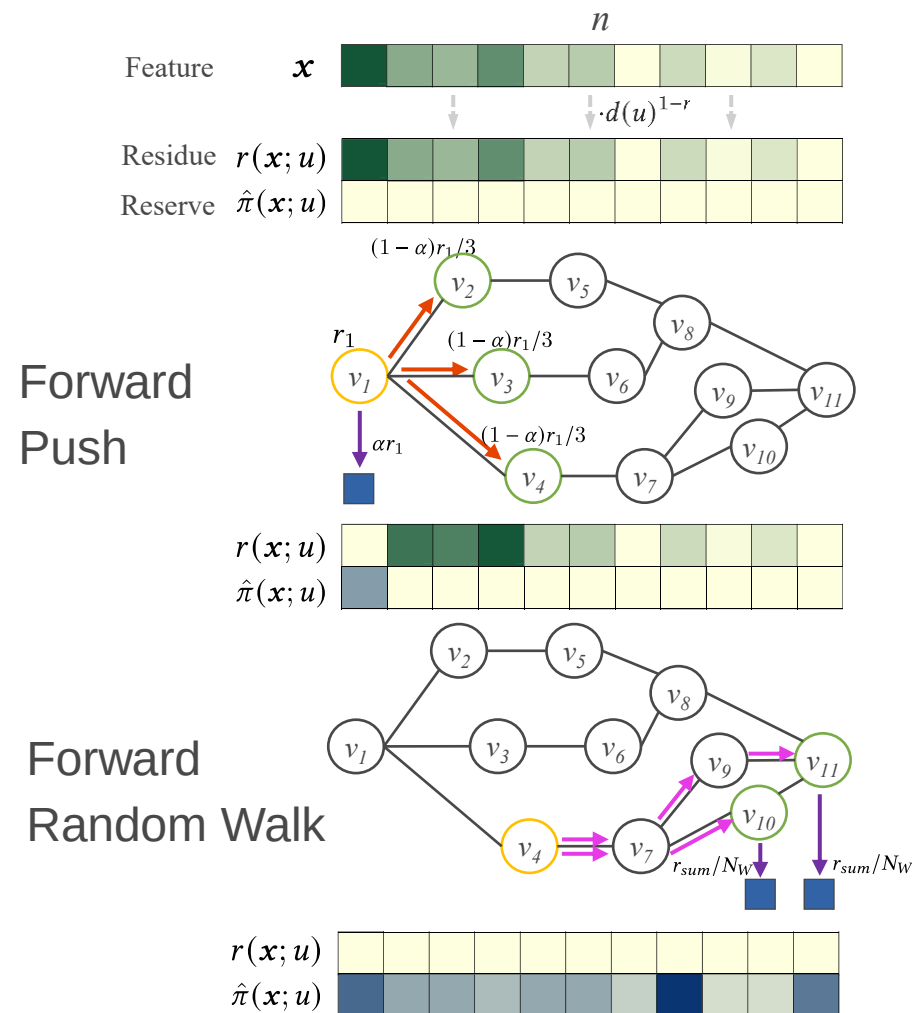
→ Backward: Reverse Push



PPR: SCARA [VLDB'22]

- Feature-oriented: start from each node attribute vector
- Pure forward scheme
 - Insignificant push: control by threshold
 - Significant push: compensate by random walk
- Reduced propagation complexity

$$O(\sqrt{m \log(n)} / \lambda)$$



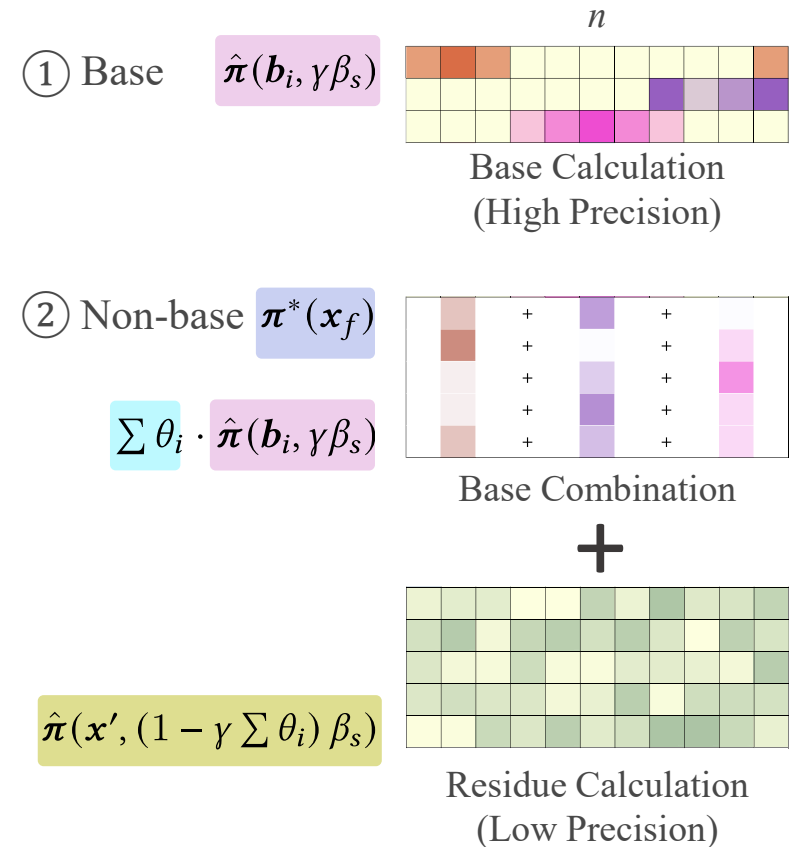


PPR: SCARA [VLDB'22]

- Conventional PPR: each node possesses a scalar value
- PPR in GNN: each node possesses a vector (feature vector)
- There are overlaps among feature dimensions



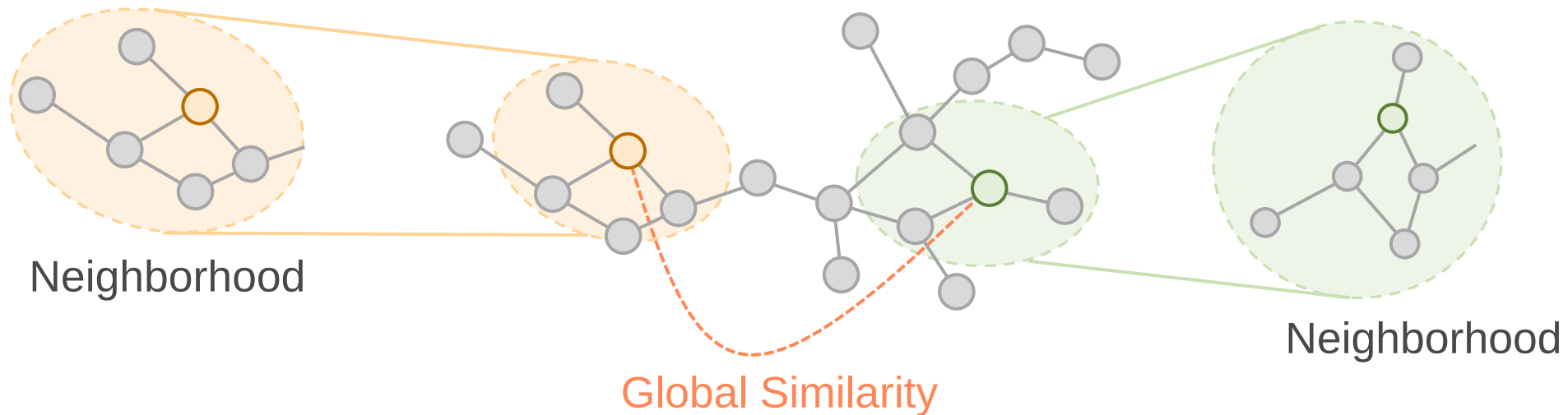
We can reuse PPR calculations among feature dimensions!





Pair-wise Similarity

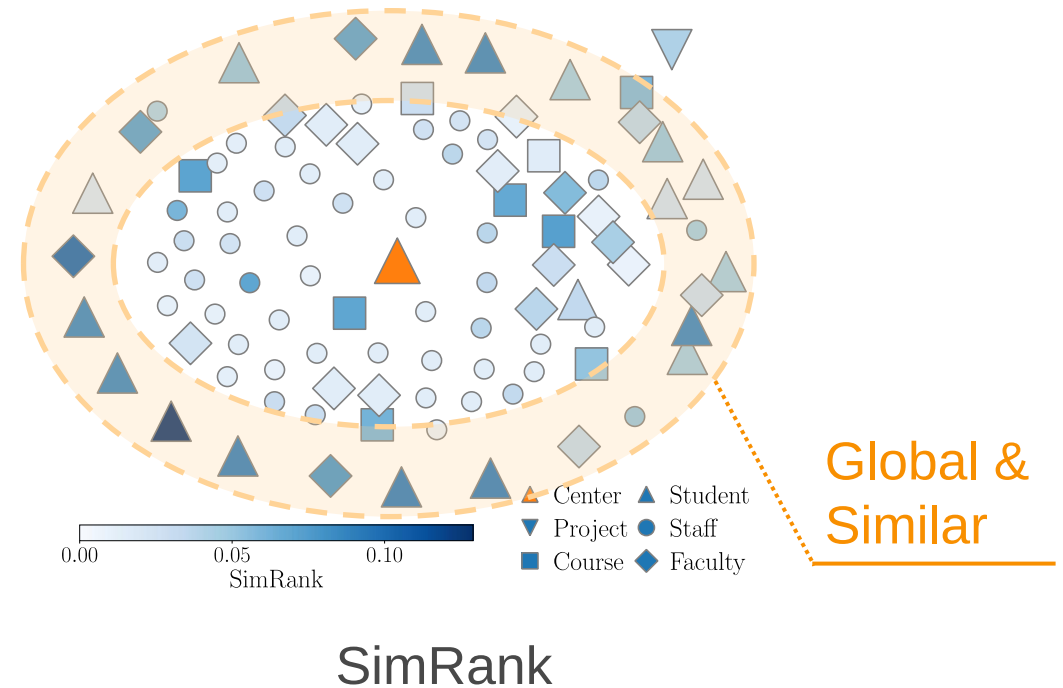
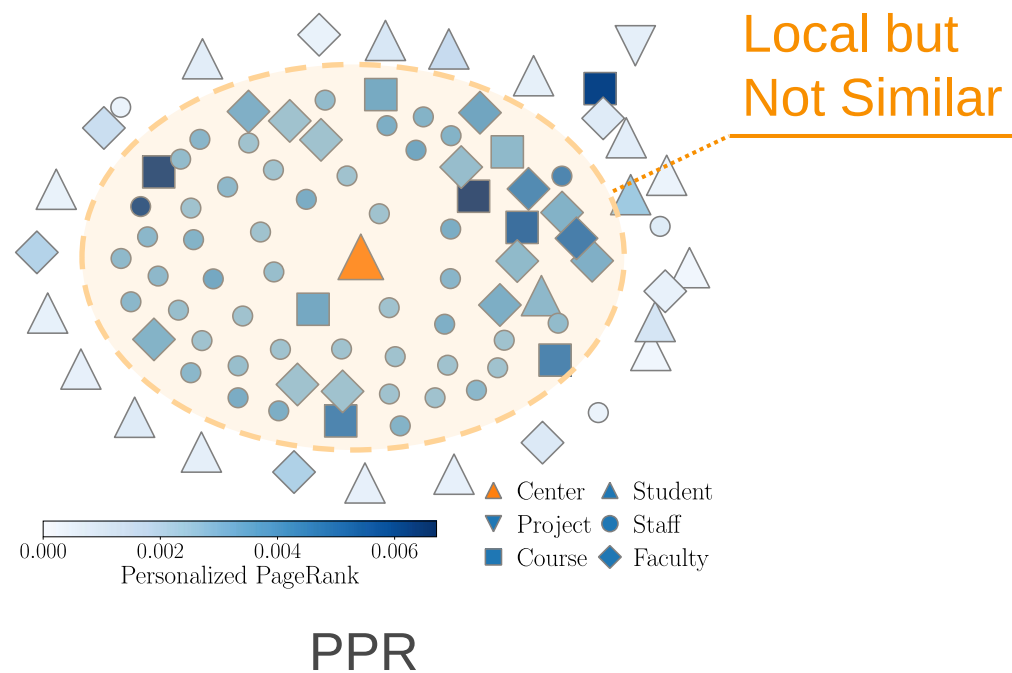
- SimRank: a measure of node similarity based on neighborhood topology
- Global Similarity: distant node pair sharing the similar topology





Pair-wise Similarity: SIGMA [ICDE'25]

- Global similarity: address heterophily by graph metric





Pair-wise Similarity: SIGMA [ICDE'25]

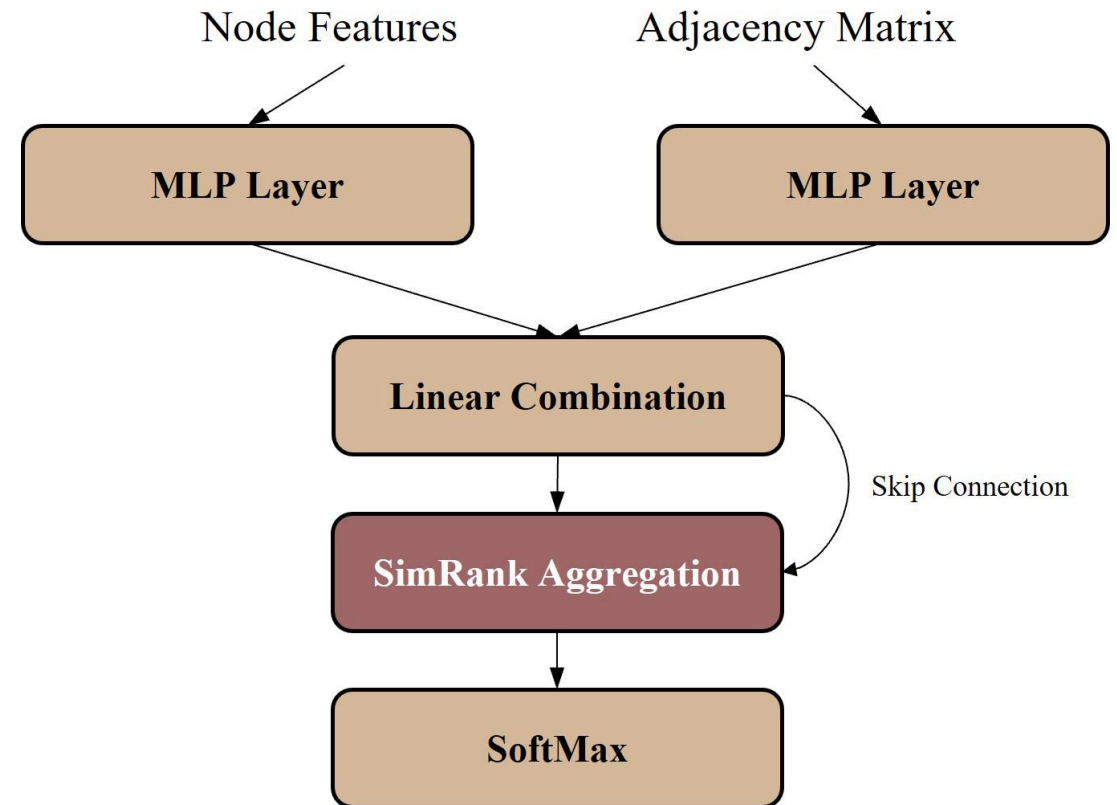
- Combining **attribute** and **topological** representations:

$$\mathbf{H} = \text{MLP}_H (\delta \cdot \mathbf{H}_X + (1 - \delta) \cdot \mathbf{H}_A).$$

- Combining **local** and **global** representations:

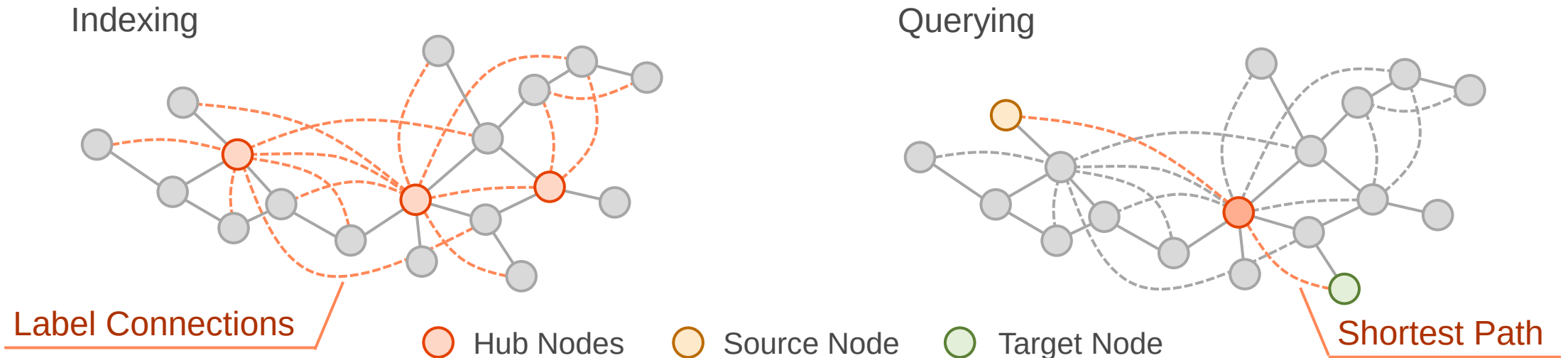
$$\text{AGG} : \hat{\mathbf{Z}}_u = \sum_{v \in V} \mathbf{S}(u, v) \cdot \mathbf{H}_v,$$

$$\text{UPD} : \mathbf{Z}_u = (1 - \alpha) \cdot \hat{\mathbf{Z}}_u + \alpha \cdot \mathbf{H}_u,$$



Hub Labeling

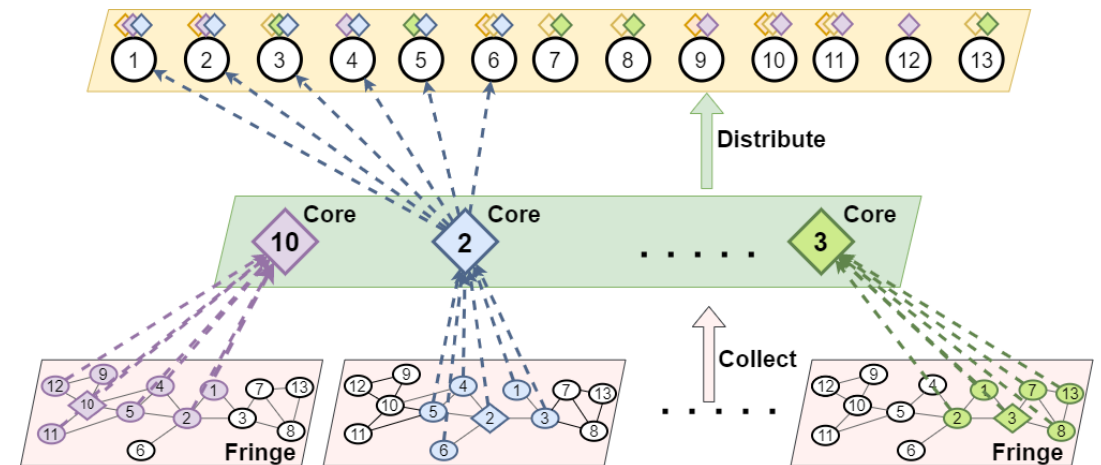
- Hub Labeling: adding new links to selected hub nodes, so that distant node pairs can be easily reached through hubs
- Shortest Path: reachability and distance can be computed through labels





Hub Labeling: CFGNN [Preprint]

- Hub labeling: identify hub nodes as the center of node clusters
- Distinctive message-passing:
 - Fringe nodes -> Hub nodes: collect
 - Hub nodes -> Fringe nodes: distribute





Hub Labeling: HubGT [Preprint]

- Hub Labeling: compute node-pair Shortest Path Distance (SPD)
- Positional Encoding: emphasize important pairs in Graph Transformer

$$\tilde{H}_i = \text{softmax} \left(\frac{Q_i K_i^\top}{\sqrt{F_K}} + P \right) V_i,$$

- Complexity: $O(n^2)$ (full graph)
- $O(s^2)$ (within subgraph)

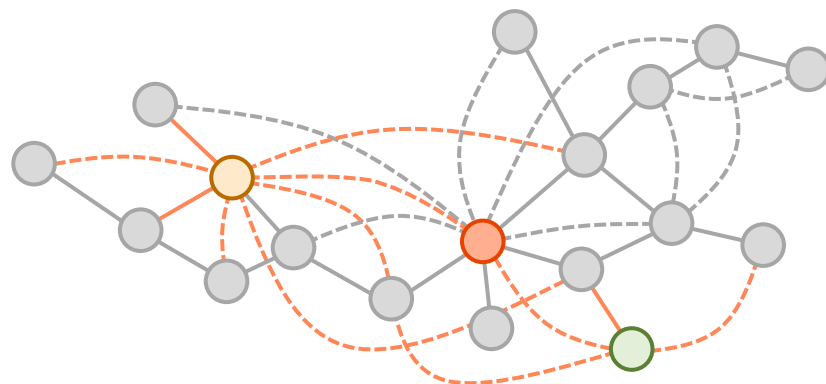


Hub Labeling: HubGT [Preprint]

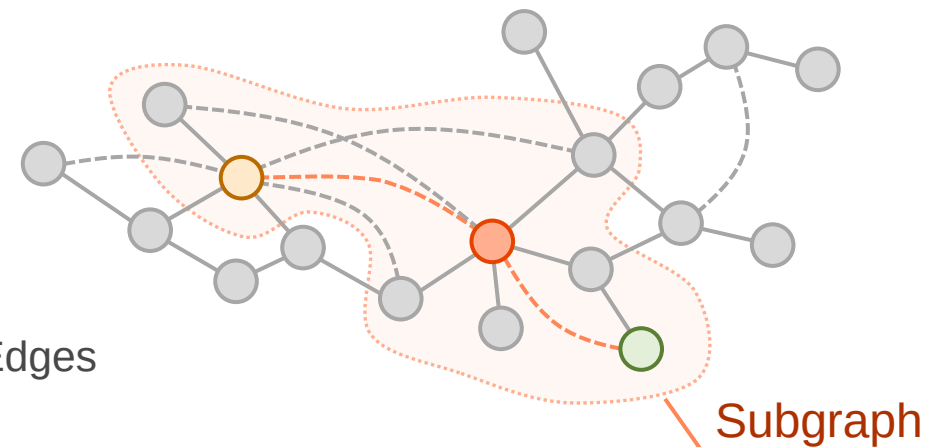


- Subgraph SPD: fast querying any node pairs within subgraph of a node

Conventional SPD



Subgraph SPD



○ Ego Nodes
--- Accessed Edges

Query Time

$O(s)$

$O(1)$

Index Size

$O(ns^2)$

$O(ns^2)$

Node Similarity: Takeaways

PPR

GBP [NeurIPS'20]

SCARA [VLDB'22]

- Similar to graph convolution with faster computation
- Scalar feature vector

SimRank

SIGMA [ICDE'25]

- Global similarity between distant nodes
- Fast push computation

Hub Labeling

CFGNN [Preprint]

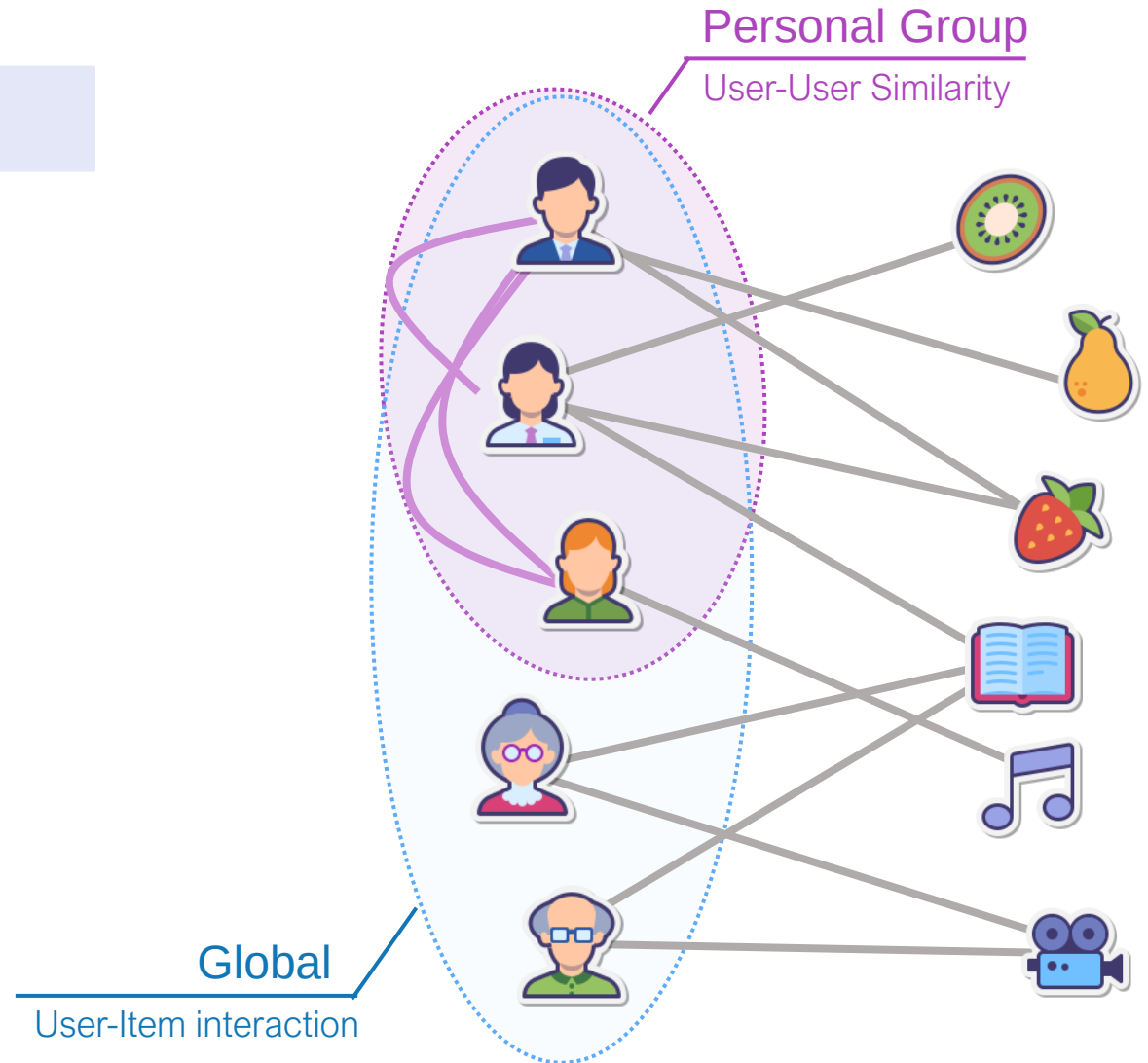
HubGT [Preprint]

- Rewiring with core hub nodes
- Applicable to MPNN & GT

Node Similarity: Applications

Debiasing Recommendation

- Graph: user-item interaction
- Task: unbiased recommendation
- **Global popularity:** biased to top popular items
- **Personal popularity:** based on user similarity



Advances in Designing Scalable GNNs

Spectral Embedding

Outline

Node-wise Similarity

Spectral Embedding

Channel Combination LD^2 [Liao et al | NeurIPS'23]

TER [Wang et al | KDD'23] & S3GCL [Wan et al | ICML'24]

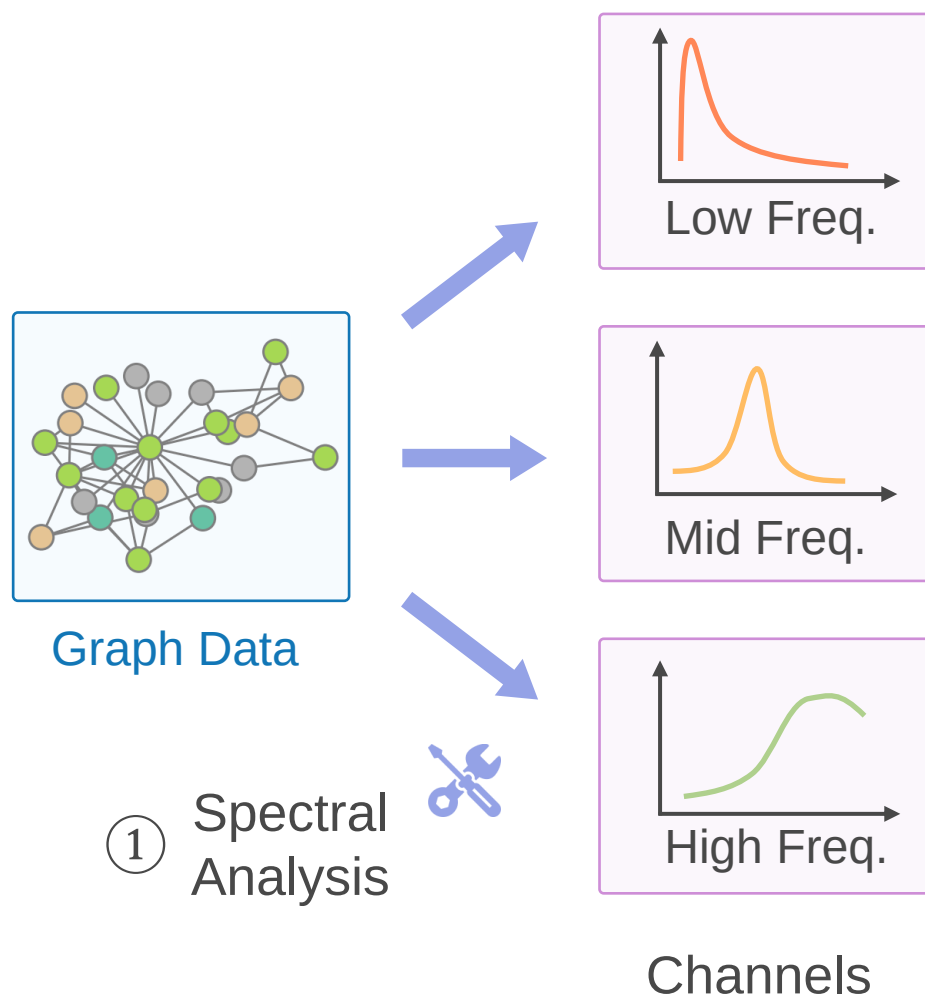
Adaptive Selection UniFilter & AdaptKry [Hunag et al | ICML & WWW'24]

Application: Image Filtering

Graph Simplification

Future Direction

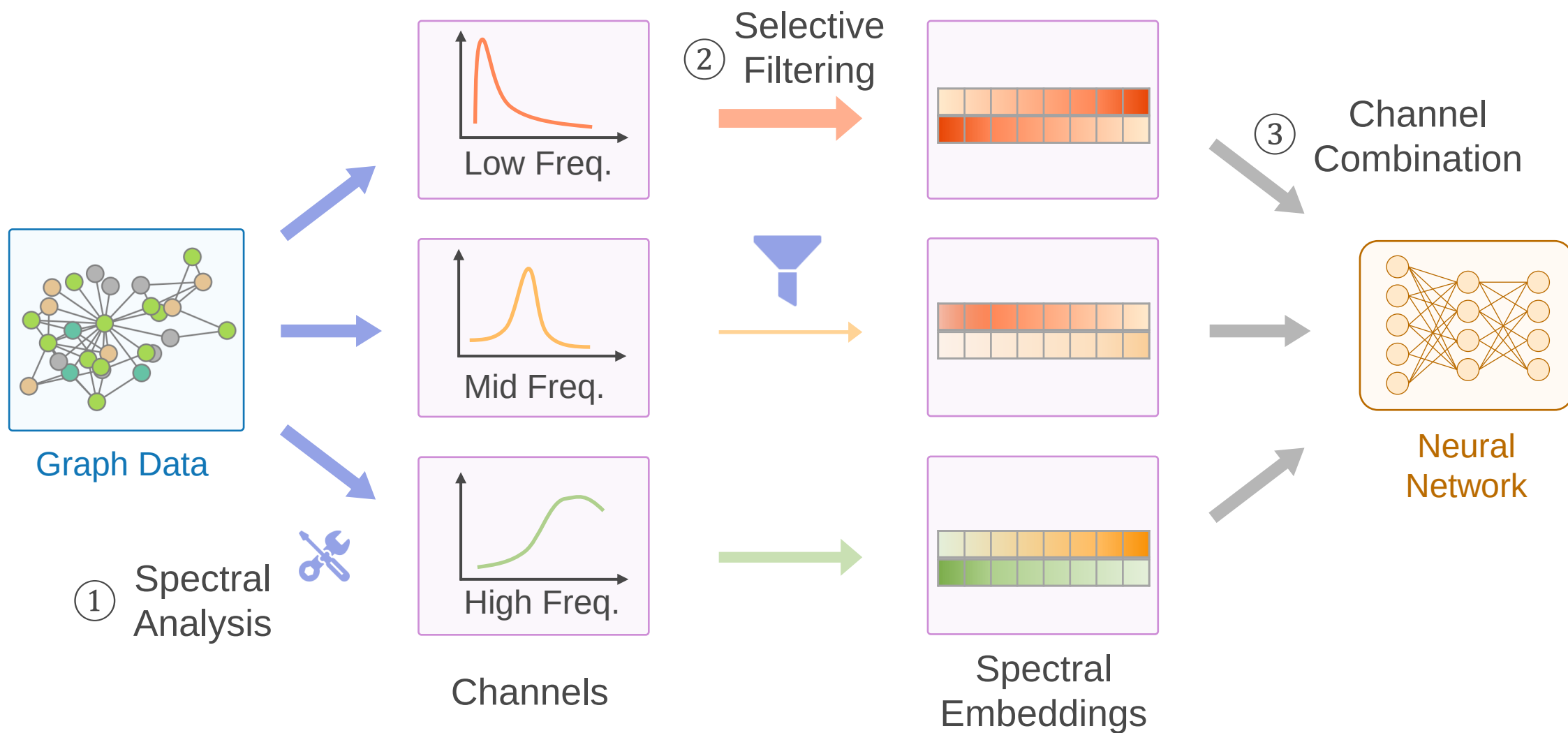
Spectral Embedding: Schema



Spectral Analysis

- Comprehensive graph information
- Explainable & controllable
- Separated graph computation

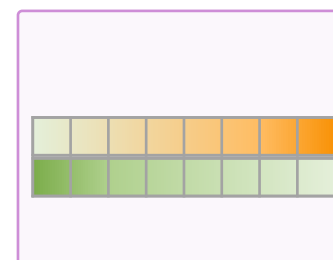
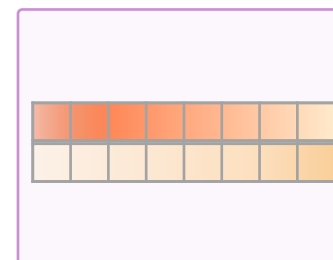
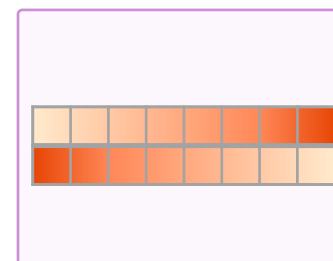
Spectral Embedding: Schema



Channel Combination



How to select & combine useful channels?

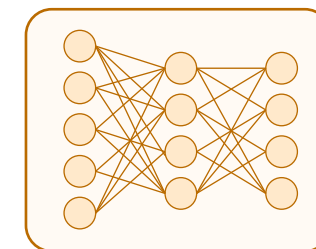


Spectral
Embeddings

Spectral Embedding

③

Channel
Combination

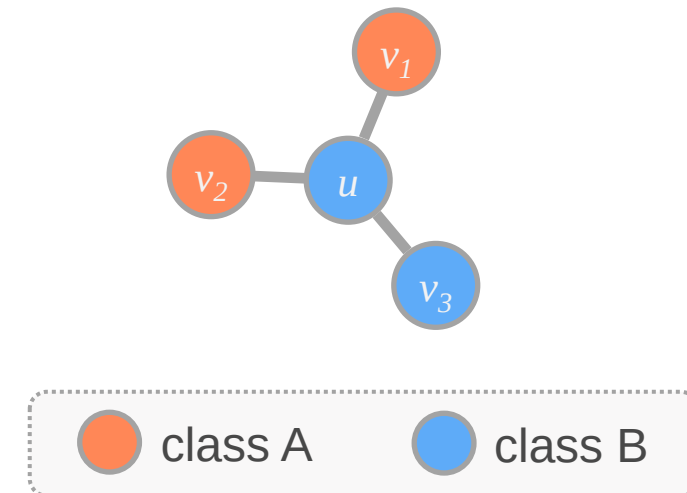
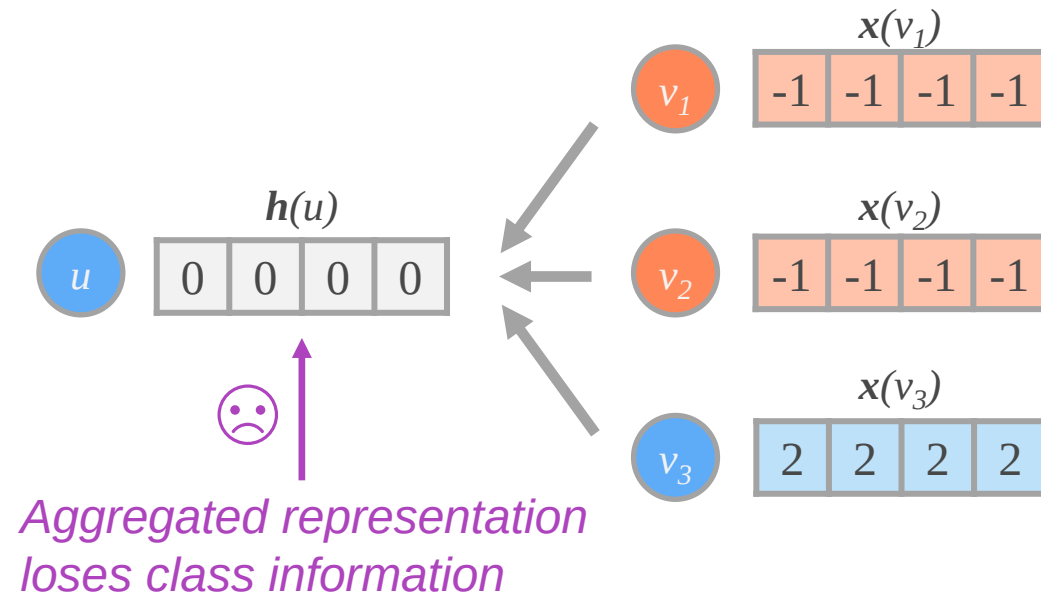


Neural
Network



Channel Combination: LD^2 [NeurIPS'23]

- Heterophily: 1-hop message passing fails





Channel Combination: LD^2 [NeurIPS'23]

- Heterophily: specifically focus 2-hop connections

2-hop Adjacency:

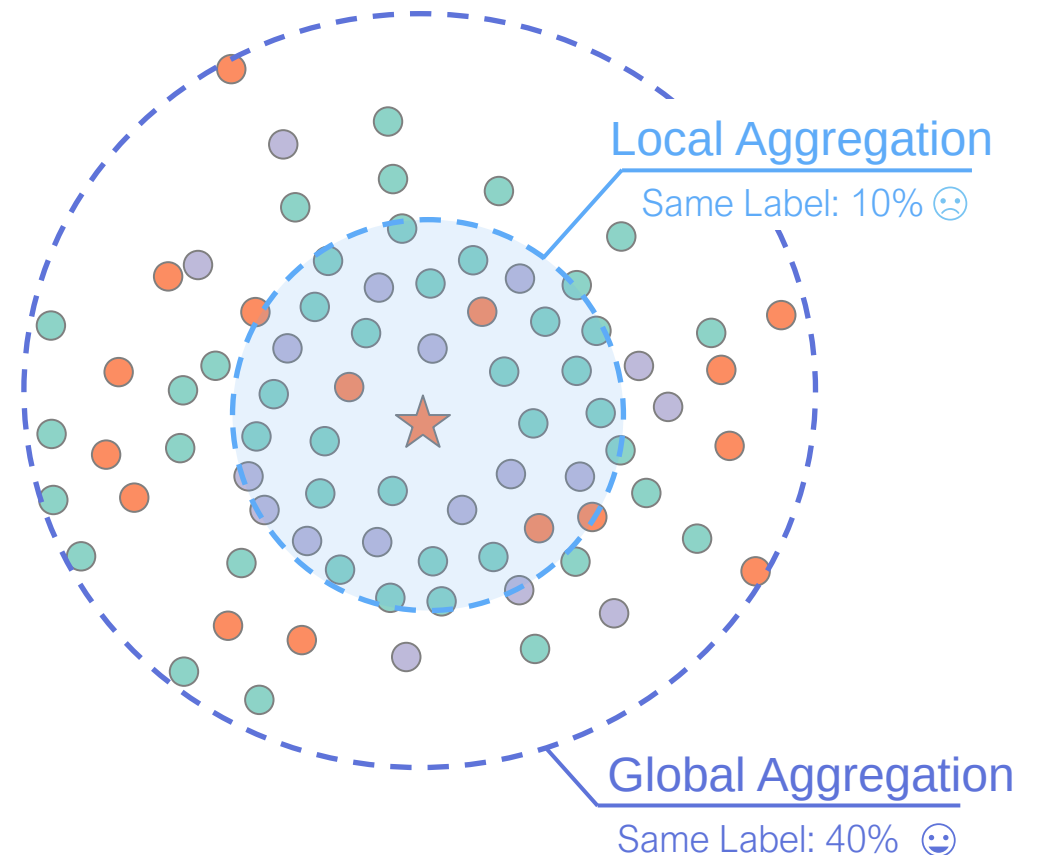
$$\mathbf{P}_A = \phi(\mathbf{A}^{2L} \cdot \mathcal{N})$$

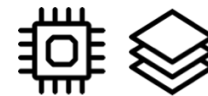
Constant Feature:

$$\mathbf{P}_{X,L2} = \sum_{l=1}^L \bar{\mathbf{A}}^{2l} \cdot \mathbf{X}$$

Inverse Feature:

$$\mathbf{P}_{X,H} = \sum_{l=1}^L \tilde{\mathbf{L}}^l \cdot \mathbf{X}$$





Channel Combination: LD^2 [NeurIPS'23]

Spatial

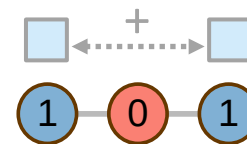
Spectral

2-hop Adjacency:

$$\mathbf{P}_A = \phi(\mathbf{A}^{2L} \cdot \mathcal{N})$$

$$\mathbf{P}_A \cdot \mathbf{P}_A^\top \approx \mathbf{A}^2$$

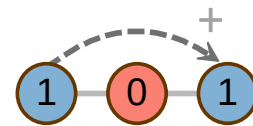
Low-rank approximation



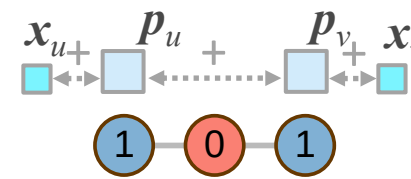
Structure smoothing

Constant Feature:

$$\mathbf{P}_{X,L2} = \sum_{l=1}^L \bar{\mathbf{A}}^{2l} \cdot \mathbf{X}$$



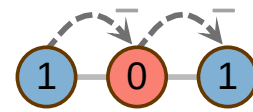
Long-range similarity



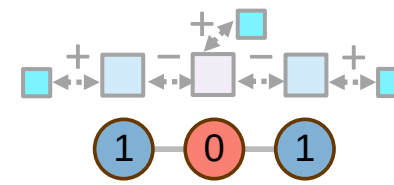
Feature smoothing

Inverse Feature:

$$\mathbf{P}_{X,H} = \sum_{l=1}^L \tilde{\mathbf{L}}^l \cdot \mathbf{X}$$



Fine-grained details

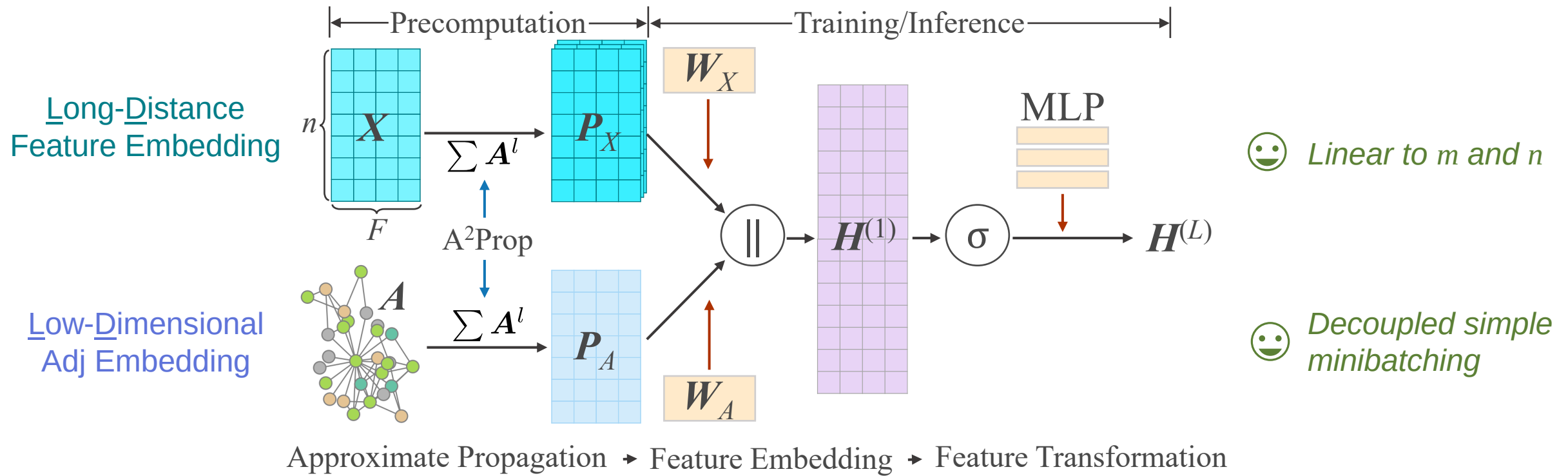


Structure difference



Channel Combination: LD^2 [NeurIPS'23]

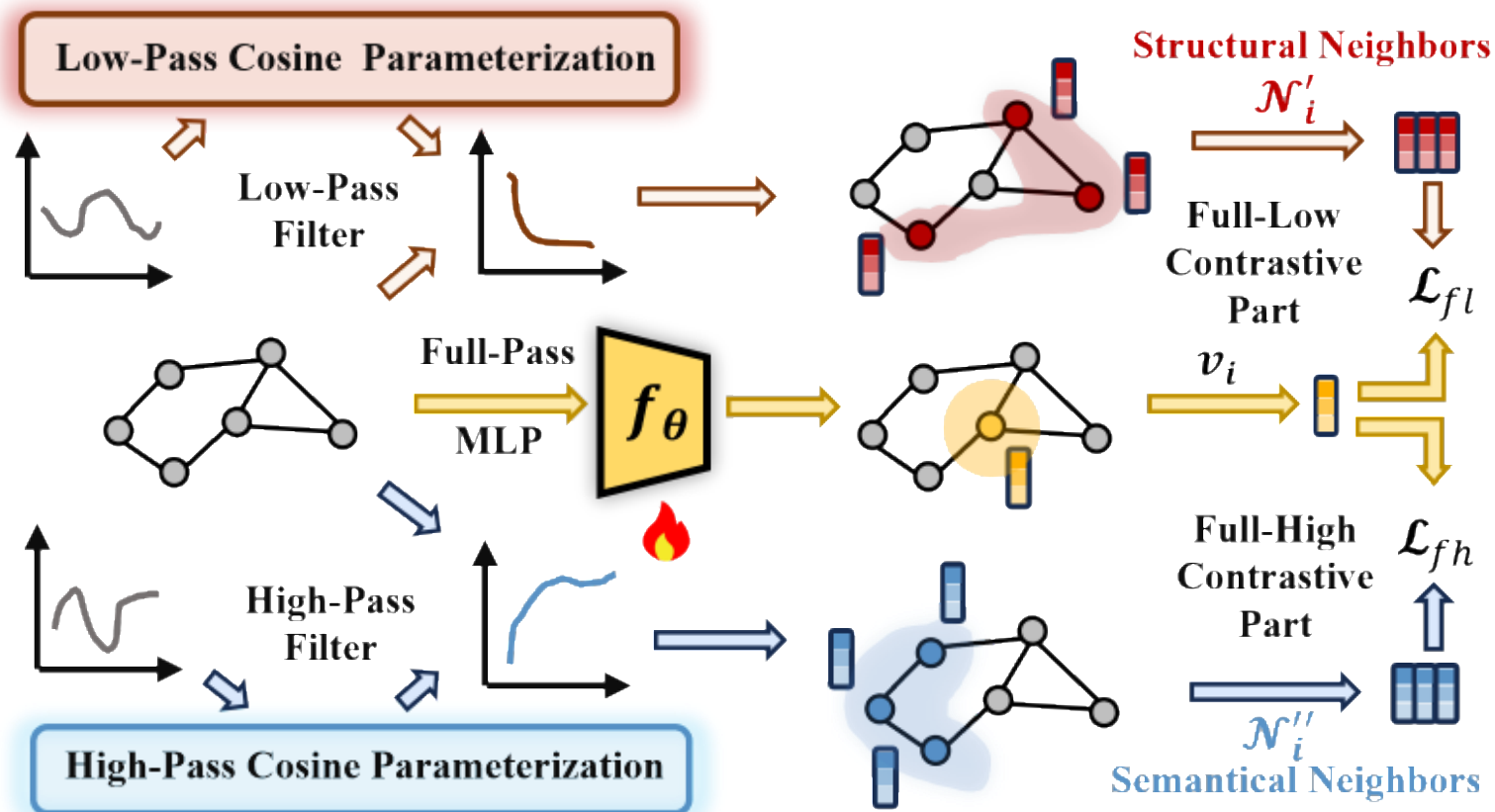
- Decoupled spectral embedding





Channel Combination: S3GCL [ICML'24]

- Contrastive learning: difference between low and high frequency signals





Channel Combination: TER [KDD'23]

- Representation based on edge-wise proximity:

$$\Pi \approx \mathbf{Z} \cdot \mathbf{Z}^\top$$

$$\min_{\mathbf{Z} \in \mathbb{R}^{m \times k}} \|\Pi - \mathbf{Z}\mathbf{Z}^\top\|_F^2.$$

Representation
 $m \times k$

$$\Pi = \sum_{t=0}^{\infty} w(t) \cdot \mathbf{P}^t$$

Proximity
⚠ Dense

Edge Transition
 $m \times m$

- Eigen-decomposition:

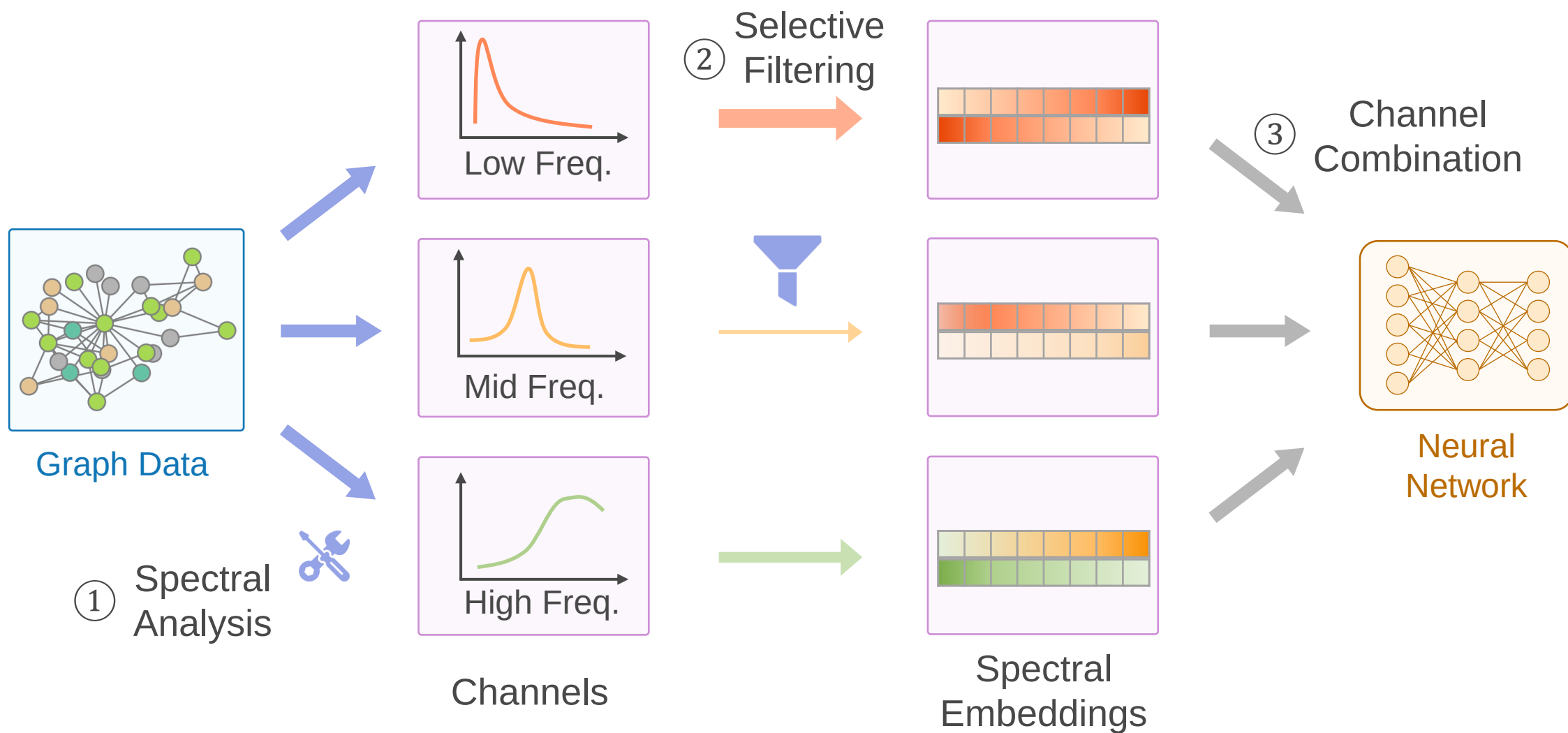
$$\mathbf{Z} = \mathbf{U} \sqrt{\sum_{t=0}^{\infty} w(t) \Lambda^t}$$

$$\mathbf{P} = \mathbf{U} \Lambda \mathbf{U}^\top$$

$$\Pi = \mathbf{U} \cdot \left(\sum_{t=0}^{\infty} w(t) \Lambda^t \right) \cdot \mathbf{U}^\top$$

Adaptive Selection

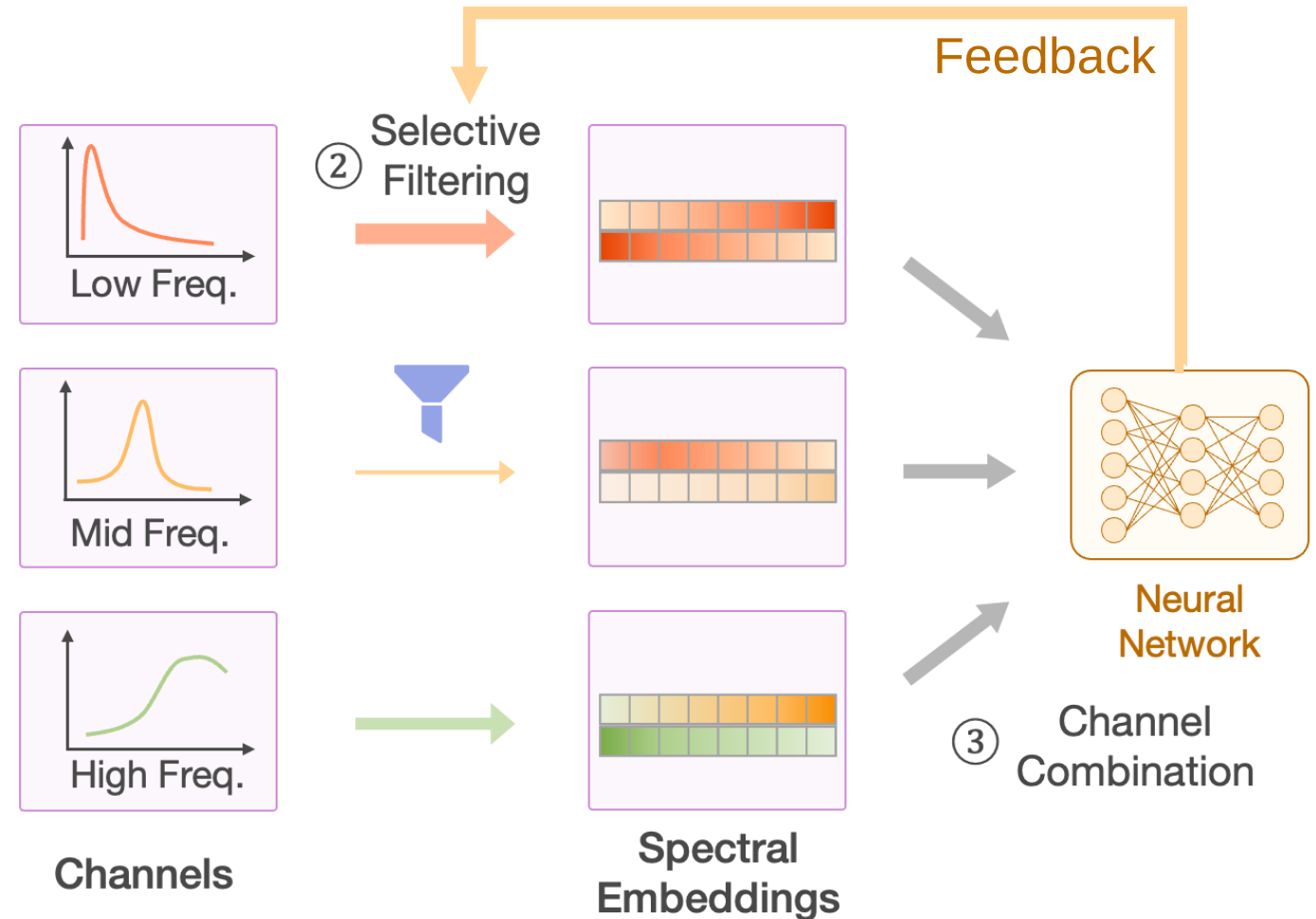
Spectral Embedding



Adaptive Selection



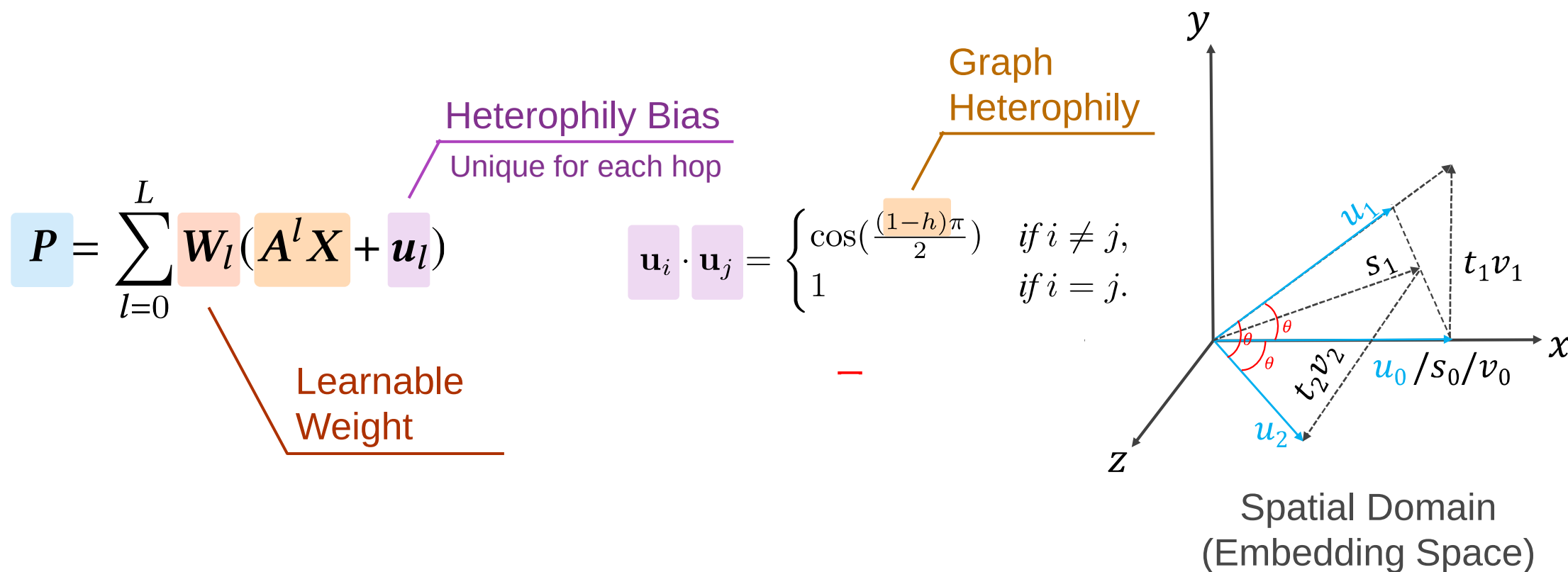
How to filter based on learning feedback?





Adaptive Selection: UniFilter [ICML'24]

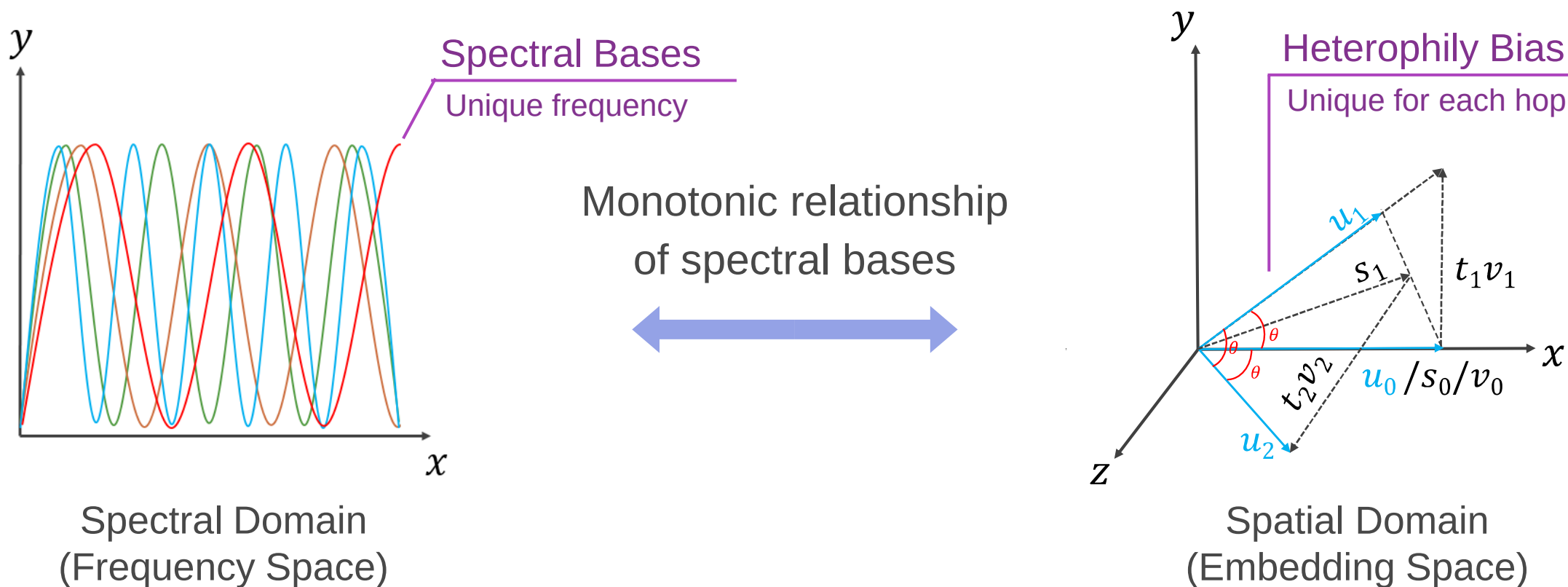
- Spatial computation





Adaptive Selection: UniFilter [ICML'24]

- Spectral explanation





Adaptive Selection: AdaptKry [WWW'24]

- Understanding graph propagation as heat diffusion:

Laplacian

$$A = I - L$$

$$H_{t+1} = (I - L)H_t \quad \xrightarrow{\quad} \quad \frac{dH_t}{dt} = -LH_t \quad \xrightarrow{\tau \rightarrow 0} \quad H_{t+\tau} = \lim_{\tau \rightarrow 0^+} H_t - \tau \mathcal{L}H_t$$

$$= \lim_{\tau \rightarrow 0^+} H_t (I - \tau \mathcal{L})$$

$$= \lim_{\tau \rightarrow 0^+} H_t \left((1 - \tau)I + \tau D^{-\frac{1}{2}} A D^{-\frac{1}{2}} \right)$$

- Krylov Subspace: fixed propagation

$$\mathcal{K}_K(A, v) = \{v, Av, A^2v, \dots, A^{K-1}v\}.$$



Adaptive Selection: AdaptKry [WWW'24]

- Renormalize propagation:

$$P = (1 - \tau)I + \tau D^{-1/2} A D^{-1/2}$$

$$P_{\tau} = (\tau D + (1 - \tau)I)^{-1/2} (\tau A + (1 - \tau)I) (\tau D + (1 - \tau)I)^{-1/2}$$

$$P_{\tau} = D_{\tau}^{-\frac{1}{2}} A_{\tau} D_{\tau}^{-\frac{1}{2}}$$

- Adaptive Krylov Subspace: control graph message

$$\mathcal{K}_K(P_{\tau}, \mathbf{x}) = \{\mathbf{x}, P_{\tau} \mathbf{x}, P_{\tau}^2 \mathbf{x}, \dots, P_{\tau}^{K-1} \mathbf{x}\}$$

Spectral Embedding: Takeaways

Channel Combination

LD² [NeurIPS'23]

- 2-hop MP for global information
- Link spatial & spectral computation

TER [KDD'23]

S3GCL [ICML'24]

- Extending spectral embedding to edge-level task and contrastive learning

Adaptive Selection

UniFilter [ICML'24]

- Link spatial & spectral computation
- Additional bias for hop-wise heterophily

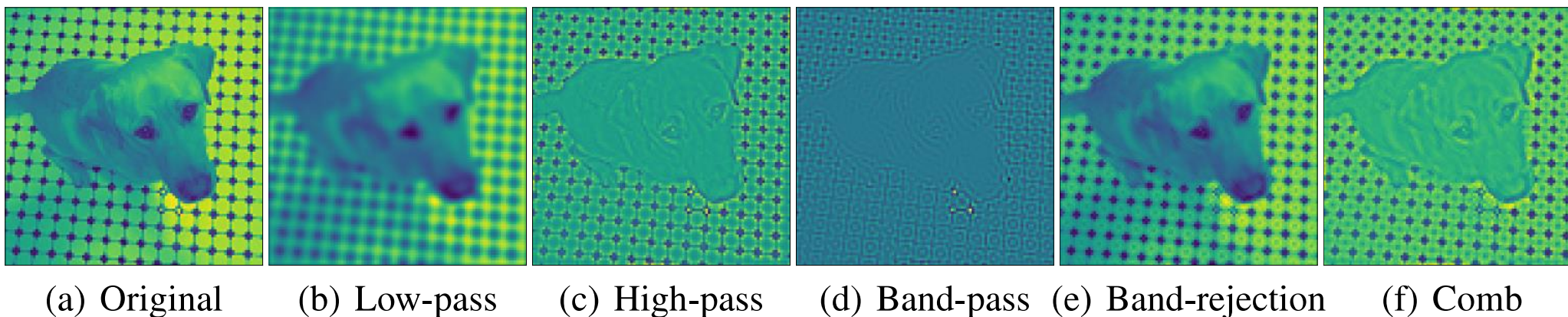
AdaptKry [WWW'24]

- MP as heat diffusion
- Adjust graph normalization

Spectral Embedding: Applications

Image Filtering

- Image as graph: better depict unstructured relationships, e.g. contextual connection of segmentations



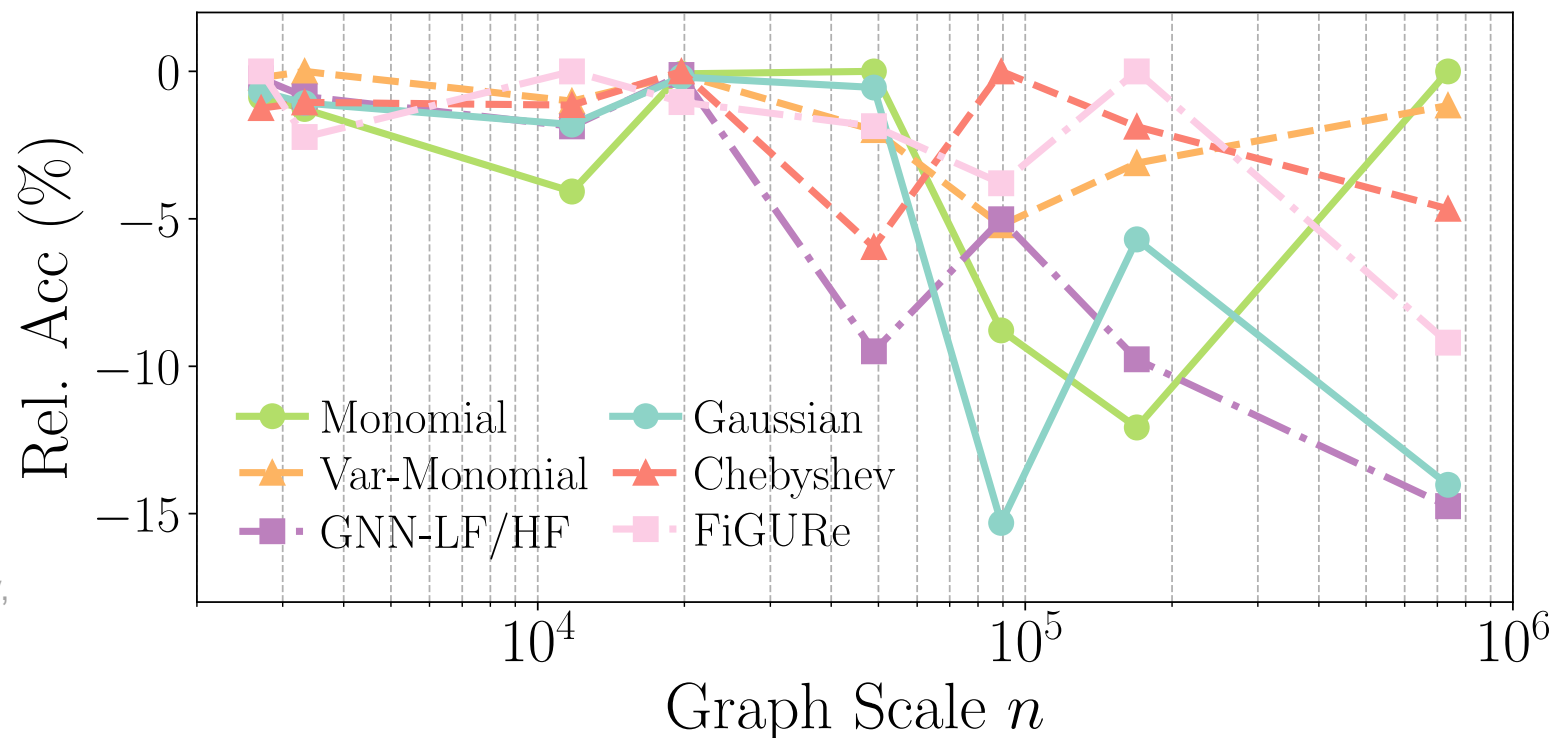
Spectral Embedding: Evaluation

Graph Propagation as Spectral Filter

- Global view is important for large-scale prediction
- Simple, invariable filters can also capture global view



N Liao et al, "A Comprehensive Benchmark on Spectral GNNs: The Impact on Efficiency, Memory, and Effectiveness", SIGMOD'26.



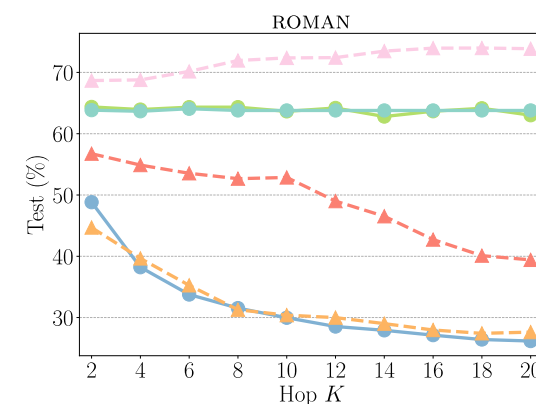
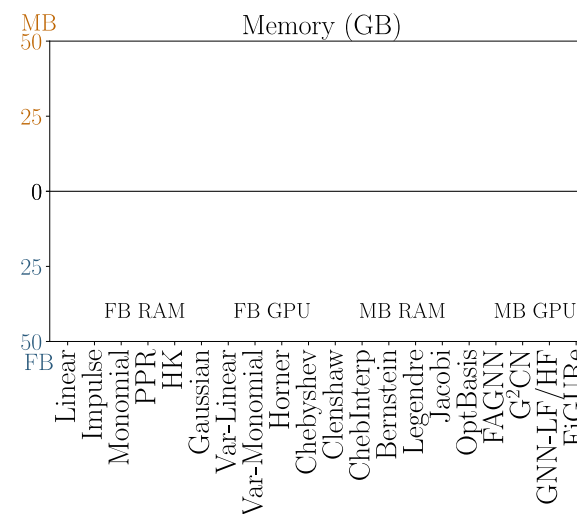
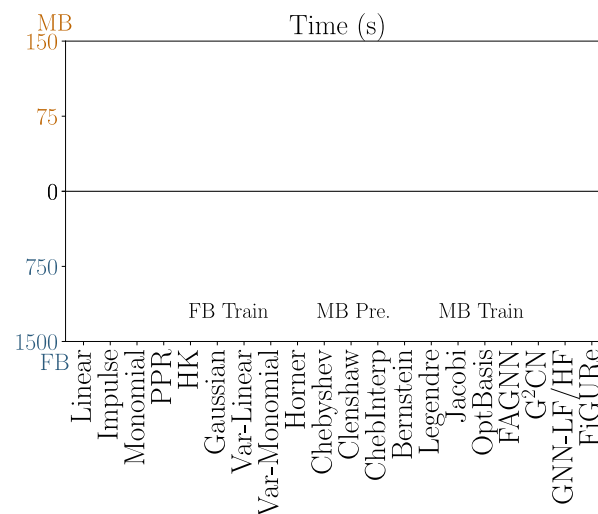
Spectral Embedding: Evaluation

Graph Propagation as Spectral Filter

- Comprehensive evaluation on: efficiency, memory footprint, propagation hops of graph filters, ...
- Insights on finding fast and effective graph filters



N Liao et al, "A Comprehensive Benchmark on Spectral GNNs: The Impact on Efficiency, Memory, and Effectiveness", SIGMOD'26.



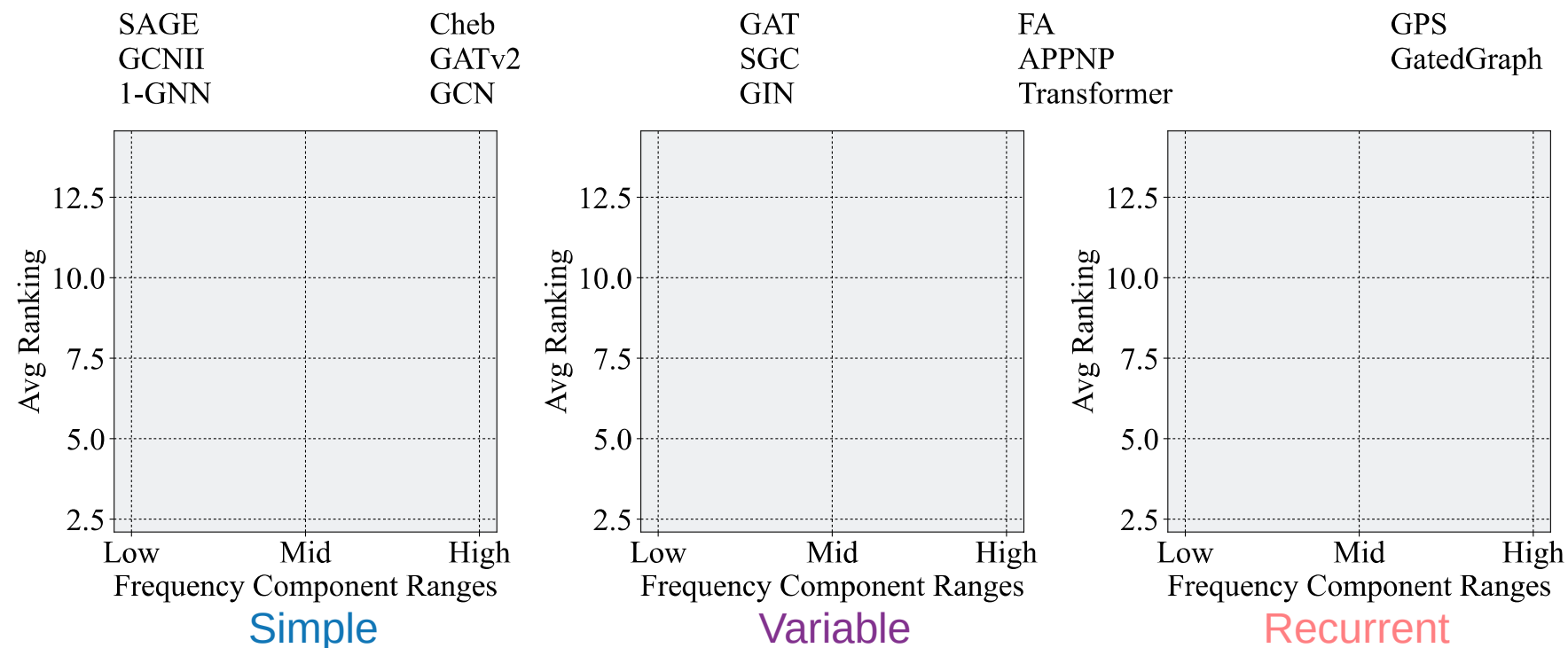
Spectral Embedding: Evaluation

Weight Transformation as Spectral Filter

- Weight transformation also affects spectral filtering



Y Dong et al, "Graph Neural Networks Are More Than Filters", *ICLR'25*.



Advances in Designing Scalable GNNs

Graph Simplification

Outline

Node-wise Similarity

Spectral Embedding

Graph Simplification

Layer-wise NIGCN [K Huang et al | WWW'23]

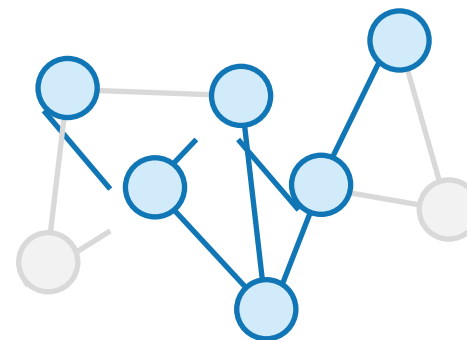
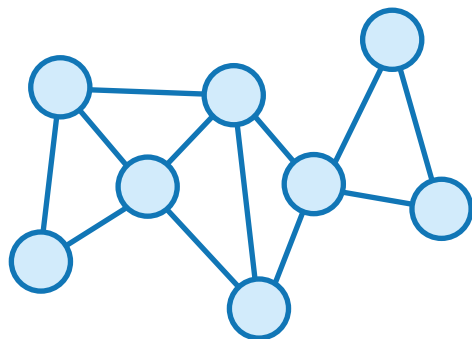
Pair-wise ATP [X Li et al | WWW'24]

Entry-wise Unifiows [N Liao et al | ICML'25]

Future Direction

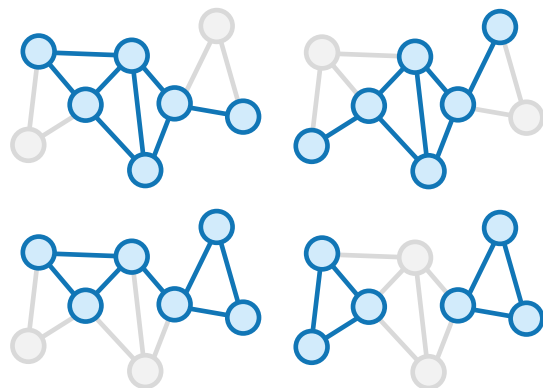
Graph Simplification: Strategies

Raw Graph



Sparsification

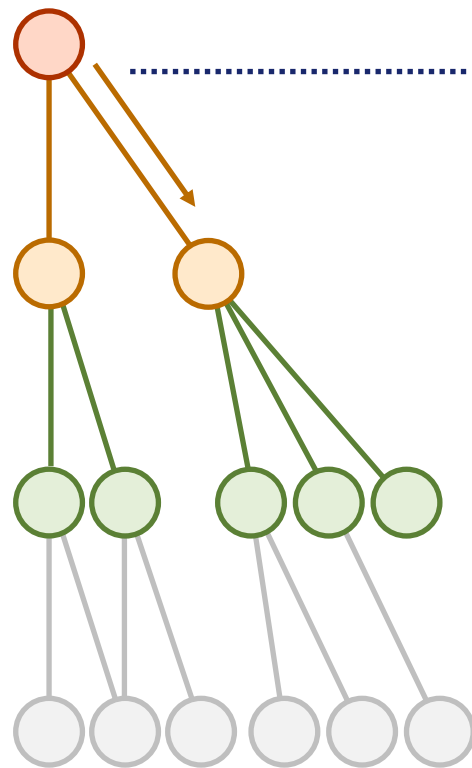
- Static graphs structure during learning
- Dedicated algorithms



Sampling

- Different graphs through training iterations
- Repetitive sampling overhead

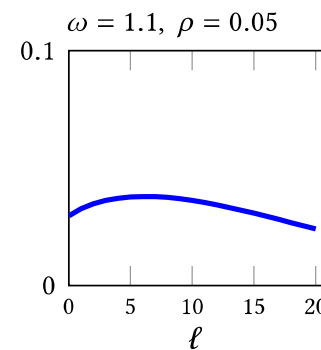
Layer-wise Sparsification: NIGCN [WWW'23]



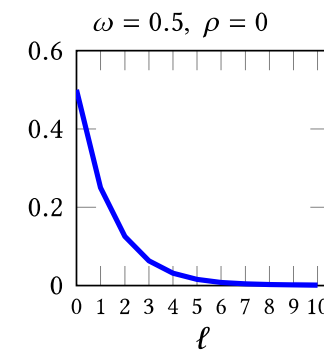
General Heat Diffusion

$$U(\omega, \rho, \ell) = \frac{\omega^\ell}{(\ell!)^\rho \cdot C}$$

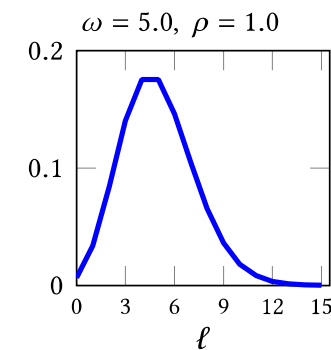
- Coverage: smooth (GCN), PPR (GBP), HKPR (AGP)
- Applicable to decoupled architecture



(a) Smooth



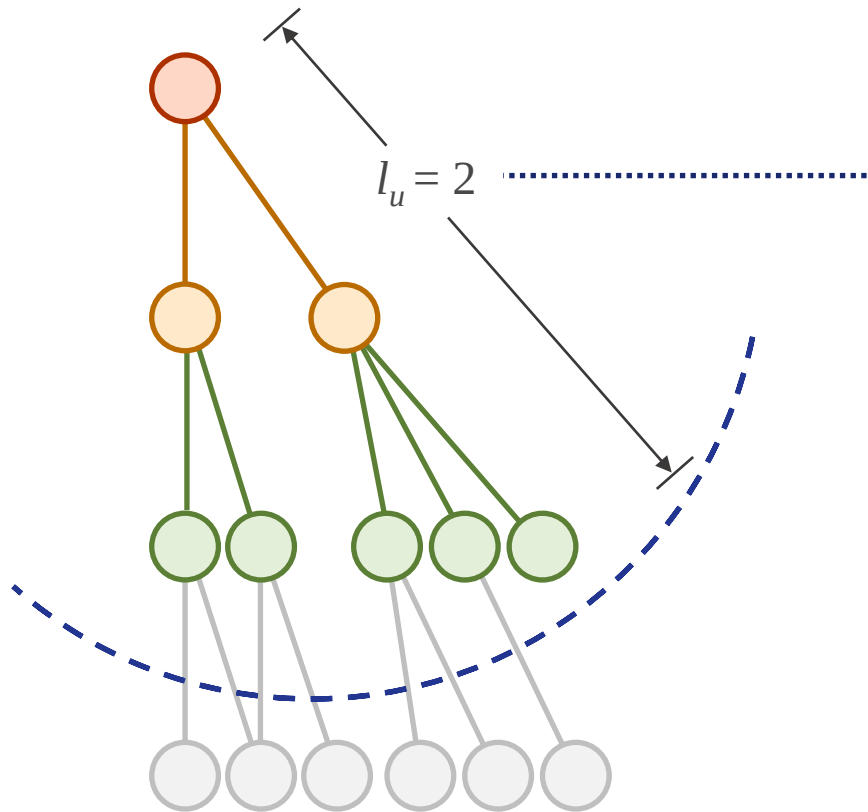
(b) PPR



(c) HKPR



Layer-wise Sparsification: NIGCN [WWW'23]



Node-wise Diffusion Distance

- Bounds computation complexity & approximation precision
- Nested in graph propagation

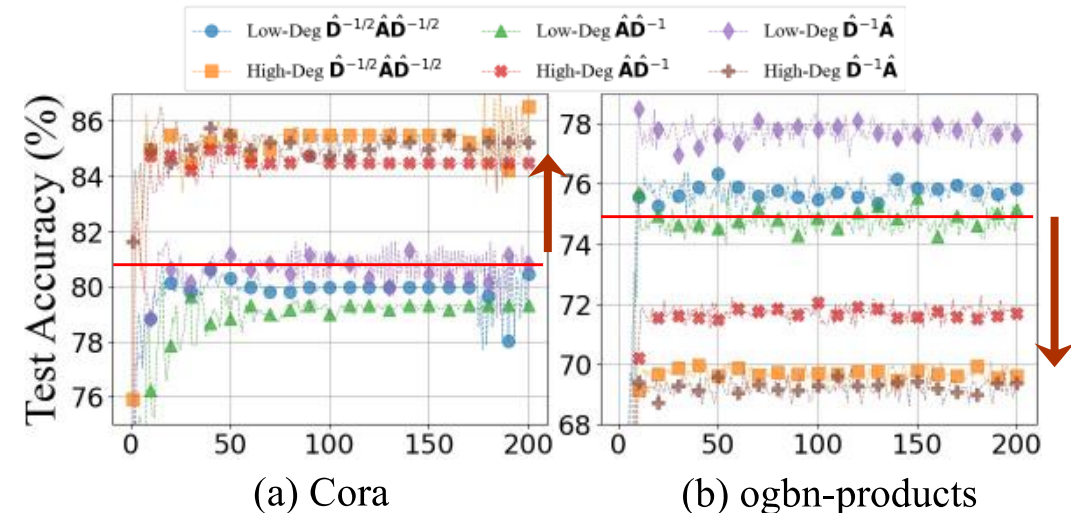
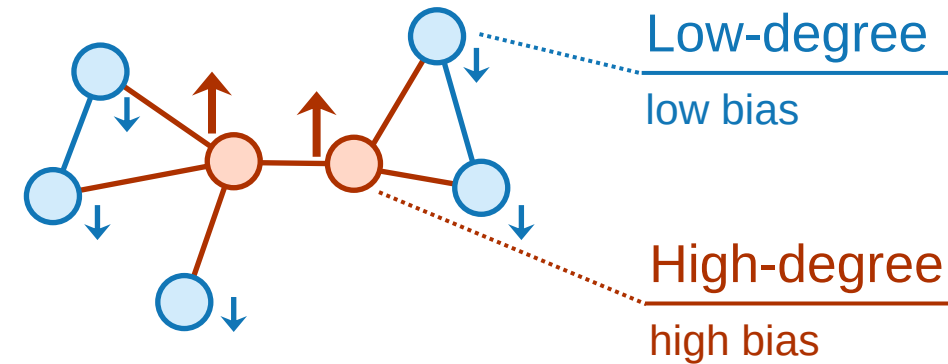
Pair-wise Sparsification: ATP [WWW'24]



Conventional Graph Propagation

$$\Pi = \sum_{i=0}^l w_i \cdot \left(\hat{\mathbf{D}}^{r-1} \hat{\mathbf{A}} \hat{\mathbf{D}}^{-r} \right)^i,$$

- Unified normalization for all nodes
- High-degree nodes produce biased accuracy





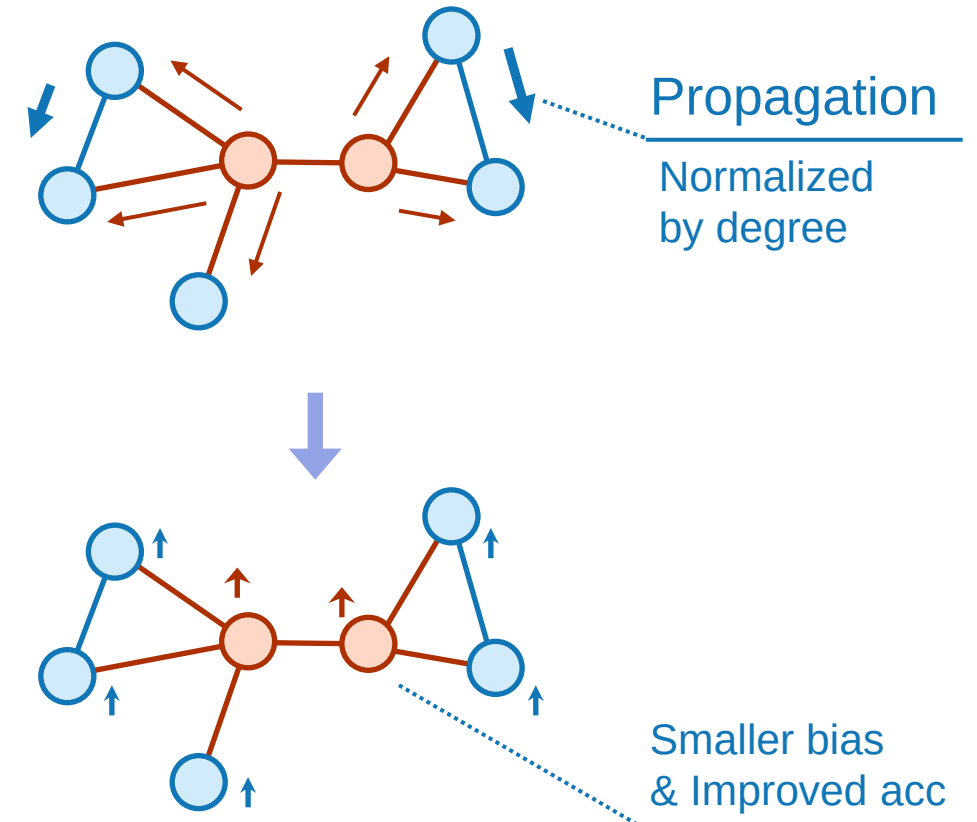
Pair-wise Sparsification: ATP [WWW'24]

Node-wise Propagation

$$\tilde{\Pi} = \sum_{i=0}^l w_i \cdot \left([\hat{\mathbf{D}}]^{\tilde{\mathbf{R}}-1} [\hat{\mathbf{A}}] [\hat{\mathbf{D}}]^{-\tilde{\mathbf{R}}} \right)^i$$

$$\mathbf{R}_0 = \text{Diag} \left(\frac{[d]_1}{n-1}, \dots, \frac{[d]_n}{n-1} \right).$$

- Specific normalization for each node
- Criteria based on degree, eigenvector, and connectivity



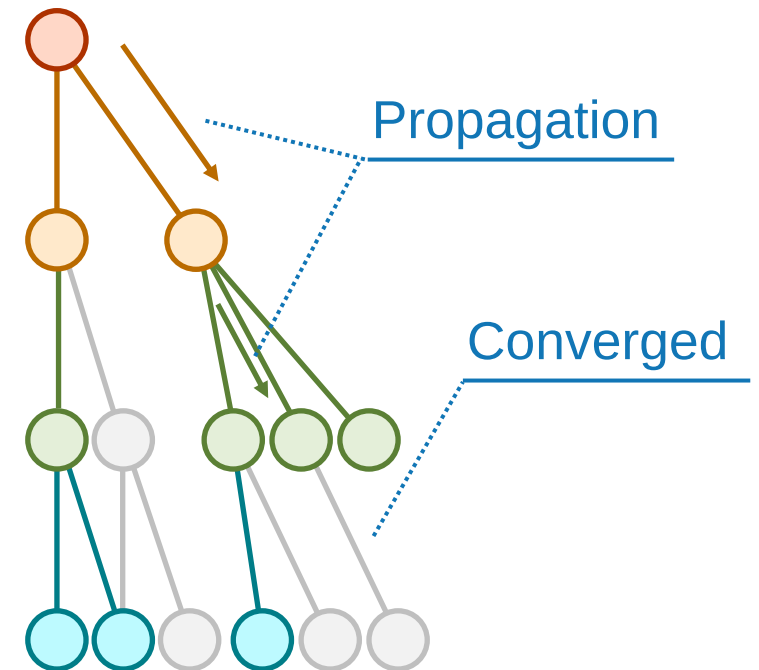


Pair-wise Sparsification: ATP [WWW'24]

Pair-wise Propagation Distance

$$\left| \left(\mathbf{P}^k e_i \right)_j - \tilde{\pi}_j \right| \leq \sqrt{\frac{\tilde{d}_j}{\tilde{d}_i}} \lambda_2^k,$$

- Precompute the converged value
- Threshold propagations closed to converged values



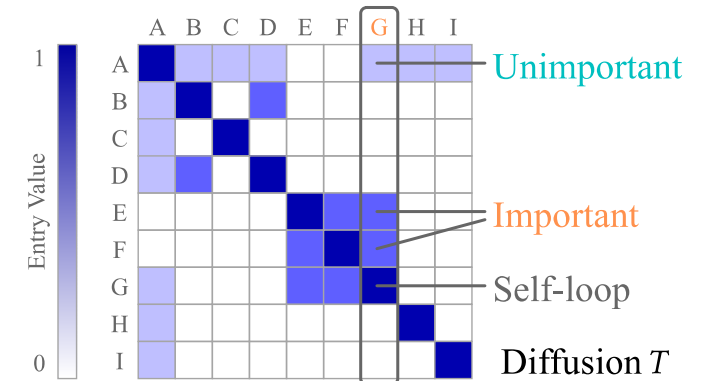
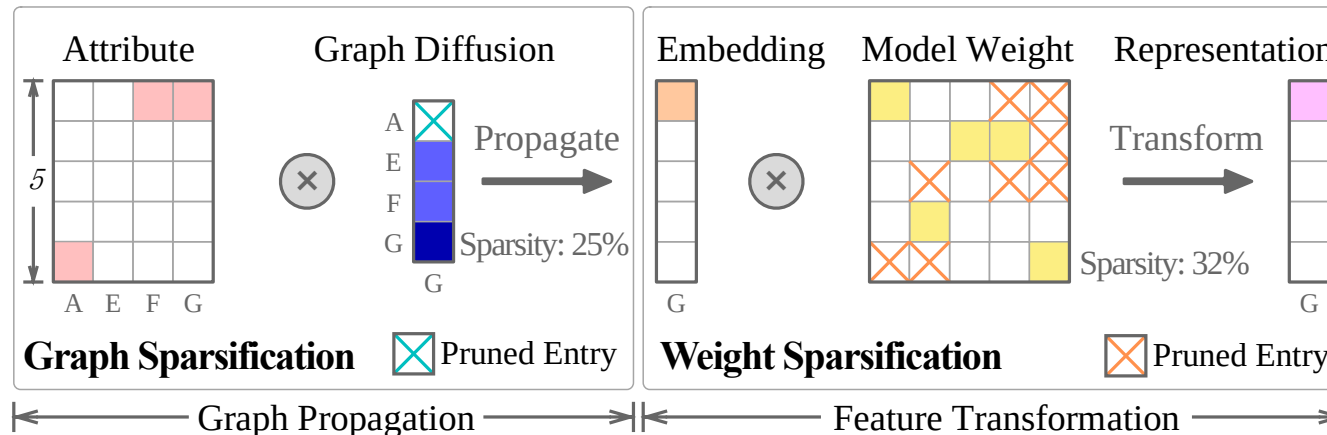
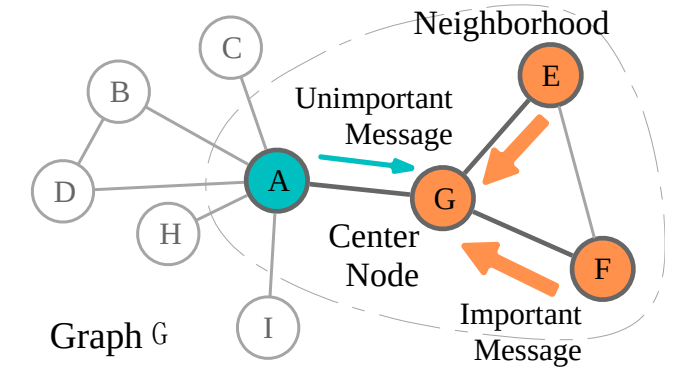


Sparsification: Unifews [ICML'25]

Entry-wise Sparsification

$$\underbrace{p^{(l+1)}[u]}_{\text{Embedding}} = \sum_{v \in \mathcal{N}(u)} \underbrace{T^{(l)}[u, v]}_{\substack{\text{Propagation} \\ \text{Subject to prune}}} \cdot \underbrace{p^{(l)}[v]}_{\text{Embedding}}$$

$$H^{(l+1)}[:, i] = \sum_{j=1}^f \underbrace{W^{(l)}[j, i]}_{\substack{\text{Weight} \\ \text{Subject to prune}}} \cdot P^{(l)}[:, j]$$





Sparsification: Unifews [ICML'25]

Sparsification $\Leftrightarrow$ Approximate Graph Smoothing

GNN Graph Smoothing:

$$\mathbf{p}^* = \arg \min_{\mathbf{p}} \mathcal{L}, \quad \mathcal{L} = \|\mathbf{p} - \mathbf{x}\|^2 + c \cdot \mathbf{p}^\top \mathbf{L} \mathbf{p},$$

Embedding
Attribute
Graph

Spectral Approximation:

$$\text{Approx Bound}$$

Error Guarantee:

(general)

$\Rightarrow$

$$q_a \delta_a \leq \epsilon \|\mathbf{p}\|$$

Sparsity
Prune Threshold

(scale-free graph)

$\Rightarrow$

$$\epsilon = O\left(\eta_a(1 - \eta_a)^{\frac{1}{1-\alpha}}\right)$$

Graph Distribution



Sparsification: Unifews [ICML'25]

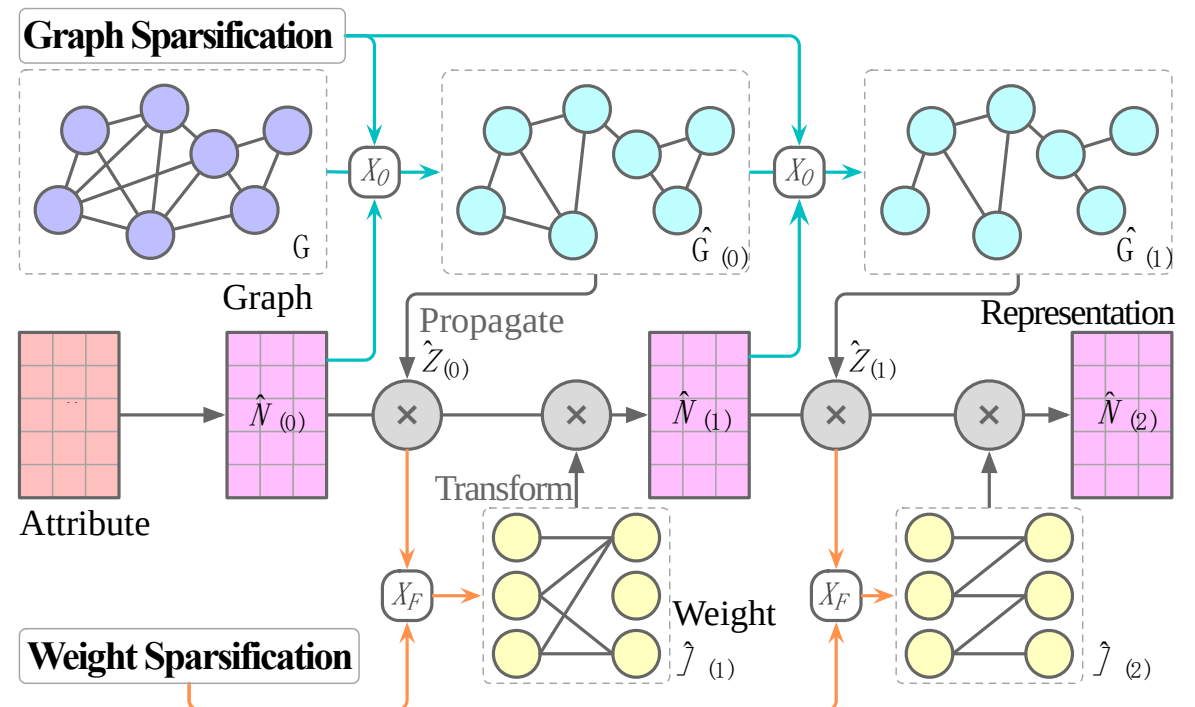
Layer-progressive Sparsification

Graph Sparsification:

$$\underbrace{\|\mathbf{p}_{(l+1)} - \hat{\mathbf{p}}_{(l+1)}\|}_{\text{Embedding}} \leq \underbrace{O(\epsilon) \cdot \|\mathbf{p}_{(l)}\| + bc\epsilon(1 + O(\epsilon))\|\mathbf{p}_{(l)}\|}_{\text{Approx Bound}}$$

w/ Weight Sparsification:

$$\underbrace{\|\hat{\mathbf{H}}_{(l+1)} - \mathbf{H}_{(l+1)}\|_F}_{\text{Representation}} \leq \underbrace{\ell_\sigma bc\epsilon\|\mathbf{H}_{(l)}\|_F\|\mathbf{W}\|_F + \ell_\sigma q_w \delta_w}_{\text{Network Distribution}}$$



Graph Simplification: Takeaways

Layer-wise

NIGCN [WWW'23]

- MP as heat diffusion
- Personalized MP hop for each node

Pair-wise

ATP [WWW'24]

- Generalized MP norm
- Personalized MP hop and norm for each node pair

Entry-wise

Unifews [ICML'25]

- MP as spectral smoothing
- Personalized MM for graph and weight entries

Advances in Designing Scalable GNNs

Conclusion & Future Directions

Outline

Node-wise Similarity

Spectral Embedding

Graph Simplification

Future Directions

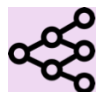
Summary

Large Model RAG | Graph Transformer

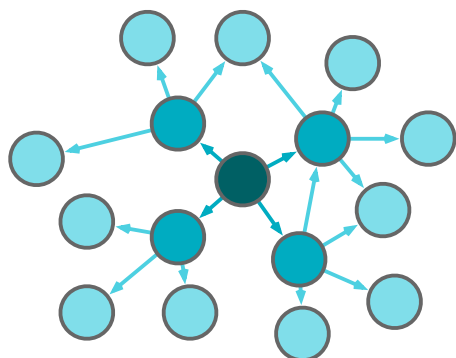
Data Variant Data Deficiency | Dynamic Data

System Co-design Device-specific | Distributed Training

Summary: Challenges & Solutions



Neighbor Explosion

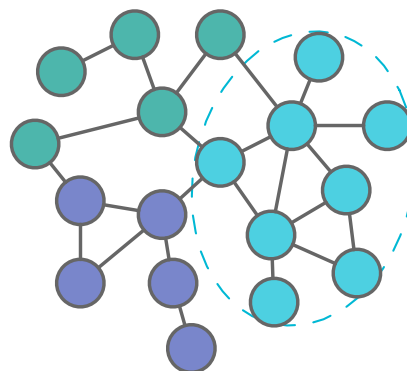


Node-wise Similarity

- PPR: faster message passing
- SimRank: global similarity
- Hub Labeling: core hub nodes



Limited Memory

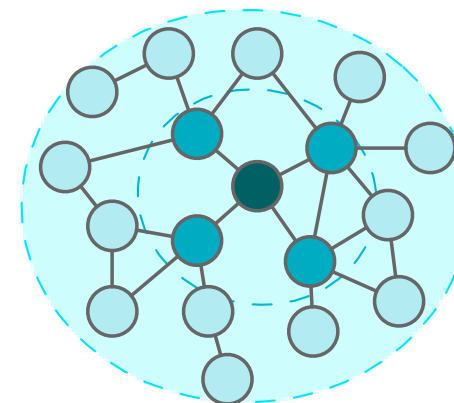


Subgraph Extraction

- Partition: reduce transmission
- Generation: tailored message passing
- Slicing: disk lookup



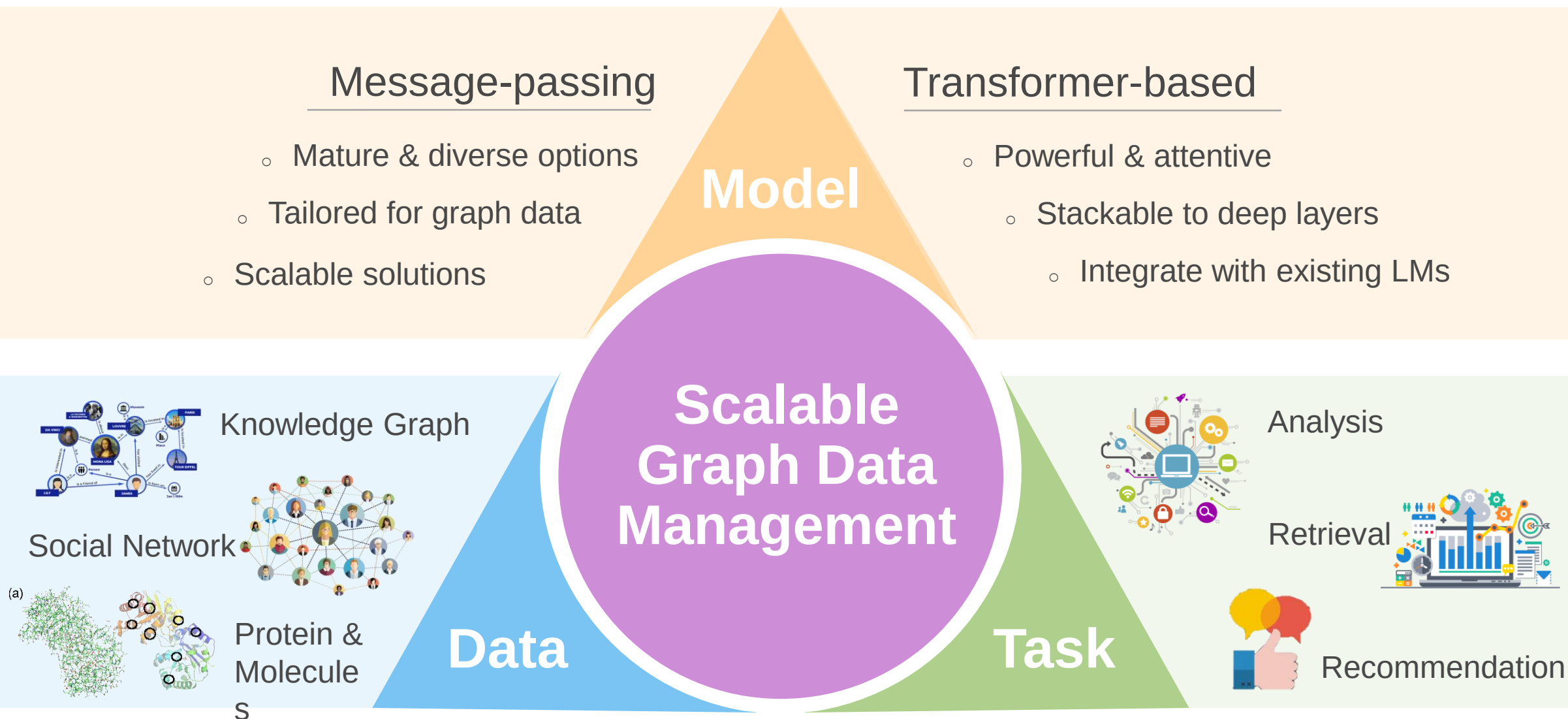
Multi-scale



Spectral Embedding

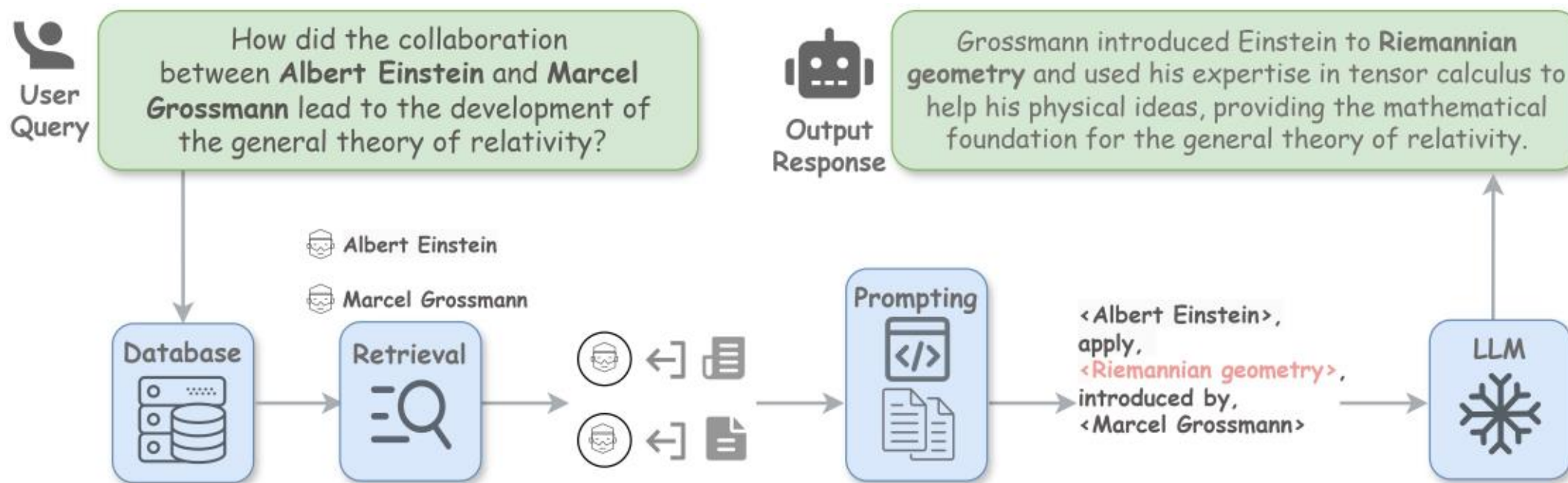
- Combination: local & global channels
- Selection: adaptive filtering

Graph + Large Models



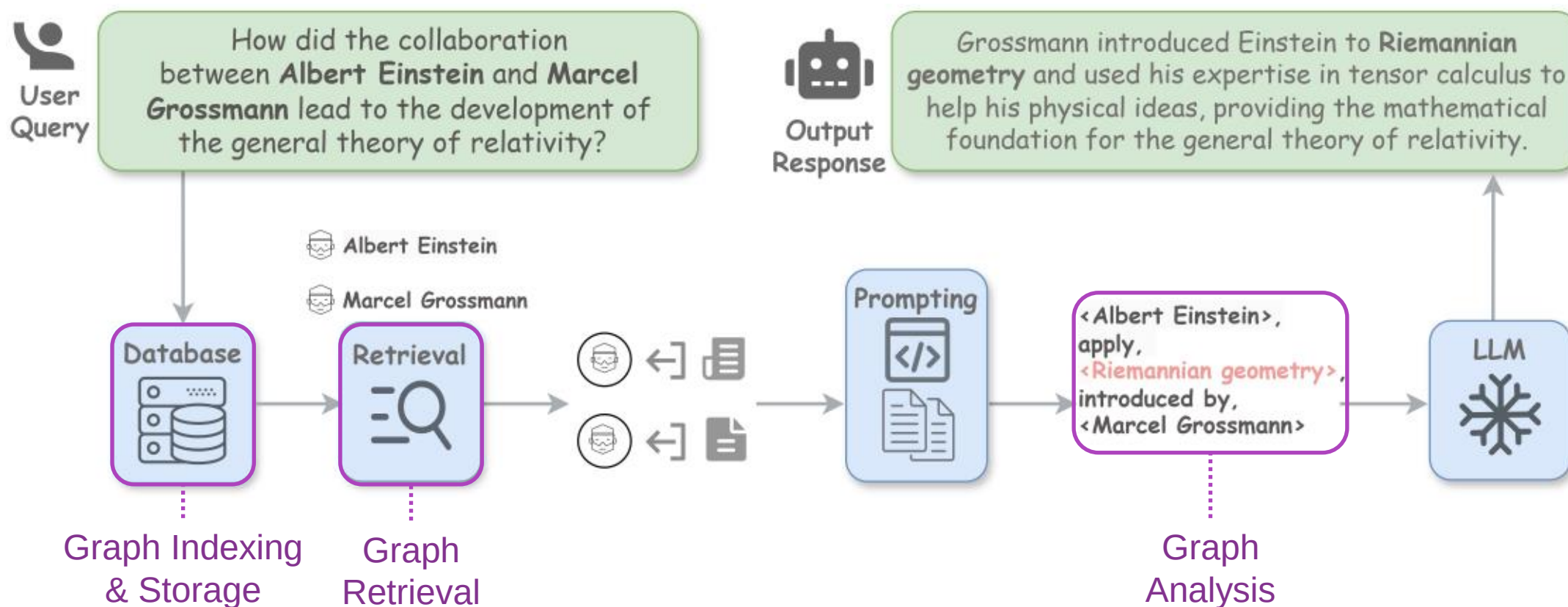
LM: Graph for Retrieval-Augmented Generation

Standard RAG



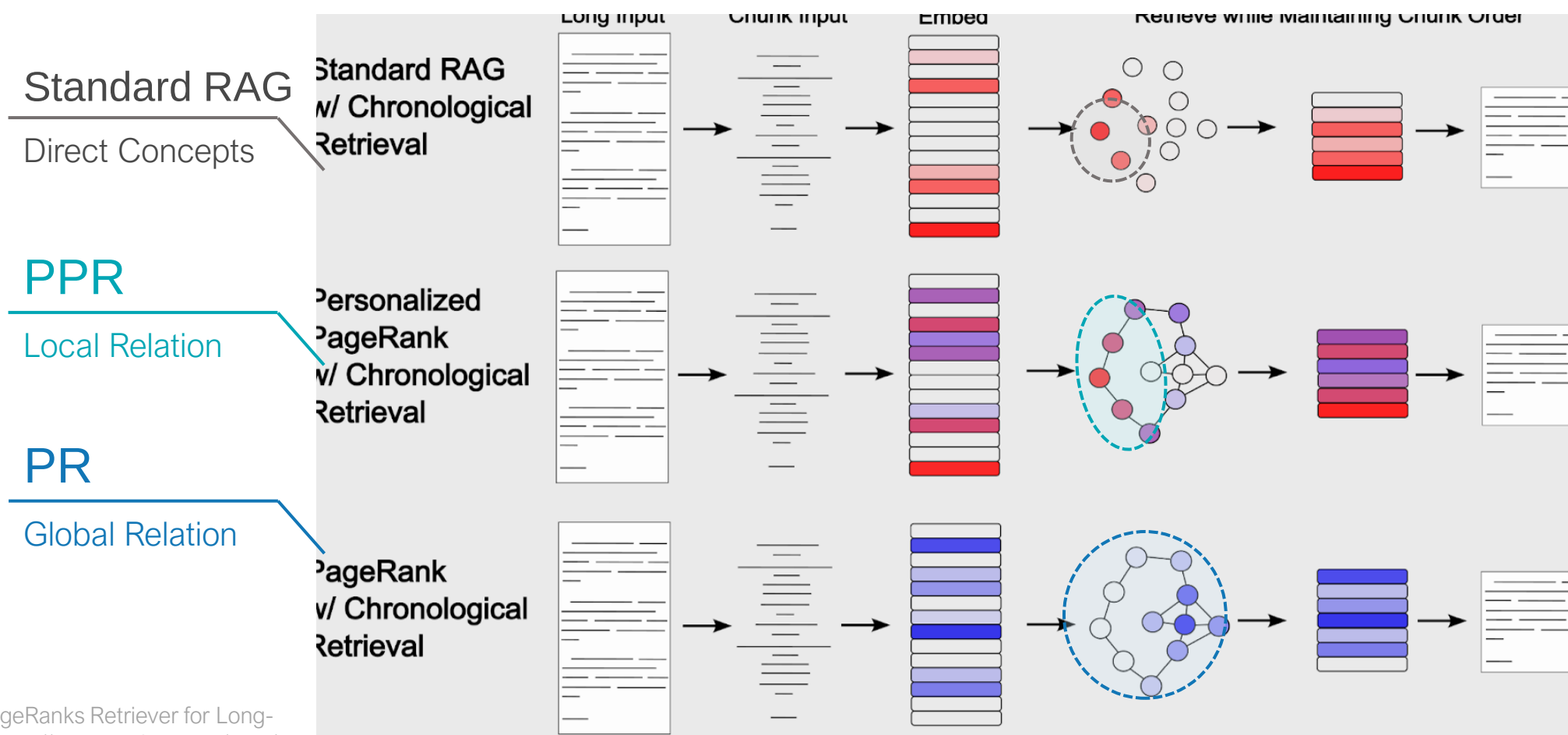
LM: Graph for Retrieval-Augmented Generation

Opportunities for Graph + RAG

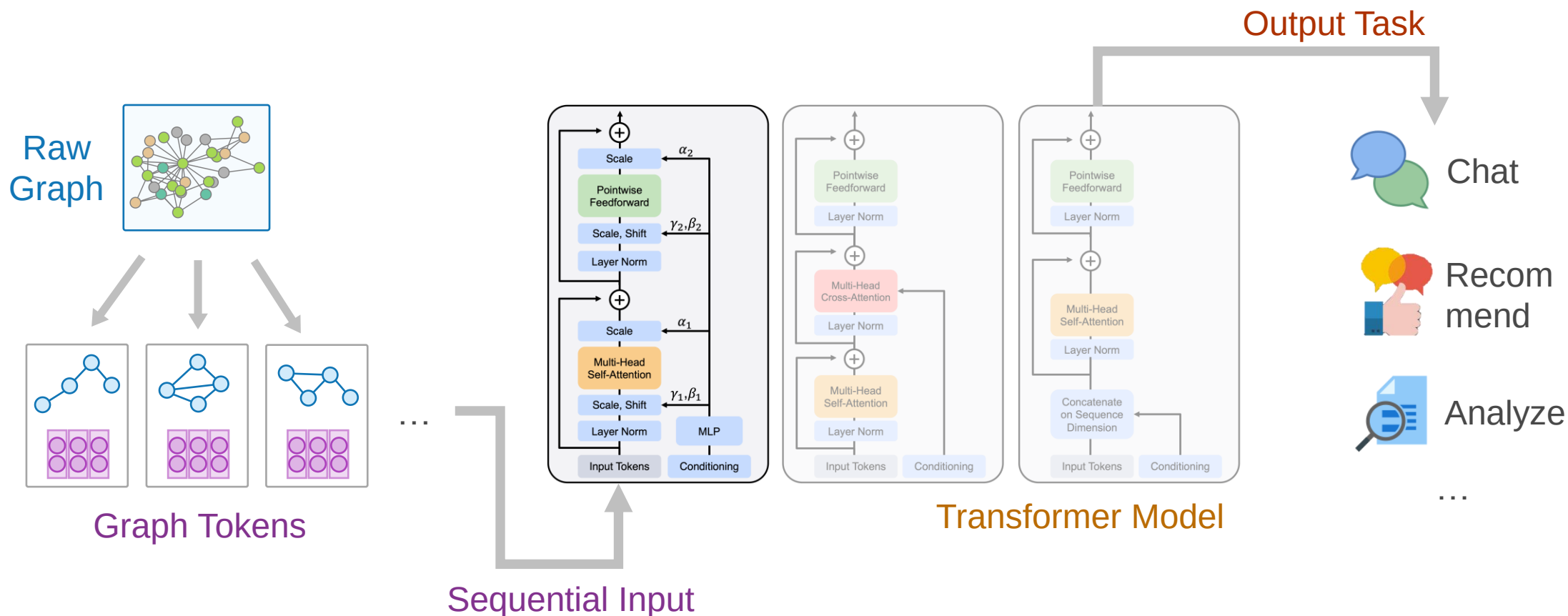


LM: Graph for Retrieval-Augmented Generation

Example: PageRank in RAG



LM: Scalable Graph Transformer



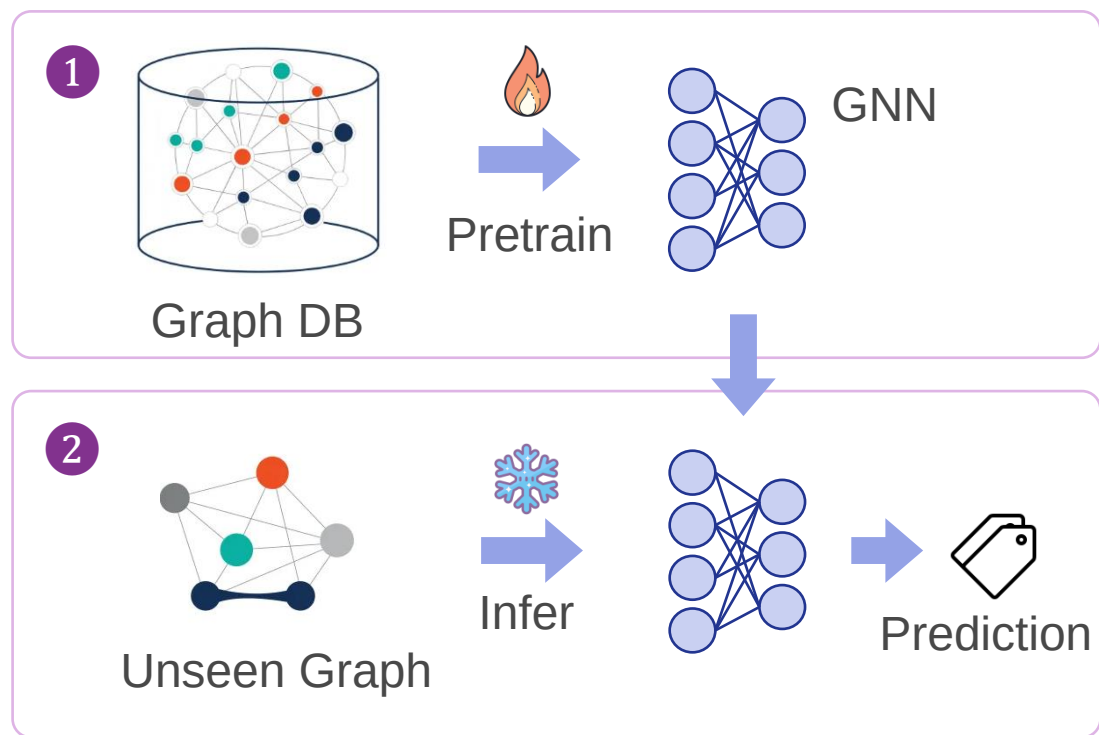
Efficient sequential graph representation?



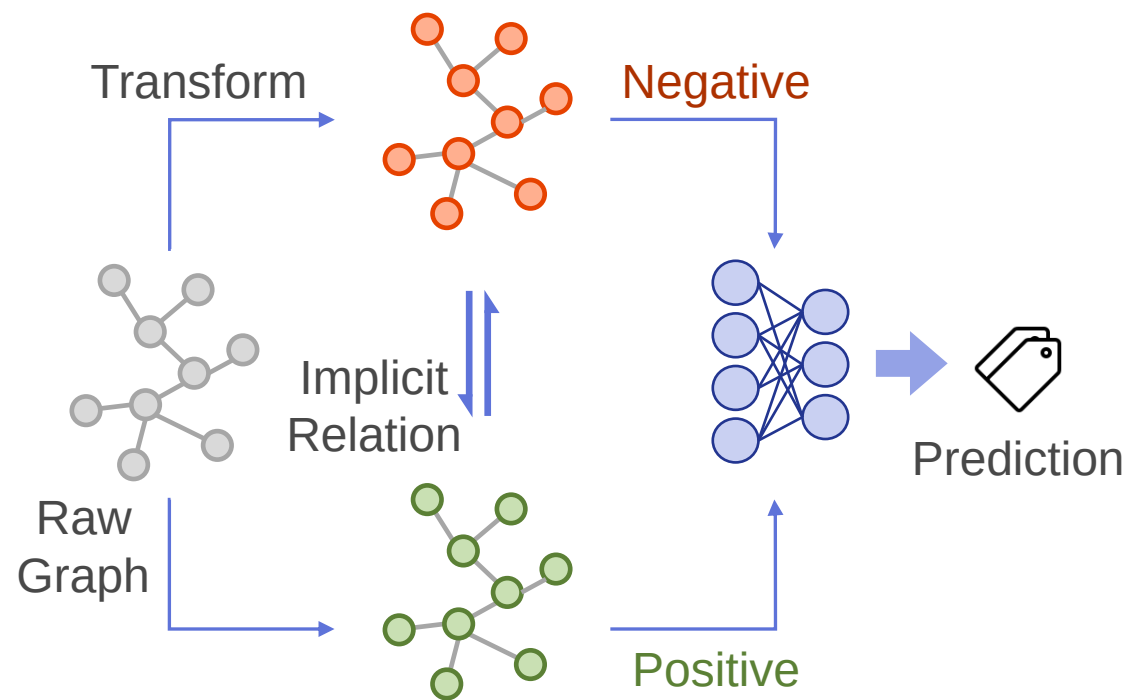
Efficient graph Transformer architecture?

Graph Variants: Data Deficiency

Pretrain



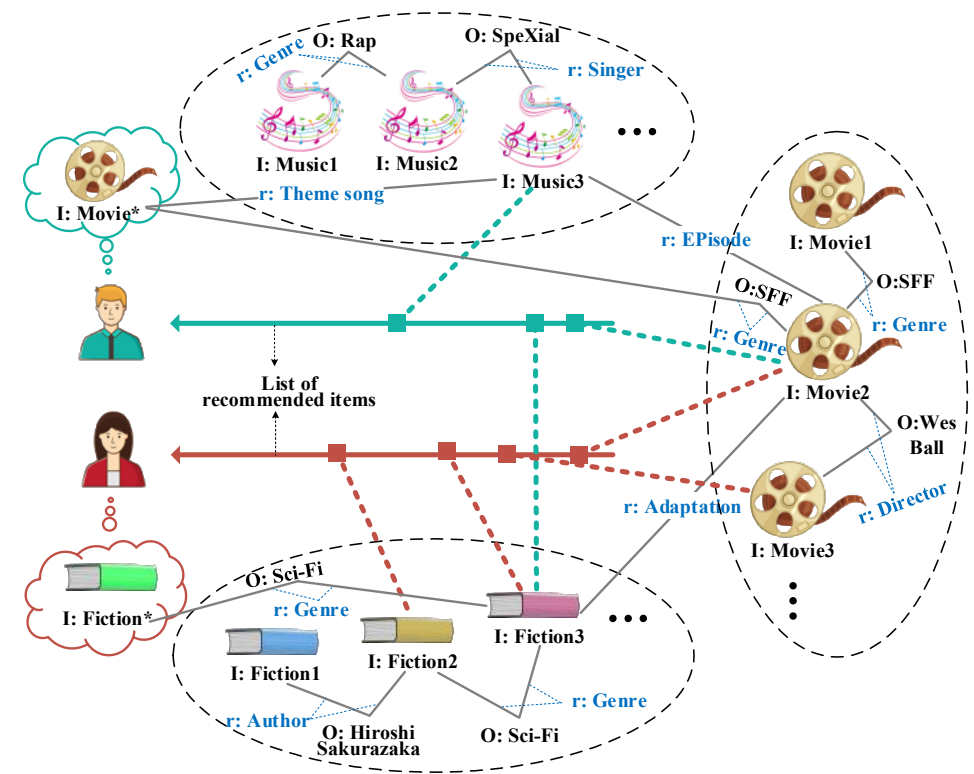
Contrastive



Graph Variants: Data Deficiency

Example: Knowledge Sharing in Recommendation

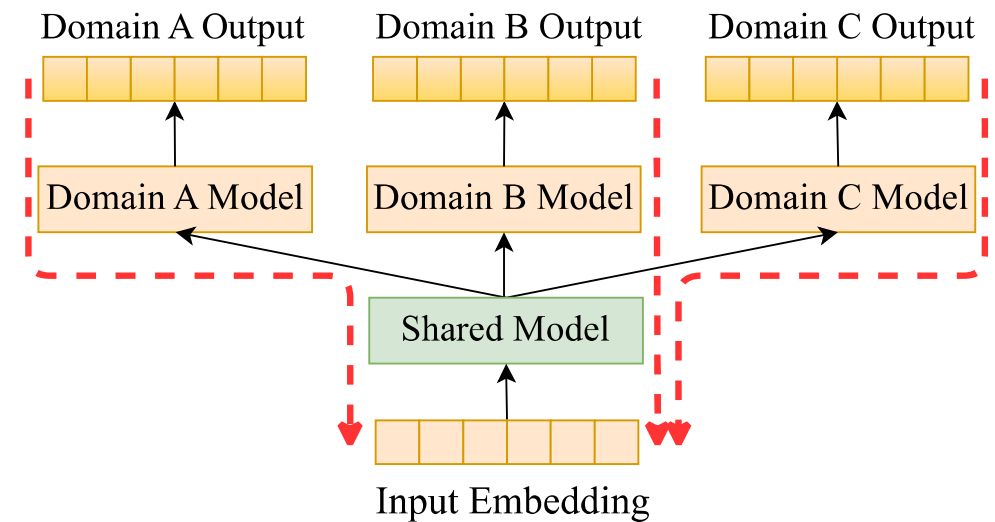
- Challenge: Data Deficiency
 - Overlap Sparsity: few overlap data between domains
 - Data Sparsity: limited data available in some domains
- Solution: Cross-domain learning by knowledge alignment



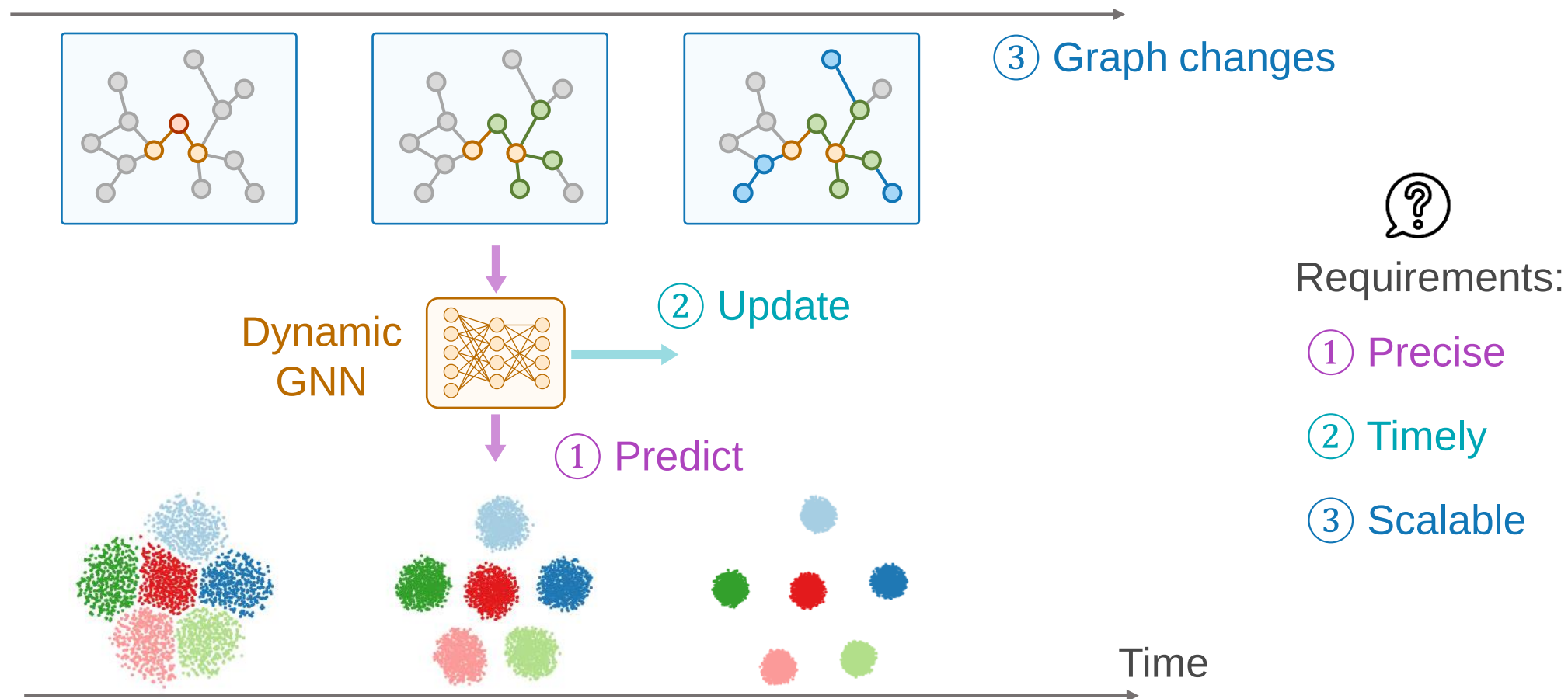
Graph Variants: Data Deficiency

Example: Knowledge Sharing in Recommendation

- Challenge: Model Scalability
 - Inter-domain: effectively learn domain-specific knowledge
 - Intra-domain: efficiently share knowledge without learning from scratch
- Solution: Model disentangling
 - Domain Model
 - Shared Model

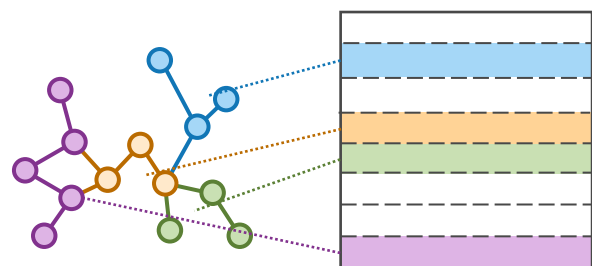


Graph Variants: Dynamic Data



Systems Co-design: Device-Specific

Graph Storage

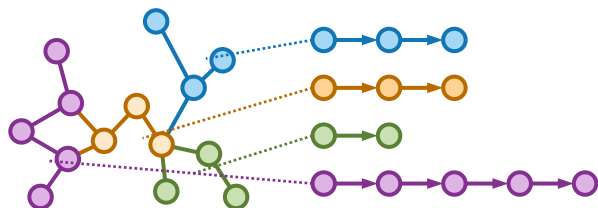


On-disk

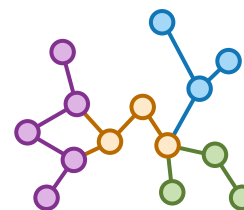
- Large storage
- Transmission overhead
- Update overhead

In-memory

- Random access
- Device loading
- Cache & buffer



Graph Computation

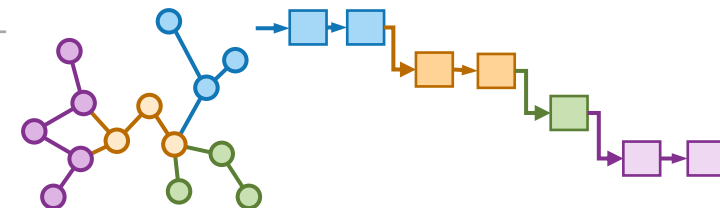


GPU-oriented

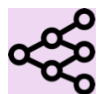
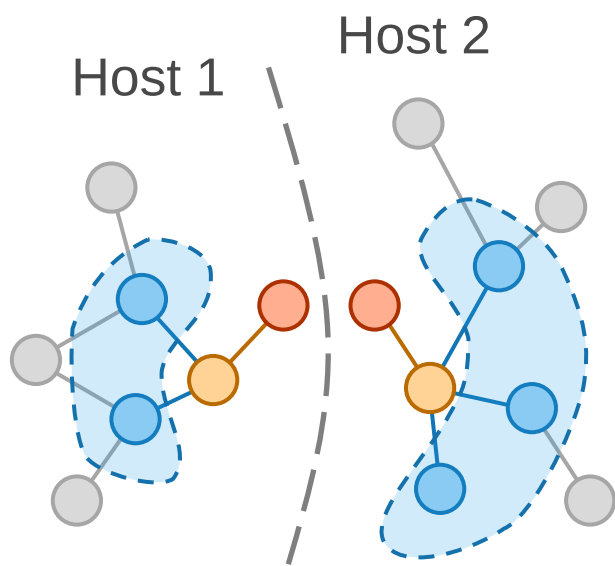
- Limited memory
- Batch processing
- Predefined operators

CPU-oriented

- Sequential/parallel
- Dedicated algorithms
- Sparse data structures

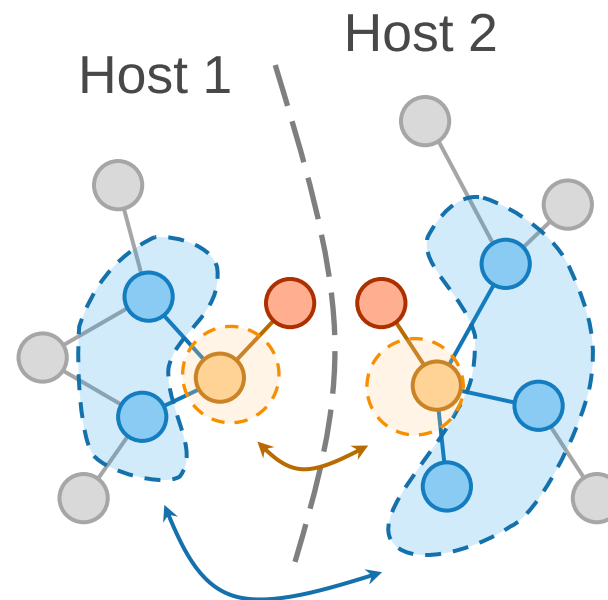


Systems Co-design: Distributed Training



Neighbor Explosion

- Neighbors affected by partitioning
- More communication for accessing neighbors



Multi-scale

- Local subgraph structure affected by partitioning
- Communication for global info

THANK YOU

Slides



香 港 大 學
THE UNIVERSITY OF HONG KONG

2025 International Conference on Management of Data
Berlin, Germany – June 22, 2025